Practical Multilayer Network Analysis with Py3plex

Release 1.1.4

Blaž Škrlj

Mar 21, 2026

CONTENTS

CHAPTER

ONE

FRONT MATTER

1.1 About This Book

Practical Multilayer Network Analysis with Py3plex is a research-oriented handbook for graduate students and applied network scientists. It provides both theoretical foundations and practical guidance for analyzing complex systems using multilayer network representations.

This book covers:

- Mathematical foundations of multilayer networks
- The py3plex library architecture and design philosophy
- Hands-on examples and workflows for real network analysis
- A SQL-like DSL for network queries
- Production deployment and reproducibility practices

DSL Feature Highlight

Py3plex includes a first-class DSL for querying networks with SQL-like syntax:

```
from py3plex.dsl import Q, L

# Find top influencers using builder API
result = (
    Q.nodes()
     .from_layers(L["social"])
     .where(degree__gt=5)
     .compute("betweenness_centrality")
     .order_by("-betweenness_centrality")
     .limit(10)
     .execute(network)
)
```

See **Part III** for complete DSL coverage!

1.2 Target Audience

This book is designed for:

- **Graduate students** in network science, computer science, and related fields
- **Applied researchers** working with complex relational data

- **Data scientists** analyzing social, biological, or infrastructure networks
- **Software engineers** building network analysis pipelines

1.2.1 Prerequisites

Readers should be familiar with:

- Python programming (intermediate level)
- Basic graph theory concepts
- Fundamental linear algebra

1.3 About the Software

py3plex (version 1.1.4) is an open-source Python library for multilayer and multiplex network analysis. It provides:

- Native support for multilayer network structures
- 17+ specialized algorithms for multilayer analysis
- A SQL-like query language (DSL) for network exploration
- NetworkX compatibility and interoperability
- High-performance I/O with Arrow/Parquet support

The library targets Python 3.8+ for runtime compatibility.

1.4 How to Use This Book

Part I establishes foundations: what multilayer networks are, when to use them, and how py3plex is designed.

Part II teaches practical usage: installation, data loading, visualization, and running core algorithms.

Part III presents the DSL query language, a major feature for expressing complex analyses concisely.

Part IV demonstrates real-world applications through detailed case studies.

Part V covers systems topics: testing, reproducibility, and the web GUI.

Appendices provide reference material, detailed configurations, and extensive validation examples that supplement the main text.

1.4.1 Code Examples

Examples in this book are intended to be runnable in the documented environment. Compatibility may vary across platforms and project revisions, so treat outputs as representative unless a section explicitly reports validated benchmark results. File paths reference the accompanying GitHub repository:

https://github.com/SkBlaz/py3plex

1.4.2 Conventions

- **Code blocks** use syntax highlighting and are self-contained
- **Mathematical notation** follows standard network science conventions
- **Feature status** is clearly marked:
  - **Stable** - Intended to be stable for the workflows described in this book

- **Experimental** - Functional but may change
- **Planned** - Future features (mentioned only briefly)

1.5 Acknowledgments

This work builds on contributions from the py3plex community and research collaborations in multilayer network analysis.

If you use py3plex in your research, please cite:

```
@Article{Skrlj2019,
  author={Skrlj, Blaz and Kralj, Jan and Lavrac, Nada},
  title={Py3plex toolkit for visualization and analysis of multilayer networks},
  journal={Applied Network Science},
  year={2019},
  volume={4},
  number={1},
  pages={94},
  doi={10.1007/s41109-019-0203-7}
}
```

1.6 License

Note

Version Information: This book edition is version 1.1.4 (2025), aligned with py3plex 1.1.4 and Python 3.8+ runtime support.

CHAPTER

TWO

INTRODUCTION & MOTIVATION

2.1 Why Multilayer Networks?

Real-world systems rarely consist of a single type of relationship. Consider:

- **Social networks:** People interact through friendship, collaboration, family ties, and online connections—each relationship type has different meanings and dynamics.
- **Transportation systems:** Cities connect via air, rail, and road networks. A disruption in one mode affects the others, and travelers make choices across modes.
- **Biological systems:** Proteins interact through physical binding, regulatory relationships, and metabolic pathways—each layer represents a different mechanism.
- **Information systems:** Software components depend on each other through API calls, data flows, and deployment relationships.

Traditional single-layer (monoplex) networks flatten these diverse relationships into a single graph, losing critical structural information. **Multilayer networks** preserve the distinct nature of different relationship types by organizing them into separate but interconnected layers.

2.1.1 The Cost of Flattening

When we flatten a multilayer network into a single graph, we lose:

1. **Relationship semantics:** A coauthor edge is fundamentally different from a Twitter follower edge, but flattening treats them identically.
2. **Community structure:** Groups that are well-separated in individual layers may appear spuriously connected when layers are combined.
3. **Node roles:** A person might be central in their professional network but peripheral in their hobby network—this context-dependent centrality disappears upon flattening.
4. **Cross-layer dynamics:** Information spreading from email to in-person meetings follows specific patterns that become invisible when layers are merged.

Example: Consider a researcher with 2 coauthors and 500 Twitter followers. In a flattened network, they have degree 502, suggesting high influence. However, their actual academic impact is modest (degree 2 in the coauthor layer). The Twitter followers don't collaborate on research papers. Multilayer analysis preserves this critical distinction between different types of influence.

2.2 What py3plex Enables

py3plex is a Python library designed for practical analysis of multilayer networks. It provides:

1. Expressive Data Structures

Native support for multilayer network representations that mirror how real systems are naturally structured:

```
from py3plex.core import multinet

# Create a multilayer network
network = multinet.multi_layer_network()

# Add edges across multiple layers
network.add_edges([
    ['Alice', 'coauthor', 'Bob', 'coauthor', 1],
    ['Alice', 'twitter', 'Carol', 'twitter', 1],
    ['Bob', 'coauthor', 'Dave', 'coauthor', 1],
], input_type="list")
```

2. Multilayer-Aware Algorithms

Algorithms specifically designed for multilayer structure:

- **Community detection** that respects layer boundaries while finding cross-layer modules
- **Centrality measures** that account for both intra-layer and inter-layer importance
- **Random walks** that can transition between layers
- **Dynamics models** (e.g., epidemic spreading) with layer-specific parameters

3. A Query Language for Networks

A SQL-like DSL (Domain-Specific Language) for expressing complex network analyses concisely:

DSL Example: Concise Network Analysis

Instead of writing loops and conditions, use declarative queries:

```
from py3plex.dsl import Q, L

# Find high-degree nodes in the coauthor layer
result = (
    Q.nodes()
     .from_layers(L["coauthor"])
     .where(degree__gt=5)
     .compute("betweenness_centrality", "pagerank")
     .order_by("-betweenness_centrality")
     .limit(10)
     .execute(network)
)

# Export for further analysis
result.to_pandas().to_csv("top_researchers.csv")
```

This replaces dozens of lines of manual filtering and iteration with a clear, composable query.

Key DSL features:

- SQL-like syntax familiar to data analysts
- Layer algebra: `L["social"] + L["work"] - L["bots"]`
- Type-safe Python builder API
- Export to pandas, NetworkX, Arrow, CSV, JSON
- EXPLAIN mode for query optimization

4. Visualization for Multilayer Networks

Visualizations that show layer structure and cross-layer connections:

```
# Visualize with layer separation
network.visualize_network(
    show=True,
    layout_style="force_directed_3d"
)
```

5. Research and Development Infrastructure

- **High-performance I/O** with Arrow/Parquet support for large networks
- **NetworkX compatibility** for interoperability with the broader Python ecosystem
- **Testing and validation** frameworks to ensure correctness
- **Docker containers** and CLI tools for deployment

2.3 Who Should Use This Book

This book is designed for:

Graduate Students and Researchers

If you're analyzing social, biological, or infrastructure networks in your research, this book provides both theoretical foundations and practical tools. You'll learn when multilayer modeling is appropriate and how to apply specialized algorithms correctly.

Data Scientists and Analysts

If you're working with complex relational data—user behavior across platforms, supply chain networks, knowledge graphs—py3plex offers a principled way to preserve and analyze structural nuances.

Software Engineers

If you're building network analysis pipelines, this book shows how to use py3plex's APIs, DSL, and deployment tools to create maintainable, efficient systems.

2.3.1 Prerequisites

This book assumes:

- **Intermediate Python** (classes, list comprehensions, basic NumPy)
- **Basic graph theory** (nodes, edges, paths, degree)
- **Fundamental linear algebra** (matrices, matrix multiplication)

You don't need prior multilayer network experience—Part I builds from fundamentals.

2.4 Real-World Applications

Multilayer network analysis has proven valuable in numerous domains:

Biological Networks

- Protein-protein interactions across multiple tissues or conditions
- Gene regulatory networks with transcription, translation, and metabolic layers
- Brain connectivity with structural and functional layers

Social Networks

- User behavior across multiple social platforms
- Organizational networks with formal and informal relationships
- Citation networks with papers, authors, and venues

Infrastructure Networks

- Transportation with air, rail, and road connections
- Power grids with generation, transmission, and distribution layers
- Communication networks with physical and logical layers

Knowledge and Information

- Heterogeneous information networks (authors-papers-venues)
- Semantic networks with multiple relationship types
- Code dependency graphs with API, data, and deployment layers

Each of these applications benefits from multilayer analysis by capturing the true structure of the system rather than forcing it into a single-layer representation.

2.5 How This Book Is Organized

Part I (Chapters 1-3): Foundations
Introduces multilayer networks, defines key concepts, and explains py3plex's design philosophy.

Part II (Chapters 4-7): Working with Py3plex
Covers installation, data loading, visualization, and core algorithms—everything needed for basic usage.

Part III (Chapters 8-11): Advanced Analysis and the DSL
Deep dive into the query language, a major feature for expressing complex analyses concisely.

Part IV (Chapters 12-14): Case Studies
Complete, reproducible analyses of real datasets across different domains.

Part V (Chapters 15-17): Systems and Deployment
Testing, reproducibility, and production deployment including the web GUI.

Appendices (A-E): Reference Material
Detailed configurations, validation scripts, error handling, and API reference.

Each chapter includes:

- **Conceptual explanations** with mathematical foundations where needed
- **Code examples** intended to be runnable in the documented environment
- **Practical tips** based on real usage patterns

- **References** for deeper study

2.6 Closing Perspective

The rest of this book is organized around one practical question: *how do you turn multilayer semantics into defensible analytical decisions?* The next chapters move from definitions to implementation details, with explicit caveats where methods are approximate, data dependent, or computationally expensive.

2.7 Further Reading

- Kivelä, M., et al. (2014). "Multilayer networks." *Journal of Complex Networks* 2(3): 203-271. [Comprehensive review of multilayer network theory]
- Boccaletti, S., et al. (2014). "The structure and dynamics of multilayer networks." *Physics Reports* 544(1): 1-122. [Physics-focused perspective]
- Bianconi, G. (2018). *Multilayer Networks: Structure and Function.* Oxford University Press. [Textbook treatment]

CHAPTER

THREE

MULTILAYER NETWORK BASICS

This chapter establishes the formal foundations of multilayer networks. We define key concepts, introduce mathematical notation, and show how different types of multilayer networks relate to each other.

3.1 Formal Definitions

3.1.1 Node-Layer Pairs

A **multilayer network** $\mathcal{M}$ consists of:

- A set of **nodes** $V = \{v_1, v_2, ..., v_N\}$
- A set of **layers** $L = \{\alpha, \beta, \gamma, ...\}$
- A set of **node-layer pairs** $V_M = V \times L$
- **Intra-layer edges** $E_\alpha \subseteq V \times V$ within each layer α
- **Inter-layer edges** $E_C \subseteq V_M \times V_M$ connecting node-layer pairs

The **node-layer pair** (v, α) is the fundamental unit: it represents node v in the context of layer α.

3.1.2 Supra-Adjacency Matrix

The **supra-adjacency matrix** $\mathbf{A}$ is a block matrix that encodes the full multilayer structure:

$$\mathbf{A} = \begin{pmatrix} \mathbf{A}_{\alpha\alpha} & \mathbf{A}_{\alpha\beta} & \cdots \\ \mathbf{A}_{\beta\alpha} & \mathbf{A}_{\beta\beta} & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}$$

Where:

- **Diagonal blocks** $\mathbf{A}_{\alpha\alpha}$ are the adjacency matrices of individual layers
- **Off-diagonal blocks** $\mathbf{A}_{\alpha\beta}$ encode inter-layer connections

For a multiplex network with N nodes and L layers, $\mathbf{A}$ is an $(N \times L) \times (N \times L)$ matrix.

Example: A 3-node, 2-layer network:

$$\mathbf{A} = \begin{pmatrix} \mathbf{A}_{\text{friends}} & \omega\mathbf{I} \\ \omega\mathbf{I} & \mathbf{A}_{\text{colleagues}} \end{pmatrix}$$

where ω is the **inter-layer coupling strength** and $\mathbf{I}$ is the identity matrix (representing identity edges: each node connects to itself across layers).

3.2 Types of Multilayer Networks

3.2.1 Multiplex Networks

Definition: A multiplex network has the same node set in all layers, with different edge sets per layer.

$$V_\alpha = V_\beta = \cdots = V \quad \text{for all layers}$$

Characteristics:

- **Same entities** appear in all layers
- **Different relationship types** per layer
- **Strong inter-layer coupling** (typically $\omega = 1.0$ for identity edges)

Example: Social Multiplex Network

```
from py3plex.core import multinet

# Create multiplex network
network = multinet.multi_layer_network(network_type="multiplex")

# Same people, different relationship types
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1],
    ['Bob', 'friends', 'Carol', 'friends', 1],
    ['Alice', 'colleagues', 'Bob', 'colleagues', 1],
    ['Bob', 'colleagues', 'Dave', 'colleagues', 1],
], input_type="list")

# Verify structure
print(f"Layers: {network.get_layers()}")
print(f"Nodes per layer: {network.get_number_of_nodes_per_layer()}")
```

Real-world applications:

- Social networks across platforms (Facebook, Twitter, LinkedIn)
- Transportation modes (air, rail, road)
- Communication channels (email, phone, chat)
- Biological interactions (genetic, protein, metabolic)

3.2.2 Heterogeneous Information Networks

Definition: A network with different node types, where edges connect specific node type pairs.

Characteristics:

- **Different node types** per layer (authors, papers, venues)
- **Type-specific edges** (author-paper, paper-venue)
- **No inter-layer coupling** (nodes don't repeat across layers)

Example: Academic Network

```
network = multinet.multi_layer_network()

# Different node types
network.add_edges([
    ['Alice', 'authors', 'Paper1', 'papers', 1],
    ['Bob', 'authors', 'Paper1', 'papers', 1],
    ['Paper1', 'papers', 'ICML', 'venues', 1],
    ['Paper2', 'papers', 'ICML', 'venues', 1],
], input_type="list")
```

This creates a **tripartite** network with three node types: authors, papers, and venues.

Real-world applications:

- Academic networks (authors, papers, venues, keywords)
- E-commerce (users, products, sellers, categories)
- Biomedical (drugs, diseases, targets, pathways)
- Knowledge graphs (entities, relations, concepts)

3.2.3 Temporal Networks

Definition: Networks that evolve over time, represented as time-sliced layers.

Characteristics:

- **Time windows as layers** (2020, 2021, 2022, ...)
- **Node presence varies** across time slices
- **Temporal edges** connect adjacent time slices

Example: Temporal Social Network

```
network = multinet.multi_layer_network()

# Time-sliced layers
network.add_edges([
    ['Alice', '2020', 'Bob', '2020', 1],
    ['Alice', '2021', 'Bob', '2021', 1],
    ['Bob', '2021', 'Carol', '2021', 1],
    ['Alice', '2022', 'Carol', '2022', 1],
], input_type="list")
```

Real-world applications:

- Communication patterns over time
- Disease spread through populations
- Financial transaction networks
- Collaboration evolution

3.2.4 Interdependent Networks

Definition: Multiple networks where the function of nodes in one layer depends on nodes in other layers.

Characteristics:

- **Dependency edges** encode functional relationships
- **Cascading failures** possible across layers
- **Critical infrastructure** applications

Example: Power grid (layer 1) depends on communication network (layer 2). If communication fails, power grid control fails.

Real-world applications:

- Infrastructure systems (power, water, communication)
- Supply chain networks
- Cyber-physical systems
- Ecological networks (species, habitats, resources)

3.3 Inter-Layer Coupling

The **coupling strength** ω controls how strongly layers are connected.

3.3.1 Identity Coupling

The most common form connects each node to itself across layers:

$$E_C = \{((v, \alpha), (v, \beta)) : v \in V, \alpha \neq \beta \in L\}$$

with edge weight ω.

Choice of ω:

- $\omega = 1.0$ — Layers are equally important, nodes fully correspond
- $\omega < 1.0$ — Layers are loosely coupled, favor intra-layer paths
- $\omega > 1.0$ — Inter-layer transitions are encouraged

Example:

```
network = multinet.multi_layer_network()

# Add intra-layer edges
network.add_edges([
    ['A', 'layer1', 'B', 'layer1', 1],
    ['A', 'layer2', 'C', 'layer2', 1],
], input_type="list")

# Add inter-layer coupling (identity edges)
network.add_edges([
    ['A', 'layer1', 'A', 'layer2', 0.5],  # omega = 0.5
], input_type="list")
```

3.3.2 General Coupling

More complex couplings allow different nodes to connect across layers:

$$E_C = \{((v, \alpha), (w, \beta)) : (v, w) \in C_{\alpha\beta}\}$$

where $C_{\alpha\beta}$ defines which nodes couple between layers α and β.

3.4 When to Use Multilayer Networks

Use multilayer modeling when:

1. **Multiple relationship types have distinct semantics**

   Example: Friendship vs. professional collaboration—these networks have different properties and dynamics.

2. **Node roles vary by context**

   Example: A person central in academic network may be peripheral in social media network.

3. **Cross-layer interactions matter**

   Example: Information spreads from Twitter to traditional media—this cross-layer flow is meaningful.

4. **Temporal evolution is important**

   Example: Community structure evolves over time—time-sliced layers preserve this evolution.

5. **System-level resilience depends on layer dependencies**

   Example: Power grid failure affects communication, which affects emergency response.

3.5 Choosing a Modeling Approach

3.5.1 Decision Tree

```
1. Do relationships have fundamentally different types?
   YES → Use layers (multiplex or heterogeneous)
   NO  → Use edge weights or attributes

2. Are the same entities present in all layers?
   YES → Multiplex network (strong coupling)
   NO  → Heterogeneous network (weak/no coupling)

3. Does time evolution matter?
   YES → Temporal layers (time-sliced)
   NO  → Static multilayer network

4. Are there functional dependencies between layers?
   YES → Interdependent network (dependency edges)
   NO  → Standard multiplex/heterogeneous
```

3.5.2 Common Mistakes

Over-aggregation
: Combining layers that should stay separate (e.g., merging email and meeting networks loses information about mode transitions).

Under-aggregation
: Creating too many sparse layers when edge weights would suffice (e.g., separate layer for each email vs. email timestamp as attribute).

Wrong coupling strength
: Using $\omega = 1.0$ when nodes don't fully correspond, or $\omega \to 0$ when they do.

Mismatched identifiers
: Using different IDs for the same entity across layers breaks multiplex structure.

3.6 Why Flattening Fails

Flattening (aggregating all layers into a single graph) loses critical information:

1. Community Structure

Multilayer communities may have:

- **Core in one layer, periphery in another**
- **Cross-layer bridges** that appear as spurious intra-layer connections when flattened

Example: Academic collaboration (dense group) + Twitter followers (different dense group). Flattening creates false bridges.

2. Centrality

A node's importance depends on layer:

- **Structural centrality** in one layer (high degree)
- **Functional centrality** in another (betweenness for information flow)

Flattening mixes these distinct roles.

3. Path Structure

Meaningful paths often cross layers:

- Email $\to$ meeting $\to$ collaboration
- Twitter mention $\to$ news coverage $\to$ policy change

Flattening hides these cross-layer transitions.

4. Dynamics

Disease spreading, information diffusion, and cascading failures all depend on layer-specific parameters and inter-layer transitions. Flattening cannot capture:

- **Layer-specific transmission rates**
- **Mode-switching dynamics**
- **Asymmetric cross-layer effects**

3.7 Key Terminology

Intra-layer edges
: Edges within a single layer, connecting (v, α) to (w, α).

Inter-layer edges
: Edges between layers, connecting (v, α) to (w, β) with $\alpha \neq \beta$.

Node-layer pair
The tuple (v, α) representing node v in layer α—the fundamental unit of a multilayer network.

Supra-adjacency matrix
The block matrix $\mathbf{A}$ encoding all intra-layer and inter-layer edges.

Coupling strength
The weight ω of inter-layer edges, controlling how strongly layers interact.

Multiplex network
Multilayer network with the same nodes in all layers.

Heterogeneous information network
Multilayer network with different node types per layer.

3.8 Working with Supra-Adjacency Matrices in Py3plex

```python
from py3plex.core import multinet, random_generators

# Generate random multiplex network
network = random_generators.random_multiplex_ER(
    n=100,
    l=3,
    p=0.05,
    directed=False
)

# Get supra-adjacency matrix (sparse format)
supra_matrix = network.get_supra_adjacency_matrix()

print(f"Matrix shape: {supra_matrix.shape}")  # (300, 300) for 100 nodes replicated across 3 layers
print(f"Matrix density: {supra_matrix.nnz / (supra_matrix.shape[0] ** 2):.4f}")

# Visualize matrix structure
network.visualize_matrix({"display": True})
```

The supra-adjacency matrix enables tensor-based algorithms and linear algebra operations on the full multilayer structure.

Use `random_multiplex_ER` when you want the classic $N \times L$ node-replica construction with identity coupling between layers. By contrast, `random_multilayer_ER` creates a general multilayer network in which each base node is assigned to a single layer, so the supra-adjacency shape is determined by the node-layer pairs that actually exist.

3.9 Common Multilayer Semantic Pitfalls

Understanding multilayer network semantics is crucial for correct analysis. Here are the most common misunderstandings and how to avoid them:

3.9.1 Pitfall 1: Node Replicas vs Physical Nodes

The Issue:

In multilayer networks, a single physical entity (e.g., person "Alice") appears as multiple node replicas across layers:

- Physical node: `"Alice"` (the person)
- Node replicas: `("Alice", "social")`, `("Alice", "work")`, `("Alice", "family")`

Most multilayer operations work on **node replicas**, not physical nodes.

Common Mistake:

```
#  Wrong: Assuming node count = unique physical nodes
result = Q.nodes().execute(multilayer_net)
n_people = result.count  # This counts REPLICAS, not people!

# If network has 100 people across 3 layers:
# result.count = 300 (not 100!)
```

Correct Approaches:

```
#  Count physical nodes explicitly
result = Q.nodes().execute(multilayer_net)
physical_nodes = set(node[0] for node in result.items)
n_people = len(physical_nodes)

#  Work per-layer (avoids the ambiguity)
result = (
    Q.nodes()
     .per_layer()
     .execute(multilayer_net)
)
# Now each group is one layer

#  Filter to single layer
result = (
    Q.nodes()
     .from_layers(L["social"])
     .execute(multilayer_net)
)
# Now result.count = physical nodes in social layer
```

When py3plex warns:

The library issues `MultilayerSemanticWarning` when operations likely confuse replicas with physical nodes:

```
from py3plex.dsl.warnings import suppress_warnings

# With warning (recommended for learning)
result = Q.nodes().execute(multilayer_net)
# Warning: Node replicas vs physical nodes...

# Suppress if you understand the semantics
with suppress_warnings("node_replica_confusion"):
    result = Q.nodes().execute(multilayer_net)
```

3.9.2 Pitfall 2: Degree Meaning Ambiguity

The Issue:

"Degree" has three different meanings in multilayer networks:

1. **Intra-layer degree**: Edges within the same layer
2. **Inter-layer degree**: Edges to other layers (coupling)

3. **Aggregate degree**: Total degree (intra + inter)

By default, py3plex computes **aggregate degree** (most common use case).

Common Mistake:

```
#  Ambiguous: Which degree?
result = Q.nodes().compute("degree").execute(multilayer_net)
# This computes AGGREGATE degree (most connections)
```

Correct Approaches:

```
#  Explicit intra-layer degree
result = (
    Q.nodes()
     .per_layer()  # Group by layer
     .compute("degree")  # Now it's per-layer degree
     .execute(multilayer_net)
)

#  Degree in specific layer
result = (
    Q.nodes()
     .from_layers(L["social"])
     .compute("degree")
     .execute(multilayer_net)
)

#  Aggregate degree (explicit)
result = (
    Q.nodes()
     .compute("degree")  # Aggregate by default
     .execute(multilayer_net)
)
# Comment: "Computing aggregate degree (intra + inter)"
```

When py3plex warns:

```
from py3plex.dsl.warnings import warn_degree_ambiguity

# Library warns when degree computation might be ambiguous
result = Q.nodes().compute("degree").execute(multilayer_net)
# Warning: Degree ambiguity - specify intra-layer, inter-layer, or aggregate
```

3.9.3 Pitfall 3: Coverage Filters Removing Expected Nodes

The Issue:

Coverage filters (`mode="all"`, `mode="at_least"`) remove nodes not meeting the criterion **across all groups**. This can remove many nodes unexpectedly.

Common Mistake:

```
#  Unexpected filtering
result = (
    Q.nodes()
```

```
    .from_layers(L["*"])  # 5 layers
    .per_layer()
    .top_k(10, "degree")  # Top 10 per layer
    .end_grouping()
    .coverage(mode="all")  # Keep only nodes in ALL 5 layers
    .execute(multilayer_net)
)
# Might return 0-2 nodes if few nodes are top-10 in ALL layers!
```

Correct Approaches:

```
#  Use less strict mode
result = (
    Q.nodes()
     .per_layer()
     .top_k(10, "degree")
     .end_grouping()
     .coverage(mode="at_least", k=3)  # In at least 3 layers
     .execute(multilayer_net)
)

#  Use fraction-based
result = (
    Q.nodes()
     .per_layer()
     .top_k(10, "degree")
     .end_grouping()
     .coverage(mode="fraction", p=0.6)  # In at least 60% of layers
     .execute(multilayer_net)
)

#  Check counts before filtering
result_before = (
    Q.nodes()
     .per_layer()
     .top_k(10, "degree")
     .end_grouping()
     .execute(multilayer_net)
)
print(f"Before coverage: {result_before.count} nodes")

result_after = (
    Q.nodes()
     .per_layer()
     .top_k(10, "degree")
     .end_grouping()
     .coverage(mode="all")
     .execute(multilayer_net)
)
print(f"After coverage: {result_after.count} nodes")
```

When py3plex warns:

```
# Library warns if coverage filter removes >50% of items
result = (
    Q.nodes()
     .per_layer()
     .top_k(10, "degree")
     .end_grouping()
     .coverage(mode="all")
     .execute(multilayer_net)
)
# Warning: Coverage filter removed 95% of items (190/200)
#          mode='all' is STRICT - consider mode='any' or mode='at_least'
```

3.9.4 Pitfall 4: Global vs Per-Layer Community Detection

The Issue:

Community detection on multilayer networks can operate:

1. **Globally**: Find communities spanning multiple layers
2. **Per-layer**: Find independent communities in each layer

The choice dramatically affects results and interpretation.

Common Mistake:

```
#  Unclear intent
result = (
    Q.nodes()
     .community(method="leiden")
     .execute(multilayer_net)
)
# This finds GLOBAL communities (span layers)
# Is this what you wanted?
```

Correct Approaches:

```
#  Explicit global communities
result = (
    Q.nodes()
     .community(method="leiden", omega=0.8)  # Explicit coupling
     .execute(multilayer_net)
)
# Comment: "Finding global communities with omega=0.8 coupling"

#  Per-layer communities
result = (
    Q.nodes()
     .per_layer()
     .community(method="leiden")
     .end_grouping()
     .execute(multilayer_net)
)
# Now each layer has independent communities

#  Layer-specific community
```

```
result = (
    Q.nodes()
     .from_layers(L["social"])
     .community(method="leiden")
     .execute(multilayer_net)
)
```

When py3plex warns:

```
# Library warns about global community detection
result = (
    Q.nodes()
     .community(method="leiden")  # No omega specified
     .execute(multilayer_net)
)
# Warning: Global community detection on 5-layer network
#          Communities will span layers. If you want per-layer:
#          Use .per_layer().community(...).end_grouping()
```

3.9.5 Best Practices Summary

1. Be explicit about layer scope:

```
# Unclear
Q.nodes().compute("degree")

# Clear
Q.nodes().per_layer().compute("degree")  # Per-layer analysis
Q.nodes().from_layers(L["social"]).compute("degree")  # Single layer
Q.nodes().compute("degree")  # with comment: "aggregate degree"
```

2. Understand replica semantics:

- Most operations return **node replicas** (node, layer) tuples
- To get physical nodes: extract first element of tuples
- Use `per_layer()` when layer structure matters

3. Choose coverage modes carefully:

- `mode="all"`: Very strict (intersection)
- `mode="any"`: Very permissive (union)
- `mode="at_least"` or `mode="fraction"`: Balanced middle ground

4. Set coupling parameters explicitly:

- Community detection: Always specify `omega` when using global mode
- Dynamics: Always specify inter-layer transition rates
- Centrality: Document whether you want per-layer or aggregate

5. Use warnings as learning tools:

```
# Let warnings guide you during development
result = Q.nodes().compute("degree").execute(net)
```

```
# Read warning, understand issue, refactor

# Suppress warnings in production when semantics are clear
from py3plex.dsl.warnings import suppress_warnings
with suppress_warnings("degree_ambiguity"):
    result = Q.nodes().compute("degree").execute(net)
```

3.10 Summary

Multilayer networks formalize systems with multiple relationship types by:

- Using **node-layer pairs** as the fundamental unit
- Encoding structure in the **supra-adjacency matrix**
- Supporting various types: **multiplex**, **heterogeneous**, **temporal**, **interdependent**
- Controlling interaction via **coupling strength** ω

Key takeaways:

1. **Multiplex** = same nodes, different edge types
2. **Heterogeneous** = different node types per layer
3. **Temporal** = time-sliced layers
4. **Coupling** = inter-layer connections (identity edges most common)
5. **Flattening loses information** — use multilayer analysis when layer structure matters

The next chapter explains how py3plex implements these concepts and why its design choices support efficient, correct multilayer analysis.

3.11 Further Reading

- **Theory:** Kivelä et al. (2014). "Multilayer networks." *J. Complex Networks* 2(3): 203-271.
- **Physics:** Boccaletti et al. (2014). "The structure and dynamics of multilayer networks." *Physics Reports* 544(1): 1-122.
- **Textbook:** Bianconi, G. (2018). *Multilayer Networks: Structure and Function.* Oxford University Press.
- **Applications:** De Domenico et al. (2013). "Mathematical formulation of multilayer networks." *Physical Review X* 3(4): 041022.

CHAPTER

FOUR

DESIGN OF PY3PLEX

This chapter explains how py3plex is architected and why it makes specific design choices. Understanding these design principles will help you use the library more effectively and debug unexpected behavior.

4.1 Architecture Overview

Py3plex is built on three foundational principles:

1. **NetworkX compatibility** — Leverage a mature, well-tested ecosystem
2. **Modular design** — Separate concerns for flexibility and extensibility
3. **Simple, off-the-shelf functionality** — Minimize barrier to entry

The library consists of several core modules:

```
py3plex/
├── core/                    # Data structures (multi_layer_network class)
├── algorithms/              # Analysis methods
│   ├── statistics/          # Multilayer statistics
│   ├── centrality/          # Centrality measures
│   ├── community_detection/ # Community algorithms
│   └── paths/               # Path algorithms
├── dynamics/                # Dynamical processes (SIS, SIR, random walks)
├── dsl/                     # SQL-like query language
├── visualization/           # Plotting and visualization
├── io/                      # Data loading/saving
└── cli.py                   # Command-line interface
```

4.2 Node-Layer Pair Representation

4.2.1 The Fundamental Unit

Py3plex represents multilayer networks using **node-layer pairs** as the fundamental unit. A node v in layer α is stored as the tuple (v, α).

```
from py3plex.core import multinet

network = multinet.multi_layer_network()

# Add nodes explicitly as (node_id, layer_id) pairs
network.add_nodes([
    ('Alice', 'friends'),
```

```
    ('Bob', 'friends'),
    ('Alice', 'colleagues'),
])

# Or implicitly through edge addition
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1],
    ['Alice', 'colleagues', 'Bob', 'colleagues', 1],
], input_type="list")
```

Why node-layer pairs?

- **Unambiguous identification** — The same logical entity (Alice) has distinct identities in different layers
- **NetworkX compatibility** — Tuples are valid NetworkX node identifiers
- **Efficient lookups** — Python's tuple hashing enables fast operations
- **Natural tensor mapping** — Node-layer pairs map directly to rows/columns of the supra-adjacency matrix

4.3 The `multi_layer_network` Class

4.3.1 Core Components

The `multi_layer_network` class wraps a NetworkX graph and adds multilayer-specific functionality:

```
network = multinet.multi_layer_network(directed=False)

# The underlying NetworkX graph (MultiGraph or MultiDiGraph)
G = network.core_network

# Layer management
layers = network.get_layers()           # ['friends', 'colleagues']
layer_map = network.layer_name_map      # {'friends': 0, 'colleagues': 1}

# Access node-layer pairs
nodes = list(network.get_nodes())       # [('Alice', 'friends'), ...]
```

Key attributes:

- `core_network` — The NetworkX graph storing all node-layer pairs and edges
- `layer_name_map` — Mapping from layer names to internal IDs
- `directed` — Whether the network is directed

4.3.2 Multilayer vs. Multiplex Mode

Py3plex supports two operational modes:

Multilayer (default):

- General multilayer networks with arbitrary node sets per layer
- No automatic inter-layer edges
- Use for heterogeneous networks (different node types)

```
# Heterogeneous network: authors and papers
network = multinet.multi_layer_network(network_type='multilayer')
network.add_edges([
    ['Alice', 'authors', 'Paper1', 'papers', 1],
    ['Paper1', 'papers', 'ICML', 'venues', 1],
], input_type="list")
```

Multiplex:

- Same node set in all layers
- Automatically creates identity (coupling) edges with `type='coupling'`
- Use when the same entities appear in all layers

```
# Social network: same people, different relationships
network = multinet.multi_layer_network(network_type='multiplex')
network.load_network('social.edges', input_type='multiplex_edges')

# Coupling edges are auto-created: (Alice, friends) <-> (Alice, colleagues)

# Get explicit edges only (exclude coupling)
explicit_edges = list(network.get_edges(multiplex_edges=False))
```

4.4 Relationship to NetworkX

4.4.1 Direct Access to NetworkX

Py3plex does not hide NetworkX—it exposes the underlying graph for direct manipulation:

```
import networkx as nx

# Access the graph directly
G = network.core_network

# Use any NetworkX function
betweenness = nx.betweenness_centrality(G)
communities = nx.community.louvain_communities(G)

# Modify the graph directly
G.nodes[('Alice', 'friends')]['age'] = 30
```

Why this matters:

- You can use **any NetworkX algorithm** on the multilayer network
- You can **extend py3plex** by writing functions that operate on `core_network`
- You don't need to learn a completely new API—if you know NetworkX, you know most of py3plex

4.4.2 NetworkX Interoperability

Convert between py3plex and NetworkX:

```
# From NetworkX to py3plex
import networkx as nx
```

```
G = nx.karate_club_graph()
network = multinet.multi_layer_network()
network.load_network_from_networkx(G, layer_name='social')

# From py3plex to NetworkX (extract a single layer)
friends_layer = network.get_layer_subgraph('friends')
```

This enables workflows that mix py3plex's multilayer capabilities with NetworkX's extensive algorithm library.

4.5 Core Modules

4.5.1 Algorithms

Statistics (`py3plex/algorithms/statistics/`)
Multilayer-specific metrics like degree distributions, layer overlap, and aggregation statistics.

Centrality (`py3plex/algorithms/centrality/`)
Centrality measures adapted for multilayer networks, including explainable centrality that breaks down scores by layer.

Community Detection (`py3plex/algorithms/community_detection/`)
Multilayer Louvain, Infomap, and modularity-based methods.

Paths (`py3plex/paths/`)
Shortest paths, random walks, and flow algorithms that respect layer structure.

4.5.2 Dynamics

Dynamical Processes (`py3plex/dynamics/`)
OOP-style classes for epidemic models (SIS, SIR, SEIR), random walks, and custom dynamics. Each model has:

- `set_seed()` for reproducibility
- `run()` for execution
- `get_measure()` for extracting results

4.5.3 DSL and Query Language

Domain-Specific Language (`py3plex/dsl/`)
SQL-like queries for network analysis:

```
from py3plex.dsl import Q, L

result = (
    Q.nodes()
     .from_layers(L["friends"])
     .where(degree__gt=5)
     .compute("betweenness_centrality")
     .execute(network)
)
```

The DSL provides:

- **Builder API** — Chainable, type-hinted query construction
- **String syntax** — SQL-like queries as strings

- **EXPLAIN mode** — Query execution plans
- **Export** — Results to pandas, JSON, CSV

4.5.4 Visualization

Visualization (`py3plex/visualization/`)
Publication-ready plots including:

- **Hairball plots** — 2D/3D force-directed layouts with layer separation
- **Matrix plots** — Supra-adjacency matrix visualization
- **Diagonal layouts** — Specialized for large multilayer networks

4.5.5 I/O

Input/Output (`py3plex/io/`)
Support for multiple formats:

- **Edgelists** — Simple text format
- **GraphML** — XML-based format
- **Arrow/Parquet** — High-performance columnar format for large networks
- **JSON** — Flexible structured format

4.6 Design Principles in Practice

4.6.1 1. Flexible Input

Multiple ways to accomplish the same task:

```
# Method 1: List format
network.add_edges([['A', 'L1', 'B', 'L1', 1]], input_type="list")

# Method 2: Dict format (explicit)
network.add_edges([{
    'source': 'A', 'target': 'B',
    'source_type': 'L1', 'target_type': 'L1'
}])

# Method 3: Load from file
network.load_network("data.edgelist", input_type="edgelist")
```

4.6.2 2. Lazy Evaluation

Expensive operations are computed only when needed:

```
# Network creation and edge addition are fast
network = multinet.multi_layer_network()
network.add_edges([...])  # No supra-adjacency matrix computed

# Matrix is computed only when requested
supra_adj = network.get_supra_adjacency_matrix()  # Computed here
```

4.6.3 3. Graceful Degradation

The library handles edge cases and provides informative warnings:

- Missing layers → created automatically
- Empty networks → return sensible defaults
- Invalid parameters → clear error messages

4.6.4 4. Extensibility

Adding new algorithms is straightforward:

```
def my_custom_metric(network):
    """Custom multilayer metric."""
    G = network.core_network
    # Implement your algorithm using G
    return result

# Use immediately
score = my_custom_metric(network)
```

4.7 Why These Choices Matter

NetworkX compatibility means you can:

- Use hundreds of existing algorithms without modification
- Mix py3plex with other NetworkX-based tools
- Leverage a mature, well-documented ecosystem

Node-layer pair representation ensures:

- Unambiguous node identity across layers
- Efficient graph operations (hashing, lookups)
- Direct mapping to mathematical definitions

Modular architecture allows:

- Importing only needed functionality
- Easy testing and debugging
- Clean separation of concerns

Multiple input methods accommodate:

- Different data sources and formats
- User preferences and workflows
- Programmatic and interactive use

4.8 Limitations and Trade-offs

No design is perfect. Py3plex makes conscious trade-offs:

Memory:
The supra-adjacency matrix for large networks can be memory-intensive. Use sparse matrix formats (default) and avoid materializing the full matrix when possible.

Performance:
Some operations (e.g., cross-layer random walks) require iteration over node-layer pairs, which is slower than pure NetworkX on single-layer graphs.

Coupling semantics:
The automatic coupling in multiplex mode assumes identity edges. For more complex inter-layer relationships, use multilayer mode and add edges explicitly.

4.9 Summary

Py3plex is designed around:

1. **Node-layer pairs** as the fundamental unit
2. **NetworkX compatibility** for interoperability and extensibility
3. **Modular architecture** for flexibility
4. **Simple, practical APIs** for ease of use

These principles enable:

- Correct multilayer analysis (proper layer semantics)
- Efficient implementation (leverage NetworkX)
- Extensible design (add your own algorithms)
- Practical workflows (multiple input formats, visualization, DSL)

The following chapters will show how to use these design elements in practice: loading data, visualizing networks, and running multilayer-specific algorithms.

4.10 Further Reading

- NetworkX documentation: https://networkx.org/documentation/stable/
- Py3plex API reference: *Appendix E: Extended API and DSL Reference* (page 169)
- Code architecture: *Appendix A: Repository Layout and Scripts* (page 137)

CHAPTER

FIVE

INSTALLATION AND GETTING STARTED

This chapter covers installing py3plex and running your first multilayer network analysis. It presents one recommended path first, then lists alternatives for contributors and containerized workflows.

DSL Quick Start

After installation, try the DSL for instant network exploration:

```
from py3plex.core import multinet
from py3plex.dsl import Q, L

# Create a simple network
network = multinet.multi_layer_network()
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1],
    ['Bob', 'friends', 'Carol', 'friends', 1],
], input_type="list")

# Query it with DSL
result = (
    Q.nodes()
     .compute("degree")
     .order_by("-degree")
     .execute(network)
)
print(result.to_pandas())
```

DSL makes network analysis intuitive from day one!

5.1 Recommended Installation Path

For most users, installation is straightforward:

```
pip install py3plex
```

This installs py3plex with core dependencies (NetworkX, NumPy, SciPy, pandas, matplotlib).

Environment recommendation: create a fresh virtual environment before installing.

```
python3 -m venv py3plex-env
source py3plex-env/bin/activate  # Linux/macOS
# py3plex-env\Scripts\activate   # Windows
```

Using uv (optional):

uv[1] is a fast Python package installer and resolver. To use uv:

```
# Install uv
curl -LsSf https://astral.sh/uv/install.sh | sh

# Install py3plex with uv
uv pip install py3plex
```

5.1.1 Verification Command

```
import py3plex
print(py3plex.__version__)  # Should print 1.x
```

5.2 System Requirements

Python: 3.8 or higher (tested on 3.8, 3.9, 3.10, 3.11, 3.12)

Platforms: Linux, macOS, Windows

Core dependencies (installed automatically):

- NetworkX (2.8) — Graph data structures and algorithms
- NumPy (1.22) — Numerical computing and array operations
- SciPy (1.8) — Sparse matrices and scientific computing
- pandas (1.4) — Data manipulation and analysis
- matplotlib (3.5) — Visualization and plotting
- scikit-learn (1.0) — Machine learning utilities

5.3 Hello Multilayer Network

Create and analyze a multilayer network in 30 seconds:

```
from py3plex.core import multinet

# Create a multilayer network
net = multinet.multi_layer_network()

# Add nodes and edges
net.add_nodes([
    {'source': 'Alice', 'type': 'friends'},
    {'source': 'Bob', 'type': 'friends'},
    {'source': 'Alice', 'type': 'work'},
])

net.add_edges([
    {'source': 'Alice', 'target': 'Bob',
     'source_type': 'friends', 'target_type': 'friends'},
    {'source': 'Alice', 'target': 'Charlie',
```

[1] https://github.com/astral-sh/uv

```
    'source_type': 'work', 'target_type': 'work'},
])

# Display statistics
net.basic_stats()
```

See also: `examples/getting_started/tutorial_10min.py` for a complete tutorial.

Output:

```
Number of nodes: 6
Number of edges: 4
Number of unique nodes (as node-layer tuples): 6
Number of unique node IDs (across all layers): 4
Nodes per layer:
  Layer 'friends': 3 nodes
  Layer 'colleagues': 3 nodes
```

Key insight: The network has 6 node-layer pairs (Alice-friends, Alice-colleagues, Bob-friends, Bob-colleagues, Carol-friends, Dave-colleagues) but only 4 unique people.

5.3.1 Visualize the Network

```
from py3plex.visualization.multilayer import draw_multilayer_default

# Simple visualization
draw_multilayer_default(network.get_layers(), display=True)
```

This creates a force-directed layout with nodes colored by layer.

5.3.2 Basic Analysis

```
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Layer density (how connected is each layer?)
print(f"Friends density: {mls.layer_density(network, 'friends'):.3f}")
# Output: Friends density: 0.667 (2 of 3 possible edges exist)

# Node activity (fraction of layers a node appears in)
print(f"Bob's activity: {mls.node_activity(network, 'Bob'):.3f}")
# Output: Bob's activity: 1.000 (appears in both layers)
```

5.3.3 Query with DSL

Use SQL-like queries to explore the network:

```
from py3plex.dsl import Q, L
from py3plex.core import multinet

# Create a network
net = multinet.multi_layer_network()
# ... add nodes and edges ...
```

```
# Query: Find nodes with degree > 3 in the social layer
result = (
    Q.nodes()
     .from_layers(L["social"])
     .compute("degree")
     .where(degree__gt=3)
     .execute(net)
)

# Convert to pandas DataFrame
df = result.to_pandas()
print(df)
```

See also: `examples/network_analysis/example_dsl_builder_api.py` for complete DSL examples.

The DSL (covered in Part III) provides a concise way to filter, compute, and analyze multilayer networks without writing explicit loops.

5.4 Alternative Installation Paths (Use Only If Needed)

5.4.1 Development Installation

For contributors or to access the latest features:

```
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex
pip install -e .
```

The `-e` flag installs in editable mode—changes to source code are immediately available.

For development with testing/linting tools:

```
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex
make setup        # Create virtual environment
make dev-install  # Install with dev dependencies
```

This installs pytest, black, ruff, mypy, and sphinx for development workflows.

5.4.2 Docker Installation

For a containerized environment with all dependencies:

```
# Clone repository
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex

# Build image
docker build -t py3plex:latest .

# Run commands
docker run --rm py3plex:latest --version
```

Benefits:

- No Python setup required
- Consistent environment across systems
- Isolated from system Python

5.4.3 Optional Dependencies

Py3plex supports optional feature sets:

```
# Advanced community detection (Infomap)
pip install py3plex[infomap]

# Extra algorithms (Louvain, cdlib)
pip install py3plex[algos]

# Advanced visualization (Plotly, igraph)
pip install py3plex[viz]

# High-performance I/O (Arrow/Parquet)
pip install py3plex[arrow]

# Install multiple extras
pip install py3plex[infomap,viz,algos,arrow]
```

Conda alternative:

```
conda create -n py3plex python=3.10
conda activate py3plex
pip install py3plex
```

5.5 Key Concepts

Before proceeding, understand these fundamental concepts:

5.5.1 Node-Layer Pairs

In py3plex, a node is always associated with a layer. The tuple `(node_id, layer_id)` is the fundamental unit.

- `('Alice', 'friends')` is different from `('Alice', 'colleagues')`
- This allows the same logical entity to have different properties and connections in different contexts

Example:

```
# Access nodes as (node_id, layer_id) tuples
for node in network.get_nodes():
    print(node)  # ('Alice', 'friends'), ('Bob', 'friends'), ...
```

5.5.2 NetworkX Compatibility

Py3plex uses NetworkX as its underlying graph representation:

```
import networkx as nx

# Direct access to the NetworkX graph
```

```
G = network.core_network

# Use any NetworkX algorithm
betweenness = nx.betweenness_centrality(G)

# Modify directly
G.nodes[('Alice', 'friends')]['age'] = 30
```

This means:

- Any NetworkX algorithm works on py3plex networks
- You can mix py3plex and NetworkX functions freely
- Learning curve is minimal if you already know NetworkX

5.5.3 Input Formats

Py3plex supports multiple edge input formats:

List format (used in examples above):

```
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1],
], input_type="list")
```

Dictionary format (explicit, Pythonic):

```
network.add_edges([{
    'source': 'Alice',
    'source_type': 'friends',
    'target': 'Bob',
    'target_type': 'friends',
    'weight': 1
}])
```

File loading (for larger networks):

```
# Edgelist format
network.load_network("data.edgelist", input_type="edgelist")

# GraphML, JSON, Arrow, etc.
# See :ref:`data-loading-chapter` for details
```

5.6 Common Issues

5.6.1 Issue: “No module named ‘py3plex’”

Solution: Ensure py3plex is installed in the active Python environment:

```
pip install py3plex
```

Verify which Python interpreter you’re using:

```
which python  # Linux/macOS
where python  # Windows
```

5.6.2 Issue: Import errors for optional dependencies

Solution: Install the relevant extra:

```
pip install py3plex[infomap]  # For Infomap
pip install py3plex[viz]      # For Plotly
```

5.6.3 Issue: Permission denied on Linux/macOS

Solution: Use a virtual environment (recommended) or install with `--user`:

```
pip install --user py3plex
```

5.7 License Considerations

Py3plex core is released under the **MIT License**, which allows commercial and non-commercial use without restrictions.

5.7.1 Optional AGPL Components

Important: Some optional features include code licensed under **AGPLv3** (a copyleft license):

- **Infomap community detection** (`py3plex[infomap]`)

If you install and use these features, your application may be subject to AGPLv3 requirements:

- You must make your application's source code available if you distribute it
- Modified versions must also be released under AGPLv3

How to avoid AGPLv3:

1. Don't install extras with AGPL components (don't use `pip install py3plex[infomap]`)
2. Use alternative algorithms (Louvain, Label Propagation) instead of Infomap
3. Check the license of any extra before installing

Default installation (`pip install py3plex`) includes only MIT-licensed code.

5.8 Verifying Installation

Run a self-test to confirm everything works:

```
from py3plex.core import multinet
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Create test network
network = multinet.multi_layer_network()
network.add_edges([
    ['A', 'L1', 'B', 'L1', 1],
    ['B', 'L1', 'C', 'L1', 1],
], input_type="list")
```

```
# Test basic operations
assert network.get_number_of_nodes() == 3
assert mls.layer_density(network, 'L1') > 0

print("✓ py3plex is working correctly")
```

5.9 Next Steps

Now that you have py3plex installed and understand the basics:

- *Data Loading and Representation* (page 37) — Learn about data loading, supported formats, and best practices for representing multilayer networks
- *Visualization and Exploration* (page 48) — Explore visualization techniques for multilayer networks
- *Core Algorithms: Communities, Centrality, Dynamics* (page 54) — Apply core algorithms: community detection, centrality measures, and dynamics

For immediate exploration:

Browse the `examples/` directory in the repository for current workflows:

- `examples/getting_started/` — first-run examples
- `examples/network_analysis/` — DSL, algorithms, and analysis patterns
- `examples/dsl_zoo/` — focused DSL query patterns
- `examples/io_and_data/` — loading and conversion workflows
- `examples/visualization/` — plotting and rendering examples

Run any example:

```
# Using uv (recommended)
uv run examples/getting_started/01_basic_query.py

# Or using python
python examples/getting_started/01_basic_query.py
```

5.10 Closing Note

If the recommended path worked, continue directly to *Data Loading and Representation* (page 37). Return to alternative paths only when you need editable installs, optional extras, or containerized execution.

CHAPTER

SIX

DATA LOADING AND REPRESENTATION

This chapter covers how to load multilayer networks from various data sources, choose appropriate formats, and represent complex network structures correctly.

DSL Tip: Validate Data After Loading

Use DSL to quickly validate loaded data:

```
from py3plex.dsl import Q, L

# Check layer sizes
for layer in network.get_layers():
    count = Q.nodes().from_layers(L[layer]).execute(network).count
    print(f"{layer}: {count} nodes")

# Find potential data issues
isolated = Q.nodes().where(degree=0).execute(network)
if isolated.count > 0:
    print(f"Warning: {isolated.count} isolated nodes")

# Verify high-degree nodes make sense
hubs = (
    Q.nodes()
     .where(degree__gt=20)
     .compute("degree")
     .execute(network)
)
print(f"Hubs (degree > 20): {hubs.count}")
```

Quick validation catches data issues early!

6.1 Data Loading Basics

6.1.1 Multiple Input Methods

Py3plex supports several ways to create multilayer networks:

1. Direct edge addition (for programmatic construction):

```
from py3plex.core import multinet

# Create network
```

```
network = multinet.multi_layer_network()

# Add edges directly
network.add_edges([
    {'source': 'Alice', 'target': 'Bob',
     'source_type': 'layer1', 'target_type': 'layer1'},
    {'source': 'Bob', 'target': 'Charlie',
     'source_type': 'layer1', 'target_type': 'layer1'},
    {'source': 'Alice', 'target': 'Charlie',
     'source_type': 'layer2', 'target_type': 'layer2'},
])

network.basic_stats()
```

See also: `examples/getting_started/tutorial_10min.py` for complete examples.

2. File loading (for external data):

```
# Load from file
network = multinet.multi_layer_network()
network.load_network("data.edgelist", input_type="edgelist")
```

3. From NetworkX (for integration):

```
import networkx as nx
from py3plex.core import multinet

# Create NetworkX graphs
G1 = nx.karate_club_graph()
G2 = nx.erdos_renyi_graph(20, 0.1)

# Convert to multilayer network
network = multinet.multi_layer_network()
network.add_layer_from_nx(G1, layer_name="social")
network.add_layer_from_nx(G2, layer_name="work")
```

See also: `examples/getting_started/example_networkx_wrapper.py` for NetworkX integration examples.

6.2 Edgelist Format

The **edgelist** format is the simplest and most common:

```
# File: network.edgelist
Alice friends Bob friends 1.0
Bob friends Carol friends 1.0
Alice colleagues Bob colleagues 0.8
```

Format: `source source_layer target target_layer weight`

Loading:

```
network = multinet.multi_layer_network()
network.load_network("network.edgelist", input_type="edgelist")
```

Best practices:

- Use consistent node identifiers across layers
- Separate fields with spaces or tabs
- Include weights (default to 1.0 if omitted)
- Comment lines start with #

6.3 Dictionary Format

For programmatic construction, the dictionary format is most explicit:

```
network.add_edges([
    {
        'source': 'Alice',
        'source_type': 'friends',
        'target': 'Bob',
        'target_type': 'friends',
        'weight': 1.0,
        'timestamp': '2023-01-15'  # Optional attributes
    },
    {
        'source': 'Bob',
        'source_type': 'colleagues',
        'target': 'Carol',
        'target_type': 'colleagues',
        'weight': 0.8
    }
])
```

Advantages:

- Self-documenting (field names explicit)
- Supports arbitrary edge attributes
- Type-safe (Python dicts)
- Pythonic and readable

6.4 Modern I/O System

Py3plex provides a modern I/O system (`py3plex.io`) for high-performance serialization:

6.4.1 Supported Formats

Format	Extension	Best For	Trade-offs
JSON	`.json`	Human-readable, small networks	Slower, larger files
JSONL	`.jsonl`	Streaming, large networks	Not human-friendly
CSV	`.csv`	Spreadsheet tools, manual editing	Limited attributes
Arrow	`.arrow`	High performance, large networks	Requires pyarrow
Parquet	`.parquet`	Storage, compression, archiving	Requires pyarrow

6.4.2 Reading and Writing

```
from py3plex.io import read, write, MultiLayerGraph

# Write (auto-detects format from extension)
write(network, 'network.json')
write(network, 'network.arrow')
write(network, 'network.parquet')

# Read (auto-detects format)
graph = read('network.json')
graph = read('network.arrow')

# Specify format explicitly
write(network, 'myfile.dat', format='json')
graph = read('myfile.dat', format='parquet')
```

6.4.3 Apache Arrow Format (Recommended for Large Networks)

Arrow provides 2-3x faster I/O and better compression:

```
# Install Arrow support
pip install 'py3plex[arrow]'
```

```
from py3plex.io import read, write

# Feather format (fast, uncompressed)
write(graph, 'network.arrow')
graph = read('network.arrow')

# Parquet format (compressed, archival)
write(graph, 'network.parquet')
graph = read('network.parquet')
```

Performance comparison (1000 nodes, 5000 edges):

Format	Write Time	Read Time	File Size
Arrow	0.016s	0.008s	0.46 MB
Parquet	0.020s	0.010s	0.35 MB
JSON	0.046s	0.030s	1.09 MB

When to use Arrow:

- Large networks (>10k nodes)
- Performance-critical pipelines
- Interoperability with pandas, Spark, DuckDB
- Production data workflows

When to use JSON:

- Small networks (<1k nodes)
- Human readability required
- Debugging and manual editing
- Maximum compatibility

6.4.4 CSV with Sidecars

CSV format supports optional sidecar files for attributes:

```
# Write with sidecars
write(graph, 'edges.csv', write_sidecars=True)
# Creates: edges.csv, nodes.csv, layers.csv

# Read with sidecars
graph = read('edges.csv',
             nodes_file='nodes.csv',
             layers_file='layers.csv')
```

6.5 Data Validation and Best Practices

6.5.1 Always Verify After Loading

After loading external data, always verify structure:

```
network.load_network("data.edgelist", input_type="edgelist")

# Verify structure
print(f"Layers: {network.get_layers()}")
print(f"Nodes per layer: {network.get_number_of_nodes_per_layer()}")

# Check for unexpected layers
expected_layers = {'friends', 'colleagues'}
actual_layers = set(network.get_layers())
if actual_layers != expected_layers:
    print(f"Warning: unexpected layers {actual_layers - expected_layers}")
```

```
# Basic statistics
network.basic_stats()
```

6.5.2 Consistent Node Identifiers

Rule: The same entity must have the same ID across all layers.

Good:

```
# Alice has the same ID in both layers
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1],
    ['Alice', 'colleagues', 'Carol', 'colleagues', 1],
], input_type="list")
```

Bad:

```
# Alice's ID differs across layers-breaks multiplex structure
network.add_edges([
    ['alice_social', 'friends', 'Bob', 'friends', 1],
    ['Alice_Work', 'colleagues', 'Carol', 'colleagues', 1],
], input_type="list")
```

6.5.3 Naming Conventions

Layer names:

- Use descriptive names: `'coauthor'`, `'twitter'`, not `'layer1'`, `'l2'`
- Lowercase with underscores: `'social_media'`, not `'SocialMedia'`
- Avoid special characters

Node IDs:

- Use stable, meaningful identifiers (user IDs, DOIs, names)
- Avoid auto-generated IDs that change across runs
- Preserve external IDs when loading from databases

6.5.4 Metadata and Attributes

Store metadata as node/edge attributes:

```
# Add node attributes
network.core_network.nodes[('Alice', 'friends')]['age'] = 30
network.core_network.nodes[('Alice', 'friends')]['country'] = 'USA'

# Add edge attributes
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1, {'type': 'close', 'since': 2018}]
], input_type="list")
```

Recommended practices:

- Use consistent attribute names across layers

- Store timestamps as ISO format strings: `'2023-01-15T10:30:00Z'`
- Use numeric types for attributes that will be analyzed
- Document attribute semantics in code comments

6.6 Representing Different Network Types

This section provides practical examples for loading different types of multilayer networks. For theoretical foundations and mathematical definitions, see *Multilayer Network Basics* (page 9).

6.6.1 Multiplex Networks (Same Nodes Across Layers)

For networks where the same entities appear in all layers:

```
network = multinet.multi_layer_network(network_type='multiplex')

# Load data
network.load_network('social.edges', input_type='multiplex_edges')

# Coupling edges are automatically created
# (Alice, friends) <-> (Alice, colleagues)

# Get explicit edges only (exclude coupling)
explicit_edges = list(network.get_edges(multiplex_edges=False))
```

When to use:

- Same people across multiple social platforms
- Same cities in transportation modes (air, rail, road)
- Same proteins in different tissue types

6.6.2 Heterogeneous Networks (Different Node Types)

For networks with different entity types per layer:

```
network = multinet.multi_layer_network(network_type='multilayer')

# Different node types
network.add_edges([
    ['Alice', 'authors', 'Paper1', 'papers', 1],
    ['Bob', 'authors', 'Paper1', 'papers', 1],
    ['Paper1', 'papers', 'ICML', 'venues', 1],
], input_type="list")
```

When to use:

- Academic networks (authors, papers, venues)
- E-commerce (users, products, sellers)
- Biomedical (drugs, diseases, targets)

6.6.3 Temporal Networks (Time-Sliced Layers)

For networks that evolve over time:

```
network = multinet.multi_layer_network()

# Time windows as layers
network.add_edges([
    ['Alice', '2020', 'Bob', '2020', 1],
    ['Alice', '2021', 'Bob', '2021', 1],
    ['Bob', '2021', 'Carol', '2021', 1],
], input_type="list")
```

Best practices:

- Use consistent time granularity (years, months, days)
- Layer names should be sortable: `'2020-01'`, `'2020-02'`, ...
- Add temporal edges connecting adjacent time slices if modeling evolution

6.7 Converting Between Formats

6.7.1 NetworkX to Py3plex

```
import networkx as nx
from py3plex.core import multinet

# Create NetworkX graph
G = nx.karate_club_graph()

# Convert to py3plex
network = multinet.multi_layer_network()
network.load_network_from_networkx(G, layer_name='social')
```

6.7.2 Py3plex to NetworkX

```
# Extract single layer as NetworkX graph
friends_layer = network.get_layer_subgraph('friends')

# Or access the full graph directly
full_graph = network.core_network  # MultiGraph or MultiDiGraph
```

6.7.3 Pandas DataFrame to Py3plex

```
import pandas as pd

# Load edges from DataFrame
df = pd.read_csv('edges.csv')
# Columns: source, source_layer, target, target_layer, weight

# Convert to list of lists
edges = df[['source', 'source_layer', 'target', 'target_layer', 'weight']].values.tolist()
```

```
network = multinet.multi_layer_network()
network.add_edges(edges, input_type="list")
```

6.8 Common Data Loading Patterns

6.8.1 Pattern 1: Load from Multiple Files

```
network = multinet.multi_layer_network()

# Load different layers from different files
for layer_name, filename in [('friends', 'friends.txt'),
                             ('colleagues', 'colleagues.txt')]:
    with open(filename) as f:
        for line in f:
            source, target, weight = line.strip().split()
            network.add_edges([
                [source, layer_name, target, layer_name, float(weight)]
            ], input_type="list")
```

6.8.2 Pattern 2: Load from Database

```
import sqlite3

conn = sqlite3.connect('network.db')
cursor = conn.execute(
    "SELECT source, source_layer, target, target_layer, weight FROM edges"
)

network = multinet.multi_layer_network()
edges = [[row[0], row[1], row[2], row[3], row[4]] for row in cursor]
network.add_edges(edges, input_type="list")
```

6.8.3 Pattern 3: Incremental Loading (Large Networks)

```
network = multinet.multi_layer_network()

# Load in batches to manage memory
BATCH_SIZE = 10000

with open('large_network.edgelist') as f:
    batch = []
    for line in f:
        parts = line.strip().split()
        batch.append(parts)

        if len(batch) >= BATCH_SIZE:
            network.add_edges(batch, input_type="list")
            batch = []

    # Add remaining edges
```

```
    if batch:
        network.add_edges(batch, input_type="list")
```

6.9 Troubleshooting

6.9.1 Issue: Duplicate Edges

Symptom: More edges than expected after loading.

Solution: Check for duplicate entries in source data:

```
# Count edge multiplicity
edge_counts = {}
for edge in network.get_edges():
    edge_counts[edge] = edge_counts.get(edge, 0) + 1

duplicates = {e: c for e, c in edge_counts.items() if c > 1}
if duplicates:
    print(f"Found {len(duplicates)} duplicate edges")
```

6.9.2 Issue: Unexpected Layers

Symptom: Layers you didn't expect appear in the network.

Solution: Verify layer names in source data:

```
expected = {'friends', 'colleagues'}
actual = set(network.get_layers())

if actual != expected:
    print(f"Unexpected layers: {actual - expected}")
    print(f"Missing layers: {expected - actual}")
```

6.9.3 Issue: Node ID Mismatches

Symptom: Same entity appears as different nodes across layers.

Solution: Normalize node IDs before loading:

```
def normalize_id(node_id):
    """Normalize node identifiers."""
    return str(node_id).strip().lower()

# Apply during loading
edges = [[normalize_id(s), sl, normalize_id(t), tl, w]
         for s, sl, t, tl, w in raw_edges]
network.add_edges(edges, input_type="list")
```

6.10 Summary

This chapter covered:

1. **Loading methods** — Direct addition, files, NetworkX import

2. **Data formats** — Edgelist, dictionary, JSON, CSV, Arrow/Parquet
3. **Best practices** — Consistent IDs, validation, metadata storage
4. **Network types** — Multiplex, heterogeneous, temporal
5. **Common patterns** — Multi-file loading, databases, incremental loading

Key recommendations:

- Use **Arrow/Parquet** for large networks (>10k nodes)
- Use **JSON** for small networks and debugging
- **Always verify** structure after loading
- Use **consistent node IDs** across layers
- Store **metadata as attributes** for richer analysis

The next chapter covers visualization techniques for exploring multilayer network structure.

6.11 Further Reading

- Arrow format specification: https://arrow.apache.org/docs/
- NetworkX I/O reference: https://networkx.org/documentation/stable/reference/readwrite/
- *Core Algorithms: Communities, Centrality, Dynamics* (page 54) for analysis after loading

CHAPTER

SEVEN

VISUALIZATION AND EXPLORATION

Multilayer networks present unique visualization challenges: how do you clearly show multiple relationship types while preserving the overall structure? This chapter focuses on practical plotting workflows, with emphasis on interpreting output and adjusting plots for your data.

7.1 What You Will Learn

By the end of this chapter, you will be able to:

1. Pick a visualization mode based on network size and analysis goal.
2. Read visual artifacts critically (layer overlap, occlusion, misleading hubs).
3. Produce exportable figures and identify where manual styling is still required.

7.2 Overview

Visualizing multilayer networks requires balancing several concerns:

- **Clarity:** Individual layers must be distinguishable
- **Structure:** Cross-layer connections should be visible
- **Scale:** Techniques must work for networks of varying sizes (10 to 10,000+ nodes)
- **Purpose:** Exploratory plots need different settings than publication figures

py3plex provides three main visualization approaches:

1. **Static multilayer plots** — Show all layers simultaneously with customizable layouts
2. **Matrix visualizations** — Display the supra-adjacency matrix structure
3. **Layer-specific views** — Examine individual layers in detail

7.3 Quick Start

7.3.1 Basic Multilayer Visualization

```
from py3plex.core import multinet
from py3plex.visualization.multilayer import draw_multilayer_default

# Load network
network = multinet.multi_layer_network()
network.load_network("network.csv", input_type="multiedgelist")
```

```
# Visualize with defaults
draw_multilayer_default(
    network.get_layers(),  # Returns tuple: (layer_names, layer_graphs, multiedges)
    display=True,
    labels=True
)
```

This produces a circular layout with each layer shown in a different color.

API Note: Input Format

`draw_multilayer_default` expects the value returned by `network.get_layers()`. This method returns a tuple `(layer_names, layer_graphs, multiedges)`, where `layer_graphs` is a list of NetworkX graph objects already prepared for plotting.

7.4 Preset Visualization Modes

Py3plex provides three preset modes optimized for different network scales.

7.4.1 Minimal Mode (Large Networks >1000 nodes)

Optimized for networks where detail isn't critical and performance matters:

```
# Large network preset
draw_multilayer_default(
    network.get_layers(),
    node_size=5,              # Small nodes
    labels=False,             # No labels (too cluttered)
    edge_size=0.5,            # Thin edges
    alphalevel=0.3,           # Transparent edges
    background_shape="circle",
    display=True,
    remove_isolated_nodes=True
)
```

Use cases: Large social networks, overview visualizations, pattern detection at macro level.

Advantages: Fast rendering, reduced clutter, shows overall structure.

7.4.2 Balanced Mode (Medium Networks 100-1000 nodes)

Default settings that work well for most networks:

```
# Balanced preset (this is the default)
draw_multilayer_default(
    network.get_layers(),
    node_size=10,             # Medium nodes
    labels=True,              # Show layer labels
    edge_size=1.0,            # Normal edges
    alphalevel=0.13,          # Semi-transparent
    background_shape="circle",
    networks_color="colorblind_safe",
```

```
    display=True
)
```

Use cases: Most research networks, exploratory analysis, publication figures with moderate detail.

Advantages: Good readability, reasonable performance, suitable for most publications.

7.4.3 Dense Mode (Small Networks <100 nodes)

Maximum detail for small networks where every node matters:

```
# Dense preset for detailed examination
draw_multilayer_default(
    network.get_layers(),
    node_size=20,              # Large nodes
    labels=True,
    node_labels=True,          # Show node IDs
    node_font_size=10,
    scale_by_size=True,        # Scale by degree
    edge_size=2.0,             # Thick edges
    alphalevel=0.8,            # Mostly opaque
    background_shape="rectangle",
    display=True
)
```

Use cases: Small case studies, detailed node-level analysis, high-quality publication figures.

Advantages: Maximum detail, clear node identification, professional appearance.

7.5 Layout Options

7.5.1 Circular Layout (Default)

Arranges layers in a circle, good for showing inter-layer connections:

```
draw_multilayer_default(
    network.get_layers(),
    background_shape="circle",
    rectanglex=1.0,  # Circle radius
    rectangley=1.0
)
```

Best for: 2-10 layers, symmetric interactions, publication figures.

7.5.2 Rectangular Layout

Arranges layers in a grid, good for many layers or hierarchical structures:

```
draw_multilayer_default(
    network.get_layers(),
    background_shape="rectangle",
    rectanglex=2.0,  # Width
    rectangley=1.0   # Height
)
```

Best for: Many layers (>10), hierarchical networks, time-series data, wide figures.

7.6 Auto-Scaling Features

7.6.1 Node Sizing by Degree

Automatically scale node sizes by their degree:

```
draw_multilayer_default(
    network.get_layers(),
    scale_by_size=True,    # Enable auto-scaling
    node_size=10           # Base size
)
```

Hub nodes become larger, making them easy to identify.

7.6.2 Color Assignment

Py3plex provides several color palette options:

```
# Automatic colors (uses default palette)
draw_multilayer_default(
    network.get_layers(),
    networks_color="auto"
)

# Custom palette
draw_multilayer_default(
    network.get_layers(),
    networks_color=["#FF6B6B", "#4ECDC4", "#45B7D1"]
)
```

Available palettes (from `py3plex.config`):

- `colorblind_safe`: 8 colors safe for colorblind viewers (default)
- `wong`: 7-color Wong palette[2] (optimized for color vision deficiency, including protanopia and deuteranopia)
- `tol_bright`: 7 bright colors
- `rainbow`: Full spectrum colors (not recommended for accessibility)

Recommendation: Use `colorblind_safe` or `wong` palettes for publications and presentations to ensure accessibility for all viewers, including those with color blindness or other color vision deficiencies.

7.7 Output-Driven Workflow

Use the following sequence when generating figures for analysis notes or papers:

1. **Render a baseline figure** with balanced mode.
2. **Check readability** (label collisions, hidden inter-layer edges, overplotted hubs).
3. **Adjust one variable at a time** (node size, alpha, labels, layout shape).
4. **Re-export and compare** before/after versions.

[2] Wong, B. (2011). Points of view: Color blindness. *Nature Methods*, 8(6), 441. doi:10.1038/nmeth.1618

```
# 1) Baseline render
draw_multilayer_default(network.get_layers(), display=True, labels=True)

# 2) Export candidate figure
draw_multilayer_default(
    network.get_layers(),
    node_size=8,
    labels=False,
    alphalevel=0.2,
    background_shape="circle",
    display=False
)
import matplotlib.pyplot as plt
plt.savefig("figure_candidate.png", dpi=300, bbox_inches="tight")
```

Treat exported figures as analysis artifacts that usually need dataset-specific styling before publication.

7.8 Matrix Visualizations

7.8.1 Supra-Adjacency Matrix

Visualize the full supra-adjacency matrix structure:

```
# Display matrix view
network.visualize_matrix({"display": True})
```

This shows the block structure: diagonal blocks are intra-layer edges, off-diagonal blocks are inter-layer edges.

Use cases: Understanding network structure, debugging layer connections, small networks only (<500 nodes).

7.9 Exploration Workflows

7.9.1 Layer Views

Examine individual layers:

```
# Extract and visualize a single layer
layer_subgraph = network.get_layer("social")

import networkx as nx
import matplotlib.pyplot as plt

nx.draw(layer_subgraph, with_labels=True)
plt.show()
```

7.9.2 Cross-Layer Pattern Analysis

Use the DSL to filter and visualize subsets:

```
from py3plex.dsl import Q, L

# Find high-degree nodes across specific layers
result = (
```

```
    Q.nodes()
     .from_layers(L["social"] + L["professional"])
     .where(degree__gt=10)
     .execute(network)
)

# Extract subgraph for visualization
high_degree_nodes = list(result.node_ids)
subgraph = network.core_network.subgraph(high_degree_nodes)
```

7.9.3 Node Neighborhoods

Extract ego networks (neighborhood around a specific node):

```
import networkx as nx

# Get 2-hop neighborhood around 'Alice'
alice_ego = nx.ego_graph(network.core_network, ('Alice', 'social'), radius=2)

# Visualize
nx.draw(alice_ego, with_labels=True)
```

7.10 Performance Considerations

Visualization performance depends heavily on network size:

- **<100 nodes:** All visualization modes work well
- **100-1000 nodes:** Use balanced or minimal mode; avoid node labels
- **1000-5000 nodes:** Use minimal mode; rendering may be slow
- **>5000 nodes:** Static visualizations become impractical; use matrix view or export to specialized tools

Tips for large networks:

1. Use `remove_isolated_nodes=True` to reduce clutter
2. Set `alphalevel=0.1` or lower for edge transparency
3. Disable labels: `labels=False, node_labels=False`
4. Consider sampling: visualize only high-degree nodes or a random subset

7.11 Closing Note

Visualization in py3plex is strongest as an exploratory and diagnostic tool. For final publication figures, expect to iterate on styling, annotation, and layout outside the first default render.

Next chapter: *Core Algorithms: Communities, Centrality, Dynamics* (page 54)

7.12 References

CHAPTER

EIGHT

CORE ALGORITHMS: COMMUNITIES, CENTRALITY, DYNAMICS

This chapter covers py3plex's three major algorithm families for multilayer network analysis: community detection, centrality measures, and dynamics/spreading processes. It also states assumptions and caveats so results are interpreted within method limits.

DSL Tip: Streamline Algorithm Workflows

The DSL simplifies many algorithmic workflows:

```
from py3plex.dsl import Q, L

# Quick centrality analysis
top_central = (
    Q.nodes()
     .from_layers(L["social"])
     .compute("betweenness_centrality", "pagerank", "degree")
     .order_by("-betweenness_centrality")
     .limit(20)
     .execute(network)
)

# Multilayer comparison
for layer in ["social", "work"]:
    result = (
        Q.nodes()
         .from_layers(L[layer])
         .compute("betweenness_centrality")
         .execute(network)
    )
    print(f"{layer}: avg BC = {sum(result.measures['betweenness_centrality'].values()) / result.count
```

See *Introduction to the Py3plex DSL* (page 63) for comprehensive DSL coverage!

8.1 Overview

py3plex provides three major algorithm families for analyzing multilayer networks:

1. **Community Detection** — Find groups of nodes that are densely connected within and across layers. Algorithms include Louvain, Infomap, Label Propagation, and multilayer modularity optimization.
2. **Centrality Measures** — Identify important nodes using degree, betweenness, closeness, PageRank, eigenvector centrality, and 30+ other measures adapted for multilayer structure.

3. **Dynamics and Processes** — Simulate epidemic spread (SIR, SIS), random walks, and diffusion processes that leverage multilayer connectivity patterns.

These algorithms can account for both intra-layer (within-layer) and inter-layer (cross-layer) connections, but this depends on the selected method and representation.

8.2 Assumptions and Caveats

Before applying any algorithm in this chapter, check the following:

1. **Graph representation:** some workflows operate on a supra-graph approximation rather than a solver with explicit multilayer coupling terms.
2. **Metric cost:** global path-based metrics (especially betweenness) can become impractical on large graphs.
3. **Model fit:** epidemic and diffusion models are stylized abstractions; parameter sensitivity should be explored, not assumed away.
4. **Result transferability:** conclusions from one layer configuration or one seed do not automatically generalize.

8.3 Community Detection

Community detection identifies groups of nodes that are more densely connected to each other than to the rest of the network. In multilayer networks, communities can span multiple layers, accounting for both intra-layer and inter-layer structure.

8.3.1 Multilayer Modularity

The multilayer modularity quality function extends Newman-Girvan modularity to account for both intra-layer and inter-layer edges:

$$Q = \frac{1}{2\mu} \sum_{ijrs} \left[A_{ijr} - \gamma_r \frac{k_{ir} k_{jr}}{2m_r} \right] \delta(g_i, g_j) \delta_{rs} + \frac{1}{2\mu} \sum_{ijrs} C_{ijrs} \delta(g_i, g_j)$$

Where:

- A_{ijr} is the adjacency matrix in layer *r*
- k_{ir} is the degree of node *i* in layer *r*
- γ_r is the resolution parameter for layer *r*
- C_{ijrs} represents inter-layer coupling
- $\delta(g_i, g_j)$ is 1 if nodes *i* and *j* are in the same community

Implementation:

```
from py3plex.algorithms.community_detection import community_louvain

# Detect communities using Louvain on the supra-graph representation
# Note: This applies standard Louvain to the flattened multilayer structure
communities = community_louvain.best_partition(network.core_network)

# Print community assignments
for node, comm_id in communities.items():
    print(f"Node {node} -> Community {comm_id}")
```

Implementation Note

The implementation shown applies standard Louvain modularity optimization to the **supra-graph representation** of the multilayer network (`network.core_network`), where all layers are combined into a single NetworkX MultiGraph. This is a practical approximation that works well for many use cases.

True multilayer modularity optimization (Mucha et al. 2010) with explicit inter-layer coupling terms requires specialized solvers. For such optimization, consider using external tools like GenLouvain (MATLAB) or multilayer-specific implementations, then importing results back into py3plex.

8.3.2 Algorithms Available

py3plex supports several community detection algorithms. See `examples/network_analysis/` for complete examples:

Example 1: Single-Layer Louvain

See `examples/network_analysis/example_community_detection.py`:

```
# Using uv
uv run examples/network_analysis/example_community_detection.py

# Or using python
python examples/network_analysis/example_community_detection.py
```

Example 2: Multilayer Detection

See `examples/network_analysis/example_multiplex_community_detection.py`:

```
# Using uv
uv run examples/network_analysis/example_multiplex_community_detection.py

# Or using python
python examples/network_analysis/example_multiplex_community_detection.py
```

Example 3: AutoCommunity Detection

See `examples/network_analysis/example_auto_select_basic.py`:

```
# Using uv
uv run examples/network_analysis/example_auto_select_basic.py

# Or using python
python examples/network_analysis/example_auto_select_basic.py
```

Available algorithms:

- **Louvain** — Fast modularity optimization, O(n log n). Best for most use cases (100-100K nodes).
- **Infomap** — Flow-based detection using random walk dynamics. Optional dependency (see below).
- **Label Propagation** — Semi-supervised approach with known seed communities. Linear time O(m + n).

8.3.3 Infomap Integration

Infomap is an optional community detection algorithm based on information-theoretic compression of random walks. It is not included in the default py3plex installation due to licensing (AGPL).

Installation:

```
pip install py3plex[infomap]
```

Usage:

```
from py3plex.algorithms.community_detection import infomap_communities

# Run Infomap on network
communities = infomap_communities(network.core_network)

# communities is a dict: {node: community_id}
```

Important notes:

- Infomap is licensed under AGPLv3 (copyleft license) - see *Installation and Getting Started* (page 29) for license implications
- If you don't need Infomap specifically, use Louvain or Label Propagation instead
- Infomap tends to find smaller, more fine-grained communities than Louvain

Time Complexity Summary:

- Louvain: O(n log n) typical, $O(n^2)$ worst case
- Label Propagation: O(m + n) per iteration
- Infomap: O(m log n) typical

8.3.4 Choosing a Community Detection Method

Selection guidelines:

- **Network size < 10,000 nodes:** Use Louvain (fast and accurate)
- **Network size > 100,000 nodes:** Use Label Propagation (linear time) or streaming Louvain
- **Strong inter-layer coupling:** Use multilayer modularity for true cross-layer communities
- **Known seed communities:** Use Label Propagation with seeds
- **Flow-based interpretation needed:** Use Infomap (installation required)

Interpretability considerations:

- Louvain and multilayer modularity produce hierarchical community structure
- Infomap optimizes for information compression, emphasizing flow patterns
- Label Propagation is less deterministic but handles semi-supervised scenarios

8.4 Centrality Measures

Centrality measures identify important nodes in the network. py3plex provides direct DSL access to core centrality measures (degree, betweenness, closeness, eigenvector, PageRank, clustering) and seamless integration with NetworkX's 30+ additional centrality algorithms.

DSL-integrated measures (available via ``Q.nodes().compute()``):

- `degree` — Node degree (number of edges)
- `degree_centrality` — Normalized degree centrality
- `betweenness_centrality` — Betweenness centrality (Brandes algorithm)
- `closeness_centrality` — Closeness centrality
- `eigenvector_centrality` — Eigenvector centrality
- `pagerank` — PageRank centrality
- `clustering` — Local clustering coefficient
- `communities` — Community detection (Louvain)

Additional measures via NetworkX (use `network.core_network` with any `nx.*_centrality` function):

- **Katz centrality** — Weighted count of walks (influence via connections)
- **Load centrality** — Traffic load through each node
- **Harmonic centrality** — Average inverse distance to all nodes
- **Current flow betweenness** — Betweenness based on electrical current flow
- **Percolation centrality** — Importance during network cascades
- **Subgraph centrality** — Based on counting closed walks
- And 24+ more available in NetworkX

See NetworkX documentation at https://networkx.org/documentation/stable/reference/algorithms/centrality.html and *Appendix E: Extended API and DSL Reference* (page 169) for complete API reference.

8.4.1 Multilayer PageRank

PageRank extends to multilayer networks by defining random walks that can jump between layers:

$$\pi_i^r = (1-d)\frac{1}{N \cdot L} + d \sum_{j,s} \frac{A_{ji}^{rs}}{k_j^s} \pi_j^s$$

Where π_i^r is the PageRank of node i in layer r, d is the damping factor (typically 0.85), and A_{ji}^{rs} accounts for both intra-layer and inter-layer edges.

Usage:

```
from py3plex.algorithms.multilayer_algorithms.centrality import MultilayerCentrality

calc = MultilayerCentrality(network)
pagerank_scores = calc.compute_pagerank(alpha=0.85)
```

8.4.2 Degree Centrality

In multilayer networks, degree centrality distinguishes between:

- **Intra-layer degree:** Number of connections within a single layer
- **Inter-layer degree:** Number of connections to other layers
- **Supra-degree:** Total degree across all layers
- **Overlapping degree:** Node-level aggregation ignoring layer distinctions

The **participation coefficient** measures how evenly a node's connections are distributed across layers:

$$P_i = \frac{L}{L-1}\left(1 - \sum_{r=1}^{L}\left(\frac{k_i^r}{k_i}\right)^2\right)$$

Where k_i^r is degree in layer *r* and k_i is total degree. Values near 1 indicate even distribution (hub-like behavior), values near 0 indicate concentration in one layer.

8.4.3 Betweenness Centrality

Multilayer betweenness counts shortest paths through the supra-adjacency graph:

$$BC(i,r) = \sum_{s \neq i, r \neq t, l} \frac{\sigma_{(s,k),(t,l)}(i,r)}{\sigma_{(s,k),(t,l)}}$$

Where $\sigma_{(s,k),(t,l)}$ is the number of shortest paths from node-layer *(s,k)* to *(t,l)*, and $\sigma_{(s,k),(t,l)}(i,r)$ counts paths passing through *(i,r)*.

Time complexity: O(n^3) for dense networks, O(nm) for sparse networks using Brandes' algorithm.

8.4.4 Explainable Centrality

Explainable centrality decomposes a node's centrality score by layer contribution, revealing which layers drive importance:

```
from py3plex.algorithms.centrality.explain import explain_node_centrality

# Get layer-wise breakdown of degree centrality
explanation = explain_node_centrality(network, 'Alice', measure='degree')

# Output:
# Layer 'social': degree = 12 (60%)
# Layer 'work':   degree = 8  (40%)
# Total degree:   20
```

This helps answer: "Is Alice influential because of social connections, work connections, or both?"

8.5 Dynamics and Processes

py3plex provides comprehensive support for simulating dynamical processes on multilayer networks, including epidemic models and random walks.

For complete documentation, see the user guide section on multilayer dynamics and the SIR epidemic simulator documentation.

8.5.1 Random Walks

Random walks simulate exploration and diffusion on networks:

```
from py3plex.dynamics import RandomWalkDynamics

# Create random walk
walk = RandomWalkDynamics(network, start_node=0, lazy_probability=0.1)
walk.set_seed(42)
results = walk.run(steps=1000)
```

```
# Analyze trajectory
trajectory = results.get_measure("trajectory")
```

Random walks are fundamental to PageRank, community detection, and network embeddings (Node2Vec, DeepWalk).

8.5.2 Epidemic Models

Simulate disease spread and information diffusion with SIR and SIS models:

```
from py3plex.dynamics import SIRDynamics, SISDynamics

# SIR: Susceptible-Infected-Recovered (with immunity)
sir = SIRDynamics(network, beta=0.3, gamma=0.1, initial_infected=0.1)
sir.set_seed(42)
results = sir.run(steps=100)

# Extract measures
prevalence = results.get_measure("prevalence")
state_counts = results.get_measure("state_counts")

print(f"Peak prevalence: {prevalence.max():.2%}")
print(f"Final recovered: {state_counts['R'][-1]}")
```

Key parameters:

- $\beta \in (0, 1)$: Transmission probability per contact (infection probability per contact)
- $\gamma \in (0, 1)$: Recovery probability per time step
- $R_0 = \frac{\beta}{\gamma}\langle k \rangle$: Basic reproduction number for homogeneous networks, with epidemic threshold at $R_0 = 1$

Models:

- **SIR** — Epidemic with immunity (measles, chickenpox). Always dies out eventually.
- **SIS** — Endemic disease without immunity (common cold, malware). Reaches stable equilibrium.

8.5.3 Multilayer Dynamics

Infection spreads through multiple interaction channels:

```
# Two-layer network: physical + digital contacts
network = multinet.multi_layer_network(directed=False)

# Add physical layer (local connections)
# Add digital layer (global connections)

# Run dynamics
sir = SIRDynamics(network, beta=0.3, gamma=0.1)
results = sir.run(steps=100)
```

The multilayer structure allows simultaneous spread through different contexts, often leading to faster and more extensive diffusion than single-layer networks.

8.6 Algorithm Complexity and Scaling

Performance characteristics:

Table 1: Algorithm Complexity Summary

Algorithm	Time Complexity	Memory	Scales to
Louvain	O(n log n)	O(n + m)	100K nodes
Label Propagation	O(m + n)	O(n)	1M+ nodes
Degree Centrality	O(n + m)	O(n)	10M+ nodes
Betweenness Centrality	O(nm)	$O(n^2)$	10K nodes
PageRank	O(m) per iteration	O(n)	1M+ nodes
SIR Dynamics	O(t·m)	O(n)	100K nodes

When to use each family:

- **Community detection:** When you need to understand group structure and identify functional modules. Use early in exploratory analysis.
- **Centrality measures:** When identifying key nodes for targeting, removal, or intervention strategies. Fast for large networks if using degree-based measures.
- **Dynamics:** When modeling spreading processes, cascades, or temporal evolution. Essential for epidemiology, information diffusion, and failure analysis.

Memory considerations:

- Betweenness centrality requires $O(n^2)$ memory for path counting; use sampling for networks > 10K nodes
- PageRank and degree-based measures scale well with sparse matrices
- Dynamics simulations require O(n) memory per time step if storing full state history

8.7 Summary

This chapter introduced three core algorithm families for multilayer network analysis:

1. **Community detection** reveals group structure using modularity optimization (Louvain), flow-based methods (Infomap), or label propagation. Choose based on network size and interpretability needs.
2. **Centrality measures** identify important nodes using degree, paths, random walks, or eigenvector-based approaches. Multilayer variants account for cross-layer influence.
3. **Dynamics and processes** simulate epidemic spread, diffusion, and random walks on multilayer networks, capturing how layer structure affects spreading patterns.

All algorithms integrate with the py3plex DSL for concise, expressive workflows. The next chapter introduces the DSL and shows how to chain queries for complex analyses.

8.8 Uncertainty Quantification

Py3plex provides tools for quantifying uncertainty in network analysis through bootstrap sampling and null model comparisons. See `examples/network_analysis/` for complete examples.

Example 1: UQ-Enabled Centrality

See `examples/network_analysis/example_dsl_uq_ergonomics.py`:

```
# Using uv
uv run examples/network_analysis/example_dsl_uq_ergonomics.py

# Or using python
python examples/network_analysis/example_dsl_uq_ergonomics.py
```

Example 2: Bootstrap Sampling

See `examples/network_analysis/example_dsl_uncertainty_bootstrap_nullmodel.py`:

```
# Using uv
uv run examples/network_analysis/example_dsl_uncertainty_bootstrap_nullmodel.py

# Or using python
python examples/network_analysis/example_dsl_uncertainty_bootstrap_nullmodel.py
```

Example 3: UQ vs Deterministic Comparison

See `examples/network_analysis/example_selection_uq_topk.py`:

```
# Using uv
uv run examples/network_analysis/example_selection_uq_topk.py

# Or using python
python examples/network_analysis/example_selection_uq_topk.py
```

These tools help assess sensitivity and uncertainty in observed network metrics; they do not by themselves establish causal claims.

See also

Further Reading:

- Community detection details: `docfiles/user_guide/community_detection.rst`
- Centrality implementations: `docfiles/tutorials/multilayer_centrality.rst`
- Dynamics API: `docfiles/sir_epidemic_simulator.rst`
- Algorithm reference: `docfiles/algorithm_guide.rst`

CHAPTER

NINE

INTRODUCTION TO THE PY3PLEX DSL

This chapter introduces the py3plex Domain-Specific Language (DSL), a SQL-like query language for expressing multilayer network analyses concisely. The focus here is the core workflow every reader should master before moving to advanced features.

DSL at a Glance

The DSL provides intuitive SQL-like syntax for network queries:

```
   from py3plex.dsl import Q, L

   # Simple: Find high-degree nodes
   result = Q.nodes().where(degree__gt=5).execute(network)

    # Multilayer analysis with export
   result = (
       Q.nodes()
        .from_layers(L["social"] + L["work"])
        .where(degree__gt=3)
        .compute("betweenness_centrality", "pagerank")
        .order_by("-betweenness_centrality")
        .limit(20)
        .execute(network)
   )

   # Export to CSV
   df = result.to_pandas()  # to_pandas() returns a pandas DataFrame
   df.to_csv("top_influencers.csv", index=False)

Express common multilayer analyses in readable, composable steps.
```

9.1 Why a DSL for Networks?

Traditional network analysis requires writing explicit loops and conditionals:

```
# Traditional approach: verbose and error-prone
high_degree_nodes = []
for node in network.get_nodes():
    if node[1] == 'social':  # Check layer
        degree = network.core_network.degree(node)
```

```
        if degree > 5:
            high_degree_nodes.append(node)

# Compute centrality for filtered nodes
G = network.core_network  # NetworkX graph
centralities = nx.betweenness_centrality(G)
filtered_centralities = {node: centralities[node] for node in high_degree_nodes}
```

The DSL provides a declarative alternative:

```
# DSL approach: clear and concise
from py3plex.dsl import execute_query

result = execute_query(network,
    'SELECT nodes WHERE layer="social" AND degree > 5 '
    'COMPUTE betweenness_centrality'
)
```

Benefits:

1. **Declarative** — State *what* you want, not *how* to get it
2. **Readable** — SQL-like syntax is familiar and self-documenting
3. **Composable** — Build complex queries from simple building blocks
4. **Maintainable** — Less code means fewer bugs

9.2 DSL Versions

Py3plex provides two DSL interfaces:

String DSL (v1)
: SQL-like queries as strings. Good for simple queries and REPL exploration.

```
result = execute_query(network, 'SELECT nodes WHERE degree > 5')
```

Builder API (v2, recommended)
: Python-native chainable API with type hints and autocompletion.

```
from py3plex.dsl import Q, L

result = (
    Q.nodes()
     .from_layers(L["social"])
     .where(degree__gt=5)
     .execute(network)
)
```

This chapter focuses on the **Builder API (v2)** as it provides a better development experience. The string DSL is covered briefly for reference.

Note

Interface trade-offs (DSL vs graph_ops vs pipeline vs CLI) are covered in *Limitations and Stability Guarantees* (page 103) to keep this chapter focused on core DSL fluency.

9.3 Basic Query Structure

A typical DSL query has four parts:

1. **Target** — What to select (nodes or edges)
2. **Layer filtering** — Which layers to include (optional)
3. **Conditions** — Filter criteria (optional)
4. **Measures** — What to compute (optional)

Example:

```
from py3plex.dsl import Q, L

result = (
    Q.nodes()                          # 1. Select nodes
     .from_layers(L["social"])         # 2. Filter to social layer
     .where(degree__gt=5)              # 3. Filter by degree > 5
     .compute("betweenness_centrality")  # 4. Compute centrality
     .execute(network)                 # Execute query
)
```

9.4 Your First Query

Let's build a simple query step-by-step. See the complete examples in `examples/network_analysis/` and `examples/dsl_zoo/`.

Example 1: Filter by layer

```
from py3plex.dsl import Q, L
from py3plex.core import multinet

# Create a simple multilayer network
net = multinet.multi_layer_network()
net.add_nodes([  # add_nodes uses 'type'; add_edges uses 'source_type'/'target_type'
    {'source': 'Alice', 'type': 'social'},
    {'source': 'Bob', 'type': 'social'},
    {'source': 'Charlie', 'type': 'work'},
])
net.add_edges([
    {'source': 'Alice', 'target': 'Bob',
     'source_type': 'social', 'target_type': 'social'},
    {'source': 'Charlie', 'target': 'Alice',
     'source_type': 'work', 'target_type': 'work'},
])

# Select nodes from the social layer
result = (
    Q.nodes()
```

```
    .from_layers(L["social"])
    .execute(net)
)

print(result.to_pandas())
```

Run complete DSL examples:

```
# Basic DSL builder API
python examples/network_analysis/example_dsl_builder_api.py

# Advanced DSL features
python examples/network_analysis/example_dsl_advanced.py

# Query Zoo examples
pytest tests/test_dsl_query_zoo.py -q
```

Example 2: Filter by degree

```
# Find high-degree nodes
result = (
    Q.nodes()
     .from_layers(L["*"])  # All layers (same as omitting .from_layers(...))
     .compute("degree")
     .where(degree__gt=3)
     .execute(net)
)
```

Example 3: Compute centrality

See `examples/network_analysis/example_dsl_builder_api.py`:

```
# Using uv
uv run examples/network_analysis/example_dsl_builder_api.py

# Or using python
python examples/network_analysis/example_dsl_builder_api.py
```

9.5 Layer Filtering

The DSL provides layer algebra operations. See `examples/network_analysis/example_dsl_layer_algebra.py` for complete examples.

DSL Layer Algebra

Layer algebra lets you combine layers with set operations:

```
from py3plex.dsl import Q, L

# Union: nodes in social OR work
Q.nodes().from_layers(L["social"] + L["work"])

# Difference: nodes in social but NOT bots
```

```
Q.nodes().from_layers(L["social"] - L["bots"])

# Intersection: nodes in both social AND work
Q.nodes().from_layers(L["social"] & L["work"])

# Complex: (social OR work) - bots
Q.nodes().from_layers(L["social"] + L["work"] - L["bots"])
```

This makes multilayer queries intuitive and expressive!

Run layer algebra examples:

```
# Using uv
uv run examples/network_analysis/example_dsl_layer_algebra.py

# Or using python
python examples/network_analysis/example_dsl_layer_algebra.py
```

9.5.1 Simple Layer Selection

```
# Single layer
Q.nodes().from_layers(L["social"])

# Multiple layers (union)
Q.nodes().from_layers(L["social"] + L["work"])
```

9.5.2 Layer Algebra

Union (OR):

```
# Nodes in social OR work layers
Q.nodes().from_layers(L["social"] + L["work"])
```

Difference (NOT):

```
# Nodes in social but NOT in bots layer
Q.nodes().from_layers(L["social"] - L["bots"])
```

Intersection (AND):

```
# Nodes present in both social AND work
Q.nodes().from_layers(L["social"] & L["work"])
```

All layers:

```
# Don't specify from_layers() to query all layers
Q.nodes().where(degree__gt=5)
```

9.6 Filtering Conditions

The `where()` method accepts filtering predicates.

9.6.1 Degree Filters

```
# Greater than
Q.nodes().where(degree__gt=5)

# Greater than or equal
Q.nodes().where(degree__gte=3)

# Less than
Q.nodes().where(degree__lt=10)

# Less than or equal
Q.nodes().where(degree__lte=5)

# Equal
Q.nodes().where(degree=2)
```

9.6.2 Layer Filters

```
# Specific layer
Q.nodes().where(layer="social")

# Exclude layer
Q.nodes().where(layer__ne="bots")
```

9.6.3 Multiple Conditions (AND)

```
# Combine conditions
Q.nodes().where(layer="social", degree__gt=3)

# Equivalent to: layer="social" AND degree > 3
```

9.7 Computing Measures

The `compute()` method calculates network measures.

9.7.1 Single Measure

```
result = (
    Q.nodes()
     .compute("degree")
     .execute(network)
)

# Access results
for node, degree in result.measures['degree'].items():
    print(f"{node}: {degree}")
```

9.7.2 Multiple Measures

```
result = (
    Q.nodes()
     .compute("degree", "betweenness_centrality", "closeness_centrality")
     .execute(network)
)

# Access each measure
degrees = result.measures['degree']
betweenness = result.measures['betweenness_centrality']
closeness = result.measures['closeness_centrality']
```

9.7.3 Available Measures

Measure	Description
`degree`	Node degree (number of neighbors)
`degree_centrality`	Normalized degree centrality
`betweenness_centrality`	Betweenness centrality (path-based importance)
`closeness_centrality`	Closeness centrality (average distance to others)
`eigenvector_centrality`	Eigenvector centrality (importance of neighbors)
`pagerank`	PageRank score
`clustering`	Clustering coefficient

9.8 Ordering and Limiting

9.8.1 Order By

```
# Order by degree (ascending)
Q.nodes().compute("degree").order_by("degree")

# Order by degree (descending)
Q.nodes().compute("degree").order_by("-degree")

# Order by multiple keys
Q.nodes().compute("degree").order_by("-degree", "node_id")
```

9.8.2 Limit

```
# Get top 10 by degree
result = (
    Q.nodes()
     .compute("degree")
     .order_by("-degree")
     .limit(10)
     .execute(network)
)
```

9.9 Working with Results

The `QueryResult` object provides multiple ways to access results.

9.9.1 Iteration

```
result = Q.nodes().execute(network)

for node in result:
    print(node)  # ('Alice', 'friends'), ...
```

9.9.2 Count

```
result = Q.nodes().where(degree__gt=5).execute(network)
print(f"Found {result.count} nodes")
```

9.9.3 Measures

```
result = Q.nodes().compute("degree").execute(network)

# Access as dict
degrees = result.measures['degree']
for node, degree in degrees.items():
    print(f"{node}: {degree}")
```

9.9.4 To Pandas

```
# Convert to DataFrame
df = result.to_pandas()
print(df.head())
```

9.10 String DSL Syntax (Reference)

For completeness, here's the string DSL syntax. See `examples/network_analysis/example_dsl_queries.py` for a complete example.

Basic queries:

```
from py3plex.dsl import execute_query

# Select nodes
execute_query(network, 'SELECT nodes')

# Filter by layer
execute_query(network, 'SELECT nodes WHERE layer="social"')

# Filter by degree
execute_query(network, 'SELECT nodes WHERE degree > 5')

# Multiple conditions
execute_query(network, 'SELECT nodes WHERE layer="social" AND degree > 3')
```

```
# Compute measures
execute_query(network, 'SELECT nodes COMPUTE betweenness_centrality')
```

Run legacy DSL examples:

```
# Using uv
uv run examples/network_analysis/example_dsl_queries.py

# Or using python
python examples/network_analysis/example_dsl_queries.py
```

Operators:

- `=` — Equal
- `!=` — Not equal
- `>` — Greater than
- `<` — Less than
- `>=` — Greater or equal
- `<=` — Less or equal
- `AND` — Logical AND
- `OR` — Logical OR
- `NOT` — Logical NOT

9.11 Example: Complete Workflow

Here's a complete example demonstrating DSL capabilities:

```
from py3plex.core import multinet
from py3plex.dsl import Q, L

# Create network
network = multinet.multi_layer_network()
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1],
    ['Bob', 'friends', 'Carol', 'friends', 1],
    ['Carol', 'friends', 'Dave', 'friends', 1],
    ['Alice', 'colleagues', 'Bob', 'colleagues', 1],
    ['Bob', 'colleagues', 'Eve', 'colleagues', 1],
    ['Eve', 'colleagues', 'Frank', 'colleagues', 1],
], input_type="list")

# Query 1: Find high-degree nodes across all layers
result = (
    Q.nodes()
     .where(degree__gte=2)
     .compute("degree", "betweenness_centrality")
     .order_by("-betweenness_centrality")
     .execute(network)
```

```
)

print("High-degree nodes:")
for node in result:
    degree = result.measures['degree'][node]
    bc = result.measures['betweenness_centrality'][node]
    print(f"  {node}: degree={degree}, BC={bc:.3f}")

# Query 2: Compare layers
for layer in ['friends', 'colleagues']:
    result = (
        Q.nodes()
         .from_layers(L[layer])
         .compute("degree")
         .execute(network)
    )
    avg_degree = sum(result.measures['degree'].values()) / result.count
    print(f"{layer} layer: {result.count} nodes, avg degree={avg_degree:.2f}")
```

9.12 Common Patterns

9.12.1 Pattern 1: Filter → Compute → Order → Limit

```
# Find top 10 central nodes in a specific layer
top_10 = (
    Q.nodes()
     .from_layers(L["social"])
     .compute("betweenness_centrality")
     .order_by("-betweenness_centrality")
     .limit(10)
     .execute(network)
)
```

9.12.2 Pattern 2: Layer Comparison

```
# Compare degree distribution across layers
for layer_name in network.get_layers():
    result = (
        Q.nodes()
         .from_layers(L[layer_name])
         .compute("degree")
         .execute(network)
    )
    degrees = list(result.measures['degree'].values())
    print(f"{layer_name}: mean degree = {sum(degrees)/len(degrees):.2f}")
```

9.12.3 Pattern 3: Conditional Export

```
# Export high-degree nodes to CSV
result = (
    Q.nodes()
```

```
    .where(degree__gt=5)
    .compute("degree", "betweenness_centrality")
    .order_by("-degree")
    .execute(network)
)
result.to_pandas().to_csv("high_degree_nodes.csv", index=False)
```

9.13 Closing Note

At this point, you should be able to read and write core DSL queries without guessing their execution intent. The next chapter focuses on explain plans, parameterization, and diagnostic tooling for debugging and reuse.

9.14 Further Reading

- The Builder API and Explain Plans (Chapter 9)
- Advanced Queries and Workflows (Chapter 10)
- `examples/dsl_zoo/` — Focused query patterns
- `examples/network_analysis/` — End-to-end DSL workflows

CHAPTER

TEN

THE BUILDER API AND EXPLAIN PLANS

The DSL Builder API provides a Pythonic, type-safe way to construct network queries using method chaining. This chapter covers the builder pattern, query execution plans, and advanced features like parameterization and query reuse.

10.1 Builder Pattern and Fluent API

The builder API uses method chaining to construct queries incrementally. Each method returns the query object, allowing you to chain operations naturally.

See `examples/network_analysis/example_dsl_builder_api.py` for a complete example:

```python
from py3plex.dsl import Q, L
from py3plex.core import multinet

# Create network
net = multinet.multi_layer_network()
# ... add nodes and edges ...

# Build query with method chaining
result = (
    Q.nodes()
     .from_layers(L["social"])
     .where(degree__gt=3)
     .compute("betweenness_centrality")
     .order_by("-betweenness_centrality")
     .limit(10)
     .execute(net)
)
```

Run this example:

```
# Basic builder API examples
python examples/network_analysis/example_dsl_builder_api.py

# Advanced DSL features
python examples/network_analysis/example_dsl_advanced.py
```

Advantages over string DSL:

- **Type safety:** IDE autocomplete and type checking
- **Composability:** Build queries programmatically
- **Readability:** Self-documenting code

- **Error detection:** Catch mistakes before execution

10.1.1 Method Chaining

Build queries step-by-step:

```
# Start with a base query
q = Q.nodes()

# Add filters
q = q.where(layer="social")
q = q.where(degree__gt=3)

# Add computation
q = q.compute("degree", "clustering")

# Sort and limit
q = q.order_by("-degree")
q = q.limit(20)

# Execute
result = q.execute(network)
```

Builder operators:

Operator	DSL Equivalent
`where(degree__gt=5)`	`WHERE degree > 5`
`where(degree__gte=5)`	`WHERE degree >= 5`
`where(degree__lt=5)`	`WHERE degree < 5`
`where(degree__lte=5)`	`WHERE degree <= 5`
`where(layer="social")`	`WHERE layer = "social"`
`where(layer__in=["a", "b"])`	`WHERE layer IN ("a", "b")`

10.1.2 Type Hints and IDE Support

The builder API is fully type-hinted for modern IDEs:

```
from py3plex.dsl import Q, L

# IDE shows available methods and parameters
result = (
    Q.nodes()           # Autocomplete suggests: where, from_layers, compute, ...
     .where(            # Autocomplete shows: degree__gt, degree__lt, layer, ...
         degree__gt=5   # Type hints ensure correct parameter types
     )
     .compute(          # Autocomplete lists available measures
         "degree",      # String literals with validation
         "betweenness_centrality"
     )
     .execute(network)  # Type checker ensures network is correct type
)
```

Benefits:

- Catch typos before runtime
- Discover available methods and measures
- Understand query structure through types
- Safer refactoring with IDE support

10.2 EXPLAIN Mode

10.2.1 Query Execution Plans

Get a query execution plan without actually running the query:

```
from py3plex.dsl import Q

# Build a complex query
q = (
    Q.nodes()
     .from_layers(L["social"])
     .where(degree__gt=5)
     .compute("betweenness_centrality", "pagerank")
     .order_by("-betweenness_centrality")
     .limit(10)
)

# Get execution plan
plan = q.explain().execute(network)

# Inspect the plan
print(f"Query: {plan.query}")
print(f"Estimated nodes: {plan.estimated_node_count}")

for step in plan.steps:
    print(f"  {step.description}")
    print(f"    Complexity: {step.estimated_complexity}")
    print(f"    Estimated time: {step.estimated_time_ms}ms")

# Check for warnings
for warning in plan.warnings:
    print(f"   {warning}")
```

Example output:

```
Query: SELECT nodes FROM layers social WHERE degree > 5
       COMPUTE betweenness_centrality pagerank
       ORDER BY -betweenness_centrality LIMIT 10

Estimated nodes: ~150

  1. Filter by layer
    Complexity: O(n)
    Estimated time: 2ms
```

```
  2. Filter by degree
    Complexity: O(n)
    Estimated time: 1ms

  3. Compute betweenness_centrality
    Complexity: O(n^3)
    Estimated time: 850ms
     Betweenness centrality is expensive for large graphs

  4. Compute pagerank
    Complexity: O(n^2)
    Estimated time: 45ms

  5. Sort results
    Complexity: O(n log n)
    Estimated time: 3ms

  6. Limit results
    Complexity: O(1)
    Estimated time: <1ms
```

10.2.2 Complexity Estimates

EXPLAIN mode provides complexity estimates for each operation:

- **O(1)** — Constant time (e.g., limit, basic metadata)
- **O(n)** — Linear in number of nodes (e.g., filtering by attribute)
- **O(m)** — Linear in number of edges (e.g., edge filtering)
- **O(n log n)** — Log-linear (e.g., sorting)
- **O(n^2)** — Quadratic (e.g., PageRank, all-pairs measures)
- **O(n^3)** — Cubic (e.g., betweenness centrality on large graphs)

Use EXPLAIN to:

1. **Identify bottlenecks** before executing expensive queries
2. **Compare alternative formulations** of the same query
3. **Estimate runtime** for large networks
4. **Optimize query order** (apply cheap filters first)

10.2.3 Optimization Tips

1. Apply filters before computing measures

```python
#  Slow: Compute for all, then filter
Q.nodes().compute("betweenness_centrality").where(degree__gt=5)

#  Fast: Filter first, then compute
Q.nodes().where(degree__gt=5).compute("betweenness_centrality")
```

2. Use layer filtering early

```
#  Slow: Compute for all layers
Q.nodes().compute("degree").where(layer="social")

#  Fast: Filter to layer first
Q.nodes().from_layers(L["social"]).compute("degree")
```

3. Limit results when possible

```
# Top 10 is much faster than computing for all nodes
Q.nodes().compute("degree").order_by("-degree").limit(10)
```

4. Choose efficient measures

- **Cheap:** degree, clustering coefficient (local measures)
- **Moderate:** PageRank, eigenvector centrality (iterative)
- **Expensive:** betweenness centrality, closeness centrality (all-pairs)

10.3 Advanced Builder Features

10.3.1 Interactive Query Building with Hints

The `hint()` method provides context-aware suggestions for next steps when building queries. It analyzes the current query state and suggests relevant methods:

```
from py3plex.dsl import Q, L

# Start building a query
q = Q.nodes().from_layers(L["social"])

# Get hints for what to do next
q.hint()
# Output:
# ========================================
#  Query Builder Hints
# ========================================
#
#  Current query state:
#   ✓ Layers selected
#
#  Suggested next steps:
#
# 1. → .where(degree__gt=3)  # Filter nodes by attributes
# 2. → .compute("degree", "betweenness_centrality")  # Compute metrics
# 3. → .per_layer()  # Group results by layer
# 4. → .execute(network)  # Execute the query

# Continue building
q = q.where(degree__gt=3)
q.hint()  # Will suggest different options based on new state
```

Hints adapt to your query context:

- After `where()`: Suggests `compute()`, `per_layer()`, `uq()`

- After `compute()`: Suggests `order_by()`, `limit()`, `explain()`
- After `per_layer()`: Suggests `aggregate()`, `coverage()`, `end_grouping()`
- After grouping: Suggests `top_k()`, `coverage()`

Key features:

- **Non-invasive**: Only displays information, doesn't modify query
- **Context-aware**: Suggestions change based on current state
- **Educational**: Includes examples and method signatures
- **Chainable**: Returns self, so you can chain `.hint().execute(net)`

10.3.2 Understanding QueryResult Objects

When you execute a query, py3plex returns a `QueryResult` object with rich metadata:

```
result = (
    Q.nodes()
     .from_layers(L["social"])
     .compute("degree", "betweenness_centrality")
     .per_layer()
     .uq(method="bootstrap", n_samples=100)
     .execute(network)
)

# Rich representation shows full context
print(result)
# Output:
# QueryResult(
#   target='nodes'
#   count=42
#   attributes=['degree', 'betweenness_centrality']
#   computed=['degree', 'betweenness_centrality']
#   grouping='per_layer' (3 groups)
#   uncertainty='bootstrap'
#   provenance='replayable'
#   replayable=True
# )
```

Reading a QueryResult:

1. **target**: What was queried ('nodes', 'edges', 'communities')
2. **count**: Number of items returned
3. **attributes**: Computed attributes/metrics available
4. **computed**: Metrics explicitly computed (vs. pre-existing)
5. **grouping**: If grouped (per_layer, per_layer_pair), shows type and count
6. **uncertainty**: UQ method if enabled (bootstrap, perturbation, etc.)
7. **provenance**: Provenance mode (none, tracked, replayable)
8. **replayable**: Whether query can be replayed deterministically

Before calling .to_pandas():

Always inspect the QueryResult to understand what you have:

```
#  Don't immediately convert without inspecting
df = result.to_pandas()

#  Inspect first to understand the data
print(result)  # See what's in the result
print(f"Got {result.count} {result.target}")
print(f"Computed: {', '.join(result.computed_metrics)}")

# Then convert with appropriate options
if result.meta.get("has_uncertainty"):
    df = result.to_pandas(expand_uncertainty=True)
else:
    df = result.to_pandas()
```

QueryResult introspection methods:

- `.count` / `len(result)`: Number of items
- `.computed_metrics`: Set of computed metrics
- `.provenance`: Provenance dictionary
- `.is_replayable`: Whether result can be replayed
- `.has_sensitivity`: Whether sensitivity analysis was run
- `.group_summary()`: Summary DataFrame for grouped results
- `.explain()`: Human-readable query explanation
- `.debug()`: Detailed debug information

10.3.3 Parameterized Queries

Use `Param` to create reusable query templates:

```
from py3plex.dsl import Q, Param

# Create a parameterized query template
influencer_query = (
    Q.nodes()
     .from_layers(L[Param.str("target_layer")])
     .where(degree__gt=Param.int("min_degree"))
     .compute("betweenness_centrality")
     .order_by("-betweenness_centrality")
     .limit(Param.int("top_n"))
)

# Execute with different parameters
social_influencers = influencer_query.execute(
    network,
    target_layer="social",
    min_degree=10,
    top_n=20
)
```

```
work_influencers = influencer_query.execute(
    network,
    target_layer="professional",
    min_degree=5,
    top_n=50
)
```

Parameter types:

- `Param.int("name")` — Integer parameter
- `Param.float("name")` — Float parameter
- `Param.str("name")` — String parameter
- `Param.ref("name")` — Untyped reference

Benefits:

- **Safety:** Type-checked at execution time
- **Reusability:** One query definition, many executions
- **Maintainability:** Parameters are self-documenting
- **Performance:** Query is parsed once, executed multiple times

10.3.4 Query Reuse

Build queries once and execute them multiple times:

```
# Define a query once
high_degree_analysis = (
    Q.nodes()
     .where(degree__gt=10)
     .compute("betweenness_centrality", "clustering")
)

# Execute on different networks
result1 = high_degree_analysis.execute(network1)
result2 = high_degree_analysis.execute(network2)
result3 = high_degree_analysis.execute(network3)

# Or execute with different parameters over time
for threshold in [5, 10, 15, 20]:
    q = Q.nodes().where(degree__gt=threshold).compute("degree")
    result = q.execute(network)
    print(f"Threshold {threshold}: {result.count} nodes")
```

10.3.5 Converting to DSL String

Convert a builder query back to string format:

```
q = (
    Q.nodes()
     .from_layers(L["social"])
     .where(degree__gt=5)
```

```
    .compute("degree")
    .limit(10)
)

# Get DSL string representation
dsl_string = q.to_dsl()
print(dsl_string)
# Output: SELECT nodes FROM layers social WHERE degree > 5
#         COMPUTE degree LIMIT 10
```

Use cases:

- **Logging:** Record queries in application logs
- **Debugging:** Inspect query structure
- **Serialization:** Save queries to files or databases
- **Auditing:** Track what analyses were run

10.3.6 Error Handling with Suggestions

The DSL provides helpful error messages with suggestions:

```
from py3plex.dsl import Q, UnknownMeasureError

try:
    # Typo in measure name
    result = Q.nodes().compute("betweenes").execute(network)
except UnknownMeasureError as e:
    print(e)
    # Output: Unknown measure 'betweenes'. Did you mean 'betweenness_centrality'?
    #         Known measures: betweenness_centrality, closeness_centrality, ...
```

10.4 Summary

The DSL Builder API provides:

- **Pythonic interface:** Natural method chaining
- **Type safety:** IDE support and autocomplete
- **Query plans:** EXPLAIN mode for optimization
- **Parameterization:** Reusable query templates
- **Error handling:** Helpful "did you mean?" messages

Best practices:

1. Use builder API for programmatic queries
2. Run EXPLAIN before executing expensive queries
3. Filter early, compute late
4. Parameterize queries that you'll reuse
5. Use type hints and IDE support to catch errors

Next chapter: Advanced query patterns and workflow integration

CHAPTER

ELEVEN

ADVANCED QUERIES AND WORKFLOWS

This chapter covers advanced DSL patterns including dynamics simulation, complex query patterns, and workflow integration.

11.1 DSL-Based Dynamics Simulation

The py3plex DSL includes a framework for declarative dynamics simulation on multilayer networks. This section demonstrates how to use the dynamics DSL for epidemic modeling, random walks, and other dynamical processes.

11.1.1 Mathematical Formalism

SIS Model (Susceptible-Infected-Susceptible)

The SIS model tracks disease spread with possible reinfection:

- **States:** S (susceptible), I (infected)
- **Update rules** (discrete time, node i):
  - $P(S_i \to I_i) = 1 - (1 - \beta)^{\sum_j A_{ij} I_j(t)}$
  - $P(I_i \to S_i) = \gamma$
- **Parameters:**
  - $\beta \in (0, 1)$: transmission probability per contact
  - $\gamma \in (0, 1)$: recovery probability per time step (also denoted μ)
- **Epidemic threshold:**
  - $R_0 = \frac{\beta}{\gamma} \lambda_{\max}(A)$ where $\lambda_{\max}(A)$ is the largest eigenvalue of the adjacency matrix. Endemic equilibria exist when $R_0 > 1$.

SIR Model (Susceptible-Infected-Recovered)

The SIR model includes permanent immunity after recovery:

- **States:** S (susceptible), I (infected), R (recovered/removed)
- **Update rules** (discrete time, node i):
  - $P(S_i \to I_i) = 1 - (1 - \beta)^{\sum_j A_{ij} I_j(t)}$
  - $P(I_i \to R_i) = \gamma$
- **Parameters:**
  - $\beta \in (0, 1)$: transmission probability per contact
  - $\gamma \in (0, 1)$: recovery probability per time step

The final outbreak size (attack rate) depends on the basic reproduction number $R_0 = \frac{\beta}{\gamma}\langle k \rangle$, where $\langle k \rangle$ is the mean degree. Epidemic threshold at $R_0 = 1$.

Random Walk

Random walk dynamics model diffusion processes:

- **State:** Current node position
- **Update rule:** Move to neighbor j with probability 1/degree(i), or stay at i with probability p_lazy (for lazy walks)
- **Parameters:**
  - p_teleport: probability of random teleportation (default: 0.05)
  - p_lazy: probability of staying at current node (default: 0.0)

11.1.2 Basic Dynamics DSL Usage

The dynamics DSL uses a builder API similar to the query DSL. See `examples/dynamics/` for complete examples.

Example 1: SIS Epidemic Model

See `examples/dynamics/sis_dynamics.py`:

```
python examples/dynamics/sis_dynamics.py
```

Example 2: SIR Epidemic Model

See `examples/dynamics/sir_epidemic.py`:

```
python examples/dynamics/sir_epidemic.py
```

Example 3: Random Walk Dynamics

See `examples/dynamics/random_walk.py`:

```
python examples/dynamics/random_walk.py
```

Accessing simulation results:

```
# result.data is a dict of numpy arrays mapping measure names to time-series data
# Each array has shape (num_runs, num_timesteps)
print(f"Mean final prevalence: {result.data['prevalence'][:, -1].mean():.3f}")

# Convert to pandas DataFrames for analysis
df_dict = result.to_pandas()
prevalence_df = df_dict['prevalence']  # DataFrame with columns for each timestep
```

Key components:

1. **D.process()** — Specify the dynamical process (SIS, SIR, RandomWalk)
2. **.initial()** — Set initial conditions
3. **.steps()** — Number of time steps to simulate
4. **.measure()** — Metrics to track during simulation
5. **.replicates()** — Number of independent runs (for averaging)
6. **.seed()** — Random seed for reproducibility
7. **.run()** — Execute the simulation

11.1.3 Initial Conditions

The `.initial()` method accepts multiple formats for setting initial infected nodes:

Float (fraction of nodes):

```
.initial(infected=0.05)  # 5% of nodes randomly infected
```

Integer (exact count):

```
.initial(infected=5)  # Exactly 5 nodes randomly infected
```

List of node tuples:

```
.initial(infected=[('Alice', 'social'), ('Bob', 'work')])
```

DSL query (dynamic selection):

```
.initial(infected=Q.nodes().where(degree__gte=5))  # Infect high-degree hubs
```

Recommended: Use float fractions (0.05) for reproducibility across different network sizes.

11.1.4 Available Measures

Different processes support different measures:

Process	Measure	Description
SIS	prevalence	Fraction of infected nodes at each time step
SIS	incidence	Number of new infections at each time step
SIS	prevalence_by_layer	Prevalence tracked separately per layer
SIR	prevalence	Fraction of infected nodes
SIR	state_counts	Counts of nodes in each state (S, I, R)
RandomWalk	visit_frequency	Frequency of visits to each node
RandomWalk	state_counts	Current position over time

11.1.5 Multilayer Dynamics

The DSL supports multilayer networks with coupling between layers:

```
from py3plex.dsl import L

# Simulate on specific layers
sim = (
    D.process(SIS(beta=0.25, mu=0.08))
     .on_layers(L["offline"] + L["online"])  # Select layers
     .coupling(node_replicas="strong")       # Shared node states
     .initial(infected=0.1)
     .steps(120)
     .measure("prevalence", "prevalence_by_layer")
     .replicates(15)
)

result = sim.run(multilayer_network)
```

Coupling modes:

- **"strong"** — Node states shared across all layers (default)
- **"independent"** — Each layer has independent node states
- **"weak"** — Partial coupling with cross-layer transmission rate

11.1.6 Integration with Query DSL

The dynamics DSL integrates seamlessly with the query DSL for targeted initial conditions:

```
from py3plex.dsl import Q

# Start infection at high-degree nodes (hubs)
sim = (
    D.process(SIS(beta=0.35, mu=0.12))
     .initial(
         infected=Q.nodes().where(degree__gte=5)  # Query for hubs
     )
     .steps(100)
     .measure("prevalence")
     .replicates(10)
)

result = sim.run(network)
```

This allows precise control over initial conditions based on network structure, centrality, or other properties.

11.1.7 Parameter Comparison Example

Compare dynamics under different parameters:

```
# Compare transmission rates
beta_values = [0.2, 0.3, 0.4, 0.5]
results = {}

for beta in beta_values:
    sim = (
        D.process(SIS(beta=beta, mu=0.1))
         .initial(infected=0.05)
         .steps(80)
         .measure("prevalence")
         .replicates(20)
         .seed(42)
    )
    results[beta] = sim.run(network)

# Analyze final prevalence
for beta, result in results.items():
    mean_final = result.data['prevalence'][:, -1].mean()
    print(f"={beta:.1f}: final prevalence = {mean_final:.3f}")
```

11.1.8 Result Analysis

The `SimulationResult` object provides rich analysis capabilities:

```
# Get summary statistics
summary = result.summary()
print(summary)

# Plot time series with confidence intervals
import matplotlib.pyplot as plt
result.plot("prevalence")
plt.show()

# Export to pandas for custom analysis
df_dict = result.to_pandas()
prevalence_df = df_dict['prevalence']

# Compute mean trajectory
mean_trajectory = (
    prevalence_df
    .groupby('t')['value']
    .agg(['mean', 'std'])
)
```

Status: Dynamics DSL

The builder API (`D.process()`) is under active development and may change in future releases. The core dynamics classes (`SIRDynamics`, `SISDynamics`, `RandomWalkDynamics`) are intended to be stable for the workflows shown in this book.

11.2 Complex Query Patterns

Beyond basic filtering and computation, the DSL supports advanced analysis patterns.

11.2.1 Parameterized Comparative Analysis

Systematic comparison across layers using parameterized queries:

```
from py3plex.dsl import Q, L, Param

# Define reusable parameterized query
hub_analysis = (
    Q.nodes()
     .from_layers(L[Param.str("layer")])
     .where(degree__gt=Param.int("threshold"))
     .compute("betweenness_centrality", "clustering")
     .order_by("-betweenness_centrality")
     .limit(Param.int("top_n"))
)

# Execute for multiple layers
for layer in ["social", "work", "family"]:
    result = hub_analysis.execute(
```

```
        network,
        layer=layer,
        threshold=5,
        top_n=20
    )
    df = result.to_pandas()
    df.to_csv(f"{layer}_hubs.csv")
    print(f"{layer}: {result.count} hubs, max BC = {df['betweenness_centrality'].max():.3f}")
```

11.2.2 Multi-Layer Statistical Summaries

Aggregate statistics across all layers:

```
# Collect statistics for each layer
layer_stats = []
for layer_name in network.get_layers():
    result = (
        Q.nodes()
         .from_layers(L[layer_name])
         .compute("degree", "betweenness_centrality", "clustering")
         .execute(network)
    )
    df = result.to_pandas()

    layer_stats.append({
        'layer': layer_name,
        'node_count': result.count,
        'avg_degree': df['degree'].mean(),
        'max_betweenness': df['betweenness_centrality'].max(),
        'avg_clustering': df['clustering'].mean(),
    })

# Convert to DataFrame for analysis
import pandas as pd
summary_df = pd.DataFrame(layer_stats)
print(summary_df)
```

11.2.3 Layer Algebra for Complex Filters

Combine layers using set operations:

```
from py3plex.dsl import L

# Nodes in social OR work layers
social_or_work = (
    Q.nodes()
     .from_layers(L["social"] + L["work"])
     .compute("degree")
     .execute(network)
)

# Nodes in social BUT NOT bots
legitimate_social = (
```

```
    Q.nodes()
     .from_layers(L["social"] - L["bots"])
     .compute("degree")
     .execute(network)
)

# Intersection of layers
overlap = (
    Q.nodes()
     .from_layers(L["layer1"] & L["layer2"])
     .execute(network)
)
```

11.2.4 Aggregations by Node Attributes

Group and aggregate by node properties:

```
from py3plex.dsl import Q, L

# Group nodes by layer and compute statistics
result = (
    Q.nodes()
     .from_layers(L["*"])
     .compute("degree", "betweenness_centrality")
     .execute(network)
)

# Convert to pandas and group
df = result.to_pandas()
layer_stats = df.groupby('layer').agg({
    'degree': ['mean', 'std', 'max'],
    'betweenness_centrality': ['mean', 'max']
})
print(layer_stats)
```

See also: `examples/network_analysis/example_dsl_advanced.py` for complete grouping examples.

Additional grouping patterns:

```
# Compute average centrality by community
result = (
    Q.nodes()
     .compute("betweenness_centrality", "community")
     .execute(network)
)

df = result.to_pandas()
community_centrality = df.groupby('community')['betweenness_centrality'].agg(['mean', 'std', 'max'])
print(community_centrality)
```

11.3 Result Conversion

The DSL provides flexible export capabilities for different analysis workflows.

11.3.1 To Pandas DataFrames

Convert query results to pandas for statistical analysis:

```
# Execute query
result = (
    Q.nodes()
     .from_layers(L["social"])
     .compute("degree", "betweenness_centrality", "clustering")
     .execute(network)
)

# Convert to DataFrame
df = result.to_pandas()

# Standard pandas operations
print(df.describe())
print(df[df['degree'] > 10])
df.to_csv("results.csv", index=False)
```

DataFrame structure:

- Each row is a node
- Columns include: node_id, layer, computed measures

11.3.2 To NetworkX

Export filtered nodes as a NetworkX subgraph:

```
# Query for high-degree nodes
result = (
    Q.nodes()
     .where(degree__gt=10)
     .execute(network)
)

# Extract as NetworkX subgraph
node_ids = list(result.node_ids)
subgraph = network.core_network.subgraph(node_ids)

# Use NetworkX algorithms
import networkx as nx
communities = nx.community.louvain_communities(subgraph)
```

11.3.3 To CSV/JSON

Direct export without intermediate DataFrame:

```
# Export to CSV
result = (
```

```
    Q.nodes()
     .from_layers(L["social"])
     .compute("degree", "betweenness_centrality")
     .execute(network)
)
result.to_pandas().to_csv("social_centrality.csv", index=False)

# Export to JSON
result = (
    Q.nodes()
     .compute("degree")
     .execute(network)
)
df = result.to_pandas()
json_str = df.to_json(orient="records", indent=2)
with open("node_degrees.json", "w") as f:
    f.write(json_str)
```

Supported formats:

- CSV (with custom delimiter)

11.4 Dplyr-Style Data Manipulation

The `graph_ops` module provides dplyr-style operations (inspired by R's dplyr data manipulation library) for manipulating network data with a chainable API.

Example 1: Filter and Mutate

```
from py3plex.graph_ops import nodes

# Filter and mutate node data
df = (
    nodes(network)
    .filter(lambda n: n["degree"] > 2)
    .mutate(score=lambda n: n["degree"] * 2)
    .arrange("degree", reverse=True)
    .to_pandas()
)
print(df)
```

Example 2: Group and Summarise

```
from py3plex.graph_ops import nodes

# Group by layer and summarise
df = (
    nodes(network)
    .group_by("layer")
    .summarise(
        mean_degree=("degree", "mean"),
        max_degree=("degree", "max"),
        count=("id", "count")
    )
```

```
    .to_pandas()
)
print(df)
```

See also: `examples/network_analysis/example_dsl_dplyr_operations.py` for complete dplyr-style examples.

Example 3: Subgraph Extraction

```
from py3plex.dsl import Q, L

# Extract subgraph for high-degree nodes
subgraph = (
    Q.nodes()
     .where(degree__gt=5)
     .to_networkx()
     .execute(network)
)
uv run examples/network_analysis/example_dsl_dplyr_operations.py

# Or using python
python examples/network_analysis/example_dsl_dplyr_operations.py
```

Supported export formats:

- JSON (various orientations)
- pandas DataFrame (in-memory)

11.5 Workflow Integration

11.5.1 Pipeline Composition

Chain multiple queries in analysis pipelines:

```
# Step 1: Find high-degree nodes
hubs = (
    Q.nodes()
     .where(degree__gt=20)
     .execute(network)
)

# Step 2: Compute centrality only for hubs
hub_centrality = (
    Q.nodes()
     .where(node_id__in=list(hubs.node_ids))
     .compute("betweenness_centrality", "closeness_centrality")
     .execute(network)
)

# Step 3: Identify super-hubs
super_hubs = (
    Q.nodes()
     .where(
         node_id__in=list(hubs.node_ids),
```

```
        betweenness_centrality__gt=0.1
    )
    .execute(network)
)

print(f"Hubs: {hubs.count}, Super-hubs: {super_hubs.count}")
```

11.5.2 Combining with sklearn

Integrate DSL results with machine learning pipelines:

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# Extract features using DSL
result = (
    Q.nodes()
     .compute("degree", "betweenness_centrality", "clustering")
     .execute(network)
)

df = result.to_pandas()

# Prepare features
features = df[['degree', 'betweenness_centrality', 'clustering']].values
scaler = StandardScaler()
features_scaled = scaler.fit_transform(features)

# Cluster nodes
kmeans = KMeans(n_clusters=5, random_state=42)
df['cluster'] = kmeans.fit_predict(features_scaled)

# Analyze clusters
print(df.groupby('cluster')[['degree', 'betweenness_centrality']].mean())
```

11.5.3 Custom Measures

While the DSL provides built-in measures, you can compute custom measures using pandas:

```
# Get base measures
result = (
    Q.nodes()
     .compute("degree", "betweenness_centrality")
     .execute(network)
)

df = result.to_pandas()

# Compute custom measure
df['influence_score'] = (
    0.6 * df['degree'] / df['degree'].max() +
    0.4 * df['betweenness_centrality'] / df['betweenness_centrality'].max()
)
```

```
# Find top nodes by custom measure
top_influential = df.nlargest(10, 'influence_score')
print(top_influential[['node_id', 'influence_score']])
```

11.6 Performance Tips

11.6.1 Query Optimization

For large-scale dynamics simulations:

- Use fewer replicates during development, scale up for production
- Consider using numpy backend for faster computation
- Track only essential measures to reduce memory usage
- Use smaller time steps when necessary but be aware of computational cost

11.6.2 Memory Management

For very large networks or long simulations:

- Avoid tracking full trajectories (`trajectory` measure) if not needed
- Use measure-specific exports rather than full result objects
- Process results in batches for very long time series

11.7 Alternative Workflow APIs

Beyond the main DSL, py3plex provides complementary APIs for different programming styles.

11.7.1 Dplyr-Style Operations (graph_ops)

For users familiar with R's dplyr or Python's pandas, `graph_ops` provides chainable verb-based operations:

```
from py3plex.graph_ops import nodes

# Dplyr-style pipeline
df = (
    nodes(network)
    .filter(lambda n: n["degree"] > 5)
    .mutate(score=lambda n: n["degree"] * 2)
    .arrange("score", reverse=True)
    .head(10)
    .to_pandas()
)
```

Key verbs: `filter()`, `mutate()`, `arrange()`, `select()`, `group_by()`, `summarise()`

Use case: Interactive exploration and data munging with familiar pandas-like syntax.

11.7.2 Sklearn-Style Pipelines (pipeline)

For reproducible workflows, `pipeline` provides scikit-learn style composition:

```
from py3plex.pipeline import Pipeline, LoadStep, AggregateLayers
from py3plex.pipeline import LouvainCommunity, ComputeStats

# Define analysis pipeline
pipe = Pipeline([
    ("load", LoadStep(path="network.graphml")),
    ("aggregate", AggregateLayers()),
    ("community", LouvainCommunity(resolution=1.0)),
    ("stats", ComputeStats()),
])

# Run pipeline
result = pipe.run()

# Reuse pipeline on different network
result2 = pipe.set_params(load__path="network2.graphml").run()
```

Key steps: `LoadStep`, `AggregateLayers`, `LeidenMultilayer`, `FilterNodes`, `SaveNetwork`

Use case: Production workflows, parameter sweeps, and reproducible analysis scripts.

11.7.3 Config-Driven Workflows (workflows)

For declarative specification of complex analyses, `workflows` supports YAML/JSON configurations:

```
from py3plex.workflows import run_workflow

config = {
    'input': {'path': 'network.edgelist', 'type': 'edgelist'},
    'steps': [
        {'type': 'community_detection', 'algorithm': 'louvain'},
        {'type': 'centrality', 'measures': ['degree', 'betweenness']},
        {'type': 'export', 'format': 'csv', 'path': 'results.csv'}
    ]
}

results = run_workflow(config)
```

Use case: Batch processing, configuration management, and non-programmer friendly analysis.

11.8 Advanced DSL Features

This section covers advanced DSL capabilities including field expressions, semiring algebra, and compositional uncertainty quantification.

11.8.1 Field Expressions (F Builder)

The **F builder** provides a Pythonic way to construct complex boolean conditions in WHERE clauses using operator overloading:

```
from py3plex.dsl import Q, L, F

# Simple comparison
result = (
    Q.nodes()
     .from_layers(L["social"])
     .compute("degree", "clustering")
     .where(F.degree > 5)
     .execute(network)
)

# Complex boolean logic with AND/OR
result = (
    Q.nodes()
     .compute("degree", "clustering")
     .where((F.degree > 10) | ((F.layer == "social") & (F.clustering < 0.5)))
     .execute(network)
)

# Negation
result = (
    Q.nodes()
     .where(~F.is_infected)
     .execute(network)
)

# Mix with kwargs
result = (
    Q.nodes()
     .where(F.degree > 5, layer="social")
     .execute(network)
)
```

Supported operators:

- Comparison: >, >=, <, <=, ==, !=
- Boolean: & (AND), | (OR), ~ (NOT)
- Parentheses for grouping: `(F.a > 5) & (F.b < 10)`

Example: See `examples/dsl_zoo/28_field_expressions.py`

```
python examples/dsl_zoo/28_field_expressions.py
```

Advantages:

- Type-safe compared to string-based queries
- IDE autocompletion for field names
- Natural Python syntax with operators
- Combines with kwargs-based filtering

11.8.2 Semiring Algebra (S Builder)

The **S builder** provides semiring-based path queries for computing shortest paths, reachability, and closure operations:

```
from py3plex.dsl import S, L

# Shortest paths using min-plus semiring
result = (
    S.paths()
     .from_node("A")
     .semiring("min_plus")
     .from_layers(L["*"])
     .execute(network)
)

# All-pairs shortest paths via closure
result = (
    S.closure()
     .semiring("min_plus")
     .from_layers(L["social"])
     .method("auto")
     .execute(network)
)

# Reachability using boolean semiring
result = (
    S.closure()
     .semiring("boolean")
     .from_layers(L["*"])
     .execute(network)
)
```

Available semirings:

- `min_plus` — Shortest paths (tropical semiring)
- `boolean` — Reachability (OR-AND algebra)
- `max_times` — Widest paths (reliability)

Path constraints:

- `.max_hops(k)` — Limit path length to k hops
- `.crossing_layers("allowed")` — Allow cross-layer edges
- `.witness(True)` — Track witness paths

Example: See `examples/dsl_zoo/24_semiring_closure.py`

```
python examples/dsl_zoo/24_semiring_closure.py
```

Use cases:

- Multi-hop analysis in multilayer networks
- Layer-aware shortest paths
- Network accessibility and connectivity analysis

11.8.3 Compositional Uncertainty Quantification

Compositional UQ extends basic uncertainty quantification to aggregate and ranking operations, providing uncertainty estimates for complex query results:

```
from py3plex.dsl import Q, L

# Per-layer aggregation with uncertainty
result = (
    Q.nodes()
     .from_layers(L["*"])
     .compute("degree", "clustering")
     .per_layer()
     .aggregate(
         avg_degree="mean(degree)",
         max_cluster="max(clustering)",
         node_count="count()"
     )
     .uq(method="bootstrap", n_samples=20, ci=0.95, seed=42)
     .execute(network)
)

# Ranking with stability metrics
result = (
    Q.nodes()
     .compute("degree")
     .order_by("-degree")
     .limit(10)
     .uq(method="perturbation", n_samples=50, seed=42)
     .execute(network)
)
```

Key features:

- **Aggregate UQ:** Mean, median, max, min with confidence intervals
- **Ranking stability:** Top-k selection with stability scores
- **Coverage probability:** Membership probability in result sets
- **Method support:** bootstrap, perturbation, seed-based resampling

Result structure:

Aggregates return dictionary values with uncertainty:

```
# Example aggregate result
{
    'mean': 5.2,           # Point estimate
    'std': 0.3,            # Standard error
    'quantiles': {         # Confidence intervals
        0.025: 4.8,
        0.975: 5.6
    },
    'n_samples': 20
}
```

Example: See `examples/dsl_zoo/42_compositional_uq.py`

```
python examples/dsl_zoo/42_compositional_uq.py
```

Use cases:

- Robust layer comparisons with statistical confidence
- Stable hub identification in noisy networks
- Uncertainty-aware decision making
- Sensitivity analysis for network metrics

11.9 Guided Quickstart Recipes

These task-oriented recipes demonstrate best practices for common multilayer network analysis tasks. Each recipe is minimal (10 lines) and uses DSL v2 with modern practices.

11.9.1 Recipe 1: Find Hubs Across Layers

Identify nodes that are highly central across multiple network layers:

```
from py3plex.dsl import Q, L
from py3plex.core import multinet

# Load your network
net = multinet.multi_layer_network()
net.load_network("network.graphml")

# Find nodes with high degree in multiple layers
result = (
    Q.nodes()
     .from_layers(L["*"])  # All layers
     .compute("degree", "betweenness_centrality")
     .per_layer()  # Group by layer
       .top_k(10, "degree")  # Top 10 per layer
     .end_grouping()
     .coverage(mode="at_least", k=2)  # Present in 2 layers
     .order_by("-degree")
     .execute(net)
)

# Export hub nodes
df = result.to_pandas()
df.to_csv("cross_layer_hubs.csv", index=False)
print(f"Found {result.count} hubs across {len(net.get_layers())} layers")
```

Best practices:

- Use `per_layer()` for layer-aware analysis
- Apply `coverage()` to find persistent hubs
- Always `end_grouping()` before applying filters

11.9.2 Recipe 2: Compare Two Multilayer Networks

Systematically compare network structure and metrics across two networks:

```
from py3plex.dsl import Q, C

# Load two networks
net1 = multinet.multi_layer_network()
net1.load_network("network_v1.graphml")

net2 = multinet.multi_layer_network()
net2.load_network("network_v2.graphml")

# Compare using C (Compare) builder
comparison = (
    C.networks(baseline=net1, treatment=net2)
     .compare_structure()  # Node/edge counts, density
     .compare_metrics("degree", "betweenness_centrality")
     .per_layer()  # Compare layer-by-layer
     .execute()
)

# Access comparison results
df_structural = comparison.structural_diff()
df_metrics = comparison.metric_diff()

print(df_structural)
print(df_metrics)
```

Alternative: Manual comparison with provenance

```
# Query with provenance for reproducibility
result1 = (
    Q.nodes()
     .compute("degree", "betweenness_centrality")
     .provenance(mode="replayable")
     .execute(net1)
)

result2 = (
    Q.nodes()
     .compute("degree", "betweenness_centrality")
     .provenance(mode="replayable")
     .execute(net2)
)

# Compare DataFrames
df1 = result1.to_pandas()
df2 = result2.to_pandas()

# Compute differences (e.g., degree changes)
merged = df1.merge(df2, on=["id", "layer"], suffixes=("_v1", "_v2"))
merged["degree_diff"] = merged["degree_v2"] - merged["degree_v1"]
```

11.9.3 Recipe 3: Run Community Detection with Uncertainty

Find communities with uncertainty quantification to assess partition stability:

```
from py3plex.dsl import Q, UQ

# Community detection with UQ
result = (
    Q.nodes()
     .from_layers(L["social"] + L["work"])
     .community(method="leiden", gamma=1.2, omega=0.8, random_state=42)
     .uq(method="ensemble", n_samples=50, seed=42)
     .execute(net)
)

# Access consensus partition and stability
consensus_partition = result.meta["consensus_partition"]
score_ci = result.meta["score_ci"]  # Confidence interval for modularity

print(f"Consensus modularity: {score_ci['mean']:.3f} ± {score_ci['std']:.3f}")
print(f"95% CI: [{score_ci['ci95_low']:.3f}, {score_ci['ci95_high']:.3f}]")

# Get community assignments with uncertainty
df = result.to_pandas()
print(df[["id", "layer", "community_id"]].head())
```

Best practices:

- Use `random_state` for reproducibility
- UQ method `ensemble` is best for community detection
- Check `score_ci` to assess partition quality

11.9.4 Recipe 4: Export Reproducible Results

Create fully reproducible analysis results with provenance tracking:

```
from py3plex.dsl import Q, L

# Query with full provenance
result = (
    Q.nodes()
     .from_layers(L["social"])
     .compute("degree", "betweenness_centrality", "clustering")
     .uq(method="bootstrap", n_samples=100, seed=42)
     .provenance(mode="replayable", capture="snapshot")
     .execute(net)
)

# Export bundle with everything needed for replay
result.export_bundle("analysis_results.bundle.json.gz", compress=True)

# Also export data tables
df = result.to_pandas(expand_uncertainty=True)
df.to_csv("metrics_with_uncertainty.csv", index=False)
```

```
# Later: Replay the analysis
from py3plex.provenance.replay import replay_query_from_bundle
replayed_result = replay_query_from_bundle("analysis_results.bundle.json.gz")

# Results should match exactly
assert replayed_result.count == result.count
```

Best practices:

- Always set `seed` for UQ to ensure reproducibility
- Use `provenance(mode="replayable")` for full replay capability
- Export bundle immediately after execution
- Store bundle with analysis outputs for audit trail

Provenance bundle contents:

- Query AST (serialized)
- Network snapshot or fingerprint
- Parameter values
- Random seeds
- Environment info (py3plex version, Python version)
- Optionally: Full result data

11.10 Closing Note

Use this chapter as a reference layer after you can already write and interpret core queries. If a workflow here feels heavy, return to Chapters 8–9, then reintroduce one advanced feature at a time (for example: first parameterization, then UQ, then semiring paths).

11.11 Further Reading

- Introduction to the Py3plex DSL (Chapter 8)
- The Builder API and Explain Plans (Chapter 9)
- `examples/dsl_zoo/` — Focused advanced query patterns
- `examples/network_analysis/` — Advanced DSL and graph-ops workflows
- `examples/advanced/` — Dynamics and process examples
- `docfiles/sir_epidemic_simulator.rst` — SIR multiplex simulator documentation

CHAPTER

TWELVE

LIMITATIONS AND STABILITY GUARANTEES

This chapter scopes feature status to the workflows documented in this book.

12.1 DSL Feature Status

12.1.1 Stable APIs

Features with strong backward-compatibility guarantees:

- **Node queries** — `Q.nodes()` with filtering and measures
- **Edge queries** — `Q.edges()` with substantial coverage of node-query workflows, including `per_layer_pair()`, `aggregate()`, `where()`, and all export formats
- **Layer algebra** — Union, difference, intersection operations
- **Core measures** — degree, betweenness, closeness, pagerank
- **Result export** — pandas, JSON, CSV formats
- **Aggregations** — `per_layer()` / `per_layer_pair()` with `aggregate()` supporting `mean`, `sum`, `min`, `max`, `std`, `var`, `median`, `quantile`, and `count`
- **Temporal queries** — `Q.edges().at(t)`, `.during(t0, t1)`, `.before(t)`, `.after(t)`, `.window(size, step)`
- **Join operations** — `.join(right, on=..., how=...)` with documented join types
- **Graph pattern matching** — `Q.pattern()` for motif-style matching

Scope statement: For the documented workflows, these APIs are expected to remain backward compatible within 1.x releases, with deprecation notice before incompatible changes.

12.2 Interface Selection Guide (Core vs Alternatives)

Use this chapter-level guide to choose the right interface once you already understand core DSL syntax.

Table 1: Interface Decision Matrix

Interface	Best For	Use When	Avoid When
DSL v2 (Q, L, UQ)	Multilayer querying and reproducible analysis scripts	Layer algebra, grouped summaries, provenance, UQ	You only need simple single-layer NetworkX operations
graph_ops	Data-frame style exploration	You want quick filter/mutate/export cycles	You need multilayer-specific query semantics
Pipeline API	Repeatable multi-step workflows	You need fit/transform-style orchestration	You are doing one-off exploratory queries
CLI	Scripted batch tasks and quick checks	You need shell-based automation	You need complex custom branching logic

12.2.1 Experimental Features

Functional but may change in future versions:

- **Nested subqueries** — Complex query composition via `join()` and `QueryResult` chaining; the exact semantics may be refined

Note: Experimental features are marked in documentation. Use with awareness that API may evolve.

12.2.2 Planned Features

Roadmap items (briefly mentioned, not detailed):

- **Streaming export** — Incremental result materialisation for very large networks
- **Custom semiring registration** — User-defined algebraic structures at runtime

12.3 Current Limitations

12.3.1 Query Capabilities

What the DSL **can** do:

- Node and edge filtering by degree, layer, and computed measures
- Compute centrality measures
- Layer algebra operations
- Export results in multiple formats
- Aggregate results per layer or per layer-pair (`mean`, `sum`, `min`, `max`, `std`, `var`, `median`, `quantile`, `count`)
- Query temporal networks with time-window and snapshot filters
- Match graph motifs and patterns via `Q.pattern()`
- Join query results across layers with relational join semantics

What the DSL **cannot** (yet) do:

- Nested subqueries — queries whose `where()` clause references the result of another live query (not yet supported as a first-class construct)

12.3.2 Performance Boundaries

- **Small networks** (<10k nodes) — No issues
- **Medium networks** (10k-100k nodes) — Most queries fast
- **Large networks** (>100k nodes) — Some queries may be slow; use filtering early

12.4 Scale and Performance

12.4.1 Query Complexity

DSL query operations have different time complexities:

Table 2: Query Operation Complexity

Operation	Time Complexity	Notes
`Q.nodes()` (no filter)	O(n)	Linear scan of nodes
`.where(degree__gt=k)`	O(n + m)	Degree computation
`.compute("degree")`	O(n + m)	Linear in edges
`.compute("betweenness")`	O(nm)	Expensive for large networks
`.compute("pagerank")`	O(k·m)	k iterations, m edges
`.order_by()`	O(n log n)	Sorting results
`.limit(k)`	O(k)	Constant after ordering
`.to_pandas()`	O(n)	Result materialization

Query optimization tips:

- Apply `where()` filters early to reduce working set size
- Use `limit()` when you only need top-k results
- Avoid betweenness centrality on networks > 10K nodes (use sampling or degree-based measures)
- Cache results with `.to_pandas()` if reusing the same query output

12.4.2 Memory Requirements

Memory usage guidelines:

- **Small networks** (<10K nodes): Negligible overhead, all queries run in-memory
- **Medium networks** (10K-100K nodes):
  - Node queries: ~1-10 MB per query result
  - Betweenness centrality: $O(n^2)$ memory, problematic > 20K nodes
  - PageRank: O(n) memory, scales well
- **Large networks** (>100K nodes):
  - Use streaming or chunked queries where possible
  - Disable `autocompute` if metrics are pre-computed: `Q.nodes().compute(..., autocompute=False)`
  - Export results incrementally with `.to_json(stream=True)` if available
  - Consider using external graph databases (Neo4j) for very large networks

Best practices for large networks:

1. Filter aggressively with `where()` before computing expensive measures
2. Use degree-based centrality instead of betweenness when possible
3. Sample nodes if exact answers aren't required
4. Leverage layer algebra to query subsets: `Q.nodes().from_layers(L["layer1"])`

12.4.3 When Not to Use the DSL

Use direct NetworkX/NumPy for:

- Single-layer graphs (DSL overhead not needed)
- Custom algorithms requiring fine-grained control
- Performance-critical inner loops

12.5 API Versioning Policy

12.5.1 Semantic Versioning

py3plex follows semantic versioning (MAJOR.MINOR.PATCH):

- **MAJOR** (1.x → 2.x) — Breaking changes possible
- **MINOR** (1.1 → 1.2) — New features; existing documented APIs are intended to remain compatible
- **PATCH** (1.1.1 → 1.1.2) — Bug fixes only

12.5.2 Deprecation Policy

Deprecated features are:

1. Marked in documentation and code (warnings)
2. Maintained for at least one minor version
3. Removed in next major version

12.5.3 Migration Guides

When breaking changes occur in major versions, py3plex provides:

- **Migration guide documentation** in the CHANGELOG
- **Deprecation warnings** for at least one minor version before removal
- **Code examples** showing old vs. new API usage
- **Automated migration scripts** where feasible (e.g., for simple renames)

Migration guides are published in the `docs/migration/` directory and linked from the main documentation.

12.6 Summary

Intended stable workflows in this book:

- Node and edge queries with filtering and broad feature coverage
- Core centrality measures
- Layer algebra

- Aggregations (mean, sum, min, max, std, var, median, quantile, count)
- Temporal queries (snapshots, windows, before/after)
- Graph pattern matching via `Q.pattern()`
- Join operations (inner, left, right, outer, semi, anti)
- Result export

Use with awareness:

- Experimental features (nested subqueries) may change
- Large network performance depends on query structure

The DSL prioritizes transparent status reporting. Use stable workflows with version pinning, and treat experimental features as opt-in.

See also

- **Full DSL reference:** `docfiles/user_guide/dsl.rst`
- **Algorithm roadmap:** `docfiles/algorithm_roadmap.rst`
- **Performance benchmarks:** `benchmarks/` directory

CHAPTER

THIRTEEN

CASE STUDY 1 — SOCIAL MULTIPLEX NETWORK

Case Study Template

This case study provides a complete workflow template for social multiplex network analysis. The structure and code patterns can be adapted to your own datasets. Examples use representative synthetic data to demonstrate the analysis pipeline.

DSL in Case Studies

This case study demonstrates DSL throughout the analysis workflow:

```
from py3plex.dsl import Q, L

# 1. Quick exploratory analysis
for platform in ["facebook", "twitter", "linkedin"]:
    stats = (
        Q.nodes()
         .from_layers(L[platform])
         .compute("degree", "clustering")
         .execute(network)
    )
    df = stats.to_pandas()
    print(f"{platform}: avg degree = {df['degree'].mean():.2f}")

# 2. Find cross-platform influencers
influencers = (
    Q.nodes()
     .from_layers(L["facebook"] + L["twitter"] + L["linkedin"])
     .where(degree__gt=50)
     .compute("betweenness_centrality")
     .order_by("-betweenness_centrality")
     .limit(20)
     .execute(network)
)
influencers.to_pandas().to_csv("influencers.csv", index=False)

# 3. Platform-specific analysis
twitter_only = (
    Q.nodes()
     .from_layers(L["twitter"] - L["facebook"] - L["linkedin"])
```

```
    .compute("degree")
    .execute(network)
)
```

DSL enables rapid iteration in exploratory research!

13.1 Domain Context

Social media users often maintain multiple online identities across platforms (Facebook, Twitter, LinkedIn, Instagram). This creates a **social multiplex network** where:

- **Nodes** represent users
- **Layers** represent platforms
- **Intra-layer edges** are friendships/follows within a platform
- **Inter-layer edges** link the same user across platforms

Research questions:

1. How do user roles vary across platforms?
2. Are influential users consistent across layers, or platform-specific?
3. Do communities align across platforms or fragment differently?
4. What cross-layer patterns emerge (e.g., Twitter influencers who are Facebook novices)?

13.1.1 Dataset Structure

Example dataset: Multi-platform social network

- **Nodes:** ~5,000 users (some present on multiple platforms)
- **Layers:** 3 platforms (Facebook, Twitter, LinkedIn)
- **Edges:** ~20,000 connections
- **Attributes:** User demographics (age, location), timestamps

Data format (edgelist):

```
# Format: source, source_layer, target, target_layer, weight
user_123, facebook, user_456, facebook, 1
user_123, twitter, user_789, twitter, 1
user_123, facebook, user_123, twitter, 1  # Inter-layer link
```

13.2 Loading the Data

13.2.1 Create and Load Network

```
from py3plex.core import multinet

# Create multilayer network
network = multinet.multi_layer_network(directed=False)

# Load from edgelist
```

```
network.load_network('social_multiplex.edgelist', input_type='edgelist')

# Verify structure
print(f"Nodes: {network.number_of_nodes()}")
print(f"Edges: {len(network.get_edges())}")
print(f"Layers: {network.get_layers()}")

# Basic statistics
network.basic_stats()
```

Expected output:

```
Nodes: 5234
Edges: 18976
Layers: ['facebook', 'twitter', 'linkedin']
```

13.3 Full Analysis Pipeline

13.3.1 Step 1: Exploratory Analysis

Start with layer-level statistics:

```
from py3plex.dsl import Q, L
import pandas as pd

# Collect statistics for each layer
layer_summary = []

for layer in ["facebook", "twitter", "linkedin"]:
    result = (
        Q.nodes()
         .from_layers(L[layer])
         .compute("degree", "clustering")
         .execute(network)
    )
    df = result.to_pandas()

    layer_summary.append({
        'layer': layer,
        'nodes': result.count,
        'avg_degree': df['degree'].mean(),
        'max_degree': df['degree'].max(),
        'avg_clustering': df['clustering'].mean()
    })

summary_df = pd.DataFrame(layer_summary)
print(summary_df)
```

Interpretation: Compare layer densities and clustering to understand platform differences.

13.3.2 Step 2: Community Detection

Find communities using multilayer Louvain:

```
from py3plex.algorithms.community_detection import multilayer_louvain

# Detect communities
communities = multilayer_louvain.best_partition(network.core_network)

# Add communities as node attributes
for node, comm_id in communities.items():
    network.core_network.nodes[node]['community'] = comm_id

# Count communities
n_communities = len(set(communities.values()))
print(f"Found {n_communities} communities")

# Largest communities
from collections import Counter
community_sizes = Counter(communities.values())
print("Top 5 communities by size:")
for comm_id, size in community_sizes.most_common(5):
    print(f"  Community {comm_id}: {size} nodes")
```

13.3.3 Step 3: Centrality Analysis

Identify influential users using multilayer PageRank:

```
from py3plex.algorithms.centrality_toolkit import multilayer_pagerank

# Compute multilayer PageRank
pagerank_scores = multilayer_pagerank(network.core_network)

# Use DSL to find top influencers
result = (
    Q.nodes()
     .compute("degree", "betweenness_centrality")
     .execute(network)
)

df = result.to_pandas()
df['pagerank'] = df['node_id'].map(pagerank_scores)

# Top influencers
top_influencers = df.nlargest(20, 'pagerank')
print(top_influencers[['node_id', 'degree', 'betweenness_centrality', 'pagerank']])
```

13.3.4 Step 4: Cross-Layer Patterns

Analyze how user importance varies across platforms:

```
# For each user, compute degree in each layer
user_profiles = {}
```

```
for layer in ["facebook", "twitter", "linkedin"]:
    result = (
        Q.nodes()
         .from_layers(L[layer])
         .compute("degree")
         .execute(network)
    )
    df = result.to_pandas()

    for _, row in df.iterrows():
        user_id = row['node_id'][0]  # Extract base user ID
        if user_id not in user_profiles:
            user_profiles[user_id] = {}
        user_profiles[user_id][f"{layer}_degree"] = row['degree']

# Convert to DataFrame
profile_df = pd.DataFrame.from_dict(user_profiles, orient='index').fillna(0)

# Find cross-layer imbalances
profile_df['max_degree'] = profile_df.max(axis=1)
profile_df['min_degree'] = profile_df.min(axis=1)
profile_df['imbalance'] = profile_df['max_degree'] / (profile_df['min_degree'] + 1)

# Users with high imbalance (platform specialists)
specialists = profile_df.nlargest(10, 'imbalance')
print("Platform specialists (high cross-layer imbalance):")
print(specialists)
```

13.3.5 Step 5: Visualization

Create analysis visuals and export draft figures:

```
from py3plex.visualization.multilayer import draw_multilayer_default
import matplotlib.pyplot as plt

# Visualize network with communities colored
draw_multilayer_default(
    network.get_layers(),
    node_size=8,
    labels=True,
    background_shape="circle",
    scale_by_size=True,  # Scale nodes by degree
    display=True
)

plt.title("Social Multiplex Network - Community Structure")
plt.savefig("social_multiplex_communities.png", dpi=300, bbox_inches='tight')
```

13.4 Key Findings (Template)

13.4.1 Community Structure

Observation: Communities tend to form around shared topics or interests. Nodes with high inter-community connectivity (high betweenness) act as bridges between different social groups. The community structure often correlates with geographic or demographic factors in real social networks.

- **Cross-platform communities:** X% of users in same community across all layers
- **Platform-specific communities:** Y% of communities confined to single layer
- **Community alignment:** Normalized Mutual Information = Z

Interpretation: Cross-platform community alignment suggests that social structures persist across different communication channels, likely reflecting offline social relationships or shared interests. Platform-specific communities indicate specialized use cases (e.g., professional networking on LinkedIn vs. casual social interaction on Facebook).

13.4.2 Cross-Layer Roles

Observation: User roles vary significantly across platforms. A highly central user on one platform may be peripheral on another, reflecting different usage patterns, social contexts, or specialization. Cross-platform influencers (high centrality on multiple layers) are relatively rare but strategically important for information diffusion.

- **Consistent influencers:** N users with high centrality on all platforms
- **Platform specialists:** M users highly central on one platform only
- **Role switching:** Average centrality correlation across layers =

Interpretation: [How do user strategies differ by platform?]

13.5 Pitfalls Encountered

13.5.1 Data Cleaning Issues

Challenge: User identity resolution across platforms

- Usernames differ across platforms
- Manual matching required for subset of users
- Missing links lead to underestimated cross-layer effects

Solution: Use email-based or profile-based matching when available

13.5.2 Performance Considerations

Challenge: Betweenness centrality computation slow for large networks

- $O(n^3)$ complexity becomes prohibitive for >10,000 nodes
- Consider sampling or approximation algorithms

Solution: Use degree or PageRank for large-scale analysis; reserve betweenness for focused subgraphs

13.6 Reproducibility

13.6.1 Code Repository

This analysis workflow is based on the examples in the repository. To reproduce similar analyses:

```
# Clone repository
git clone https://github.com/SkBlaz/py3plex.git
cd py3plex

# Install dependencies
pip install -e .

# Run relevant examples
uv run examples/network_analysis/example_community_detection.py
uv run examples/network_analysis/example_dsl_builder_api.py
uv run examples/advanced/example_multiplex_dynamics.py
```

13.6.2 Data Availability

Synthetic data for this case study is available in: `examples/datasets/synthetic_social_multiplex.edgelist`

For real datasets, consider:

- **Twitter + Facebook:** Similar methodology can be applied to real datasets (subject to API access and data use agreements)
- **Academic multiplex:** DBLP + ArXiv coauthorship networks
- **Contact datasets:** See `multilayer_datasets/` directory

13.7 Summary

This case study demonstrated:

1. **Data loading** from edgelist format
2. **Exploratory analysis** using DSL for layer statistics
3. **Community detection** with multilayer Louvain
4. **Centrality analysis** with multilayer PageRank
5. **Cross-layer pattern detection** (role switching, specialists)
6. **Visualization** for publication

Key workflow:

1. Load → 2. Explore → 3. Detect communities → 4. Compute centrality → 5. Analyze patterns → 6. Visualize

Adapt this workflow to your own social multiplex datasets by:

- Adjusting layer names
- Modifying centrality thresholds
- Adding domain-specific measures
- Customizing visualizations

CHAPTER

FOURTEEN

CASE STUDY 2 — BIOLOGICAL MULTILAYER NETWORK

Case Study Template

This case study demonstrates a complete workflow for biological multilayer network analysis. The methodology and code patterns are applicable to real datasets. Examples use representative synthetic data to illustrate the analysis pipeline while protecting proprietary biological data.

14.1 Domain Context

Biological systems exhibit multilayer structure through different interaction types:

- **Protein-protein interactions** — Physical binding
- **Regulatory relationships** — Gene expression control
- **Metabolic pathways** — Enzyme-substrate relationships
- **Co-expression networks** — Correlated expression across conditions

A **biological multiplex network** represents these different interaction mechanisms as separate layers while capturing the same entities (genes/proteins) across layers.

Research questions:

1. How do protein functions differ across interaction types?
2. Which genes are central in regulation but peripheral in metabolism?
3. Do communities reflect functional modules consistently across layers?
4. How do dynamics (e.g., epidemic-like spreading) differ by interaction type?

14.1.1 Dataset Structure (Template)

Example dataset: Multi-omic interaction network

- **Nodes:** ~2,000 genes/proteins
- **Layers:** Physical interaction, regulatory, metabolic
- **Edges:** ~10,000 interactions across layers
- **Attributes:** GO terms, tissue specificity, expression levels
- **Focus:** Dynamics simulation and layer-specific analysis

Data format:

```
# Format: gene_A, interaction_type, gene_B, interaction_type, confidence
BRCA1, protein_interaction, TP53, protein_interaction, 0.95
BRCA1, regulation, ATM, regulation, 0.87
TP53, metabolic, MDM2, metabolic, 0.92
```

14.2 Loading and Preprocessing

14.2.1 Create and Load Network

```
from py3plex.core import multinet

# Create biological multilayer network
network = multinet.multi_layer_network(directed=True)  # Regulation is directed

# Load from biological database format
network.load_network('biological_multiplex.edgelist', input_type='edgelist')

# Add gene annotations (GO terms, etc.)
# annotations = load_annotations('gene_annotations.tsv')
# for node in network.get_nodes():
#     network.core_network.nodes[node]['GO_terms'] = annotations.get(node[0], [])

print(f"Loaded {network.number_of_nodes()} genes")
print(f"Layers: {network.get_layers()}")
```

14.3 Analysis Pipeline Sketch

14.3.1 Step 1: Network Topology

Analyze structural properties of each interaction layer:

```
from py3plex.dsl import Q, L

# Compare topology across interaction types
for layer in ["protein_interaction", "regulation", "metabolic"]:
    result = (
        Q.nodes()
         .from_layers(L[layer])
         .compute("degree", "clustering")
         .execute(network)
    )
    df = result.to_pandas()
    print(f"{layer}: avg degree = {df['degree'].mean():.2f}, "
          f"avg clustering = {df['clustering'].mean():.3f}")
```

Illustrative pattern expectations (template guidance):

- Protein interaction: High clustering (modules)
- Regulation: Lower clustering (hierarchical)
- Metabolic: Medium clustering (pathways)

14.3.2 Step 2: Dynamics Simulation

Model information spreading or cascade dynamics using SIR model:

```
from py3plex.dynamics import SIRDynamics

# Simulate disease/perturbation spreading
sir = SIRDynamics(
    network,
    beta=0.3,      # Transmission rate
    gamma=0.1,     # Recovery rate
    initial_infected=0.05  # Start with 5% infected
)
sir.set_seed(42)

results = sir.run(steps=100)

# Analyze outbreak size
final_recovered = results.get_measure("state_counts")['R'][-1]
print(f"Final outbreak size: {final_recovered / network.number_of_nodes():.1%}")
```

Research question: How does perturbation spread differently through physical vs. regulatory interactions?

14.3.3 Step 3: Layer-Specific Analysis

Identify genes that are central in one layer but not others:

```
# Find regulatory hubs
regulatory_hubs = (
    Q.nodes()
     .from_layers(L["regulation"])
     .where(degree__gt=20)
     .compute("betweenness_centrality")
     .execute(network)
)

# Find protein interaction hubs
ppi_hubs = (
    Q.nodes()
     .from_layers(L["protein_interaction"])
     .where(degree__gt=20)
     .compute("betweenness_centrality")
     .execute(network)
)

# Compare overlap
reg_hub_ids = set(regulatory_hubs.node_ids)
ppi_hub_ids = set(ppi_hubs.node_ids)

overlap = reg_hub_ids & ppi_hub_ids
reg_only = reg_hub_ids - ppi_hub_ids
ppi_only = ppi_hub_ids - reg_hub_ids

print(f"Overlap: {len(overlap)} genes")
print(f"Regulatory only: {len(reg_only)} genes")
```

```
print(f"PPI only: {len(ppi_only)} genes")
```

14.3.4 Step 4: Intervention Analysis

Simulate what-if scenarios: node removal, layer removal:

```
import copy

# Baseline dynamics
baseline_sir = SIRDynamics(network, beta=0.3, gamma=0.1)
baseline_sir.set_seed(42)
baseline_results = baseline_sir.run(steps=100)
baseline_outbreak = baseline_results.get_measure("state_counts")['R'][-1]

# Intervention: Remove top hub
network_intervened = copy.deepcopy(network)
top_hub = regulatory_hubs.node_ids[0]
network_intervened.core_network.remove_node(top_hub)

# Dynamics after intervention
intervened_sir = SIRDynamics(network_intervened, beta=0.3, gamma=0.1)
intervened_sir.set_seed(42)
intervened_results = intervened_sir.run(steps=100)
intervened_outbreak = intervened_results.get_measure("state_counts")['R'][-1]

reduction = (baseline_outbreak - intervened_outbreak) / baseline_outbreak
print(f"Outbreak reduced by {reduction:.1%} after removing hub")
```

14.4 Key Findings (Worked Template)

14.4.1 Spreading Patterns

Illustrative interpretation: Cascade dynamics often depend on layer structure

- **Protein interaction layer:** Slow, localized spreading (high clustering)
- **Regulatory layer:** Fast, global spreading (low clustering, directed)
- **Metabolic layer:** Intermediate spreading

Interpretation: Regulatory interactions enable rapid perturbation propagation

14.4.2 Layer Interactions

Illustrative interpretation: Cross-layer hub genes may emerge

- Genes highly central in both protein interaction and regulation
- These genes are critical: removing them has amplified impact
- Potential drug targets or disease genes

14.5 Summary

This chapter is a **worked template** rather than a completed empirical study. It demonstrates:

1. **Biological network loading** with proper directionality
2. **Layer-specific topology** analysis
3. **Dynamics simulation** (SIR model for perturbation spreading)
4. **Intervention analysis** (what-if scenarios)
5. **Cross-layer hub identification**

For a completed empirical study, replace the synthetic placeholders with a curated dataset, explicit validation criteria, and externally checkable outputs.

Relevant examples:

- `examples/advanced/sis_dynamics.py` — SIS dynamics
- `examples/advanced/example_multiplex_dynamics.py` — Multilayer epidemic-style simulation
- `examples/network_analysis/example_dsl_builder_api.py` — Centrality analysis
- `docfiles/sir_epidemic_simulator.rst` — SIR documentation

CHAPTER

FIFTEEN

CASE STUDY 3 — TRANSPORTATION NETWORK

Case Study Template

This case study outlines a complete workflow for transportation multilayer network analysis. The approach demonstrates advanced techniques including temporal analysis and large-scale DSL queries. Examples use representative synthetic data to illustrate methodology applicable to real transportation datasets.

15.1 Domain Context

Urban transportation systems are inherently multilayer:

- **Nodes:** Locations (stations, stops, intersections)
- **Layers:** Transportation modes (subway, bus, bike-share, walking)
- **Edges:** Direct connections between locations within each mode
- **Inter-layer edges:** Transfer points between modes
- **Temporal dimension:** Schedules, rush hours, seasonal patterns

Research questions:

1. How do disruptions in one mode affect overall connectivity?
2. Which stations are critical hubs across multiple modes?
3. How do communities of well-connected areas differ by mode?
4. What is the optimal multi-modal route between locations?

15.2 Potential Analysis Directions

15.2.1 Resilience Analysis

Simulate mode disruptions and measure impact:

```
from py3plex.dsl import Q, L

# Baseline connectivity
baseline = (
    Q.nodes()
     .compute("betweenness_centrality")
     .execute(network)
)
```

```
# Remove subway layer
reduced_network = network.remove_layer("subway")

# Recompute connectivity
disrupted = (
    Q.nodes()
     .compute("betweenness_centrality")
     .execute(reduced_network)
)

# Identify most affected locations
baseline_df = baseline.to_pandas()
disrupted_df = disrupted.to_pandas()

# Compare betweenness changes
for idx in baseline_df.index:
    node = baseline_df.loc[idx, 'node']
    layer = baseline_df.loc[idx, 'layer']
    base_bc = baseline_df.loc[idx, 'betweenness_centrality']

    # Find matching node in disrupted network
    disrupted_row = disrupted_df[
        (disrupted_df['node'] == node) & (disrupted_df['layer'] == layer)
    ]
    if not disrupted_row.empty:
        disrupt_bc = disrupted_row.iloc[0]['betweenness_centrality']
        impact = (base_bc - disrupt_bc) / base_bc if base_bc > 0 else 0
        if impact > 0.2:  # More than 20% reduction
            print(f"High impact: {node} in {layer}, {impact:.1%} reduction")
```

15.2.2 Multi-Modal Path Analysis

Find optimal paths using multiple transportation modes:

```
# Shortest path across modes
import networkx as nx

path = nx.shortest_path(
    network.core_network,
    source=('location_A', 'bus'),
    target=('location_B', 'subway')
)

# Identify mode switches in path
mode_switches = 0
for i in range(len(path) - 1):
    current_mode = path[i][1]  # Layer is second element of tuple
    next_mode = path[i+1][1]
    if current_mode != next_mode:
        mode_switches += 1
        print(f"Switch from {current_mode} to {next_mode} at {path[i][0]}")
```

```
print(f"Total mode switches: {mode_switches}")
print(f"Path length: {len(path)} stops")
```

15.2.3 Temporal Dynamics

Analyze how network properties change throughout the day:

```
from py3plex.dsl import Q
import pandas as pd

# Example: Analyze network at different time windows
time_windows = [(0, 6), (6, 12), (12, 18), (18, 24)]  # Hours of day
temporal_stats = []

for start_hour, end_hour in time_windows:
    # Filter edges by time window (assuming temporal metadata)
    result = (
        Q.nodes()
         .compute("degree", "betweenness_centrality")
         .execute(network)
    )

    stats = {
        'window': f"{start_hour:02d}:00-{end_hour:02d}:00",
        'avg_degree': result.to_pandas()['degree'].mean(),
        'avg_betweenness': result.to_pandas()['betweenness_centrality'].mean()
    }
    temporal_stats.append(stats)

# Display temporal patterns
temporal_df = pd.DataFrame(temporal_stats)
print(temporal_df)
```

15.2.4 DSL-Heavy Analysis

Demonstrate advanced DSL patterns:

```
from py3plex.dsl import Q, L, Param

# Parameterized cross-modal analysis
hub_query = (
    Q.nodes()
     .from_layers(
         L[Param.str("mode1")] + L[Param.str("mode2")]
     )
     .where(degree__gt=Param.int("threshold"))
     .compute("betweenness_centrality", "closeness_centrality")
     .order_by("-betweenness_centrality")
     .limit(20)
)

# Execute for different mode combinations
for mode1, mode2 in [("subway", "bus"), ("bus", "bike"), ("subway", "bike")]:
```

```
    result = hub_query.execute(
        network,
        mode1=mode1,
        mode2=mode2,
        threshold=10
    )
    print(f"{mode1} + {mode2}: {result.count} hubs")
```

15.3 Summary

This chapter is presented as a **workflow template** rather than a completed empirical case study. It demonstrates:

1. Temporal multilayer network framing
2. Resilience analysis through layer-removal scenarios
3. Multi-modal routing analysis
4. Parameterized DSL query patterns for comparative runs
5. A roadmap for geospatial extension once coordinate data is available

Readers adapting this template to their own dataset should add a documented data source, fixed preprocessing scripts, pinned environment metadata, and reproducible result artifacts before treating conclusions as empirical findings.

Relevant resources:

- GTFS (General Transit Feed Specification) for public transit data
- OpenStreetMap for walking/cycling networks
- `examples/dsl_zoo/` for DSL patterns
- `examples/network_analysis/` for network building and data loading patterns

CHAPTER

SIXTEEN

TESTING AND VALIDATION

py3plex uses a broad testing strategy to check correctness across algorithms, data structures, and the DSL. This chapter provides a high-level overview of what is validated and what still requires reader-side verification. **For detailed validation scripts and test code, see Appendix C.**

16.1 Testing Philosophy

py3plex testing follows four principles:

1. **Correctness first:** Algorithms must produce mathematically correct results
2. **Conservation laws:** Physical constraints (e.g., probability conservation) must hold
3. **Regression prevention:** Known-good outputs are compared against current runs
4. **Property testing:** Invariants should hold for random inputs

16.1.1 Test Categories

- **Unit tests** — Individual functions and methods in isolation
- **Integration tests** — Module interactions and end-to-end workflows
- **Property-based tests** — Hypothesis-driven testing with random inputs
- **Regression tests** — Compare against reference runs for dynamics models

16.2 Test Organization

The test suite is organized by module and functionality:

```
tests/
├── test_core.py                  # Core data structures
├── test_dsl*.py                  # DSL functionality (10+ files)
├── test_dynamics*.py             # Dynamics models
├── test_algorithms*.py           # Algorithm correctness
├── test_centrality*.py           # Centrality measures
├── test_community*.py            # Community detection
├── test_io*.py                   # I/O and serialization
├── test_uncertainty*.py          # Uncertainty quantification
└── property/                     # Property-based tests
```

Coverage is uneven across modules. Confirm current coverage in your own checkout before citing any specific percentages.

16.3 Key Validation Strategies

16.3.1 Random Walk Conservation

Random walks must conserve probability—transition probabilities from any state must sum to 1:

```
def test_random_walk_conservation():
    """Verify random walk conserves probability."""
    # Implementation in tests/test_random_walk_conservation.py
    # Computes transition matrix and checks row sums  1.0
```

Tests: `tests/test_paths.py` includes walk conservation checks.

16.3.2 Node2Vec Validation

Validate that Node2Vec biases (parameters p and q) have the intended effects:

- Higher p → reduced probability of returning to previous node
- Higher q → preference for local exploration vs. distant jumps

Tests: `tests/test_node2vec.py` validates bias behavior.

16.3.3 Community Detection Validation

Verify that community detection algorithms satisfy basic properties:

- **Modularity bounds:** Q [-0.5, 1.0]
- **Partition completeness:** Every node assigned to exactly one community
- **Singleton handling:** Isolated nodes form their own communities

Tests: `tests/test_community*.py` files validate Louvain, Infomap, and other algorithms.

16.3.4 Dynamics Validation

Epidemic models must satisfy conservation laws:

- **SIS:** S(t) + I(t) = N for all t
- **SIR:** S(t) + I(t) + R(t) = N for all t
- **Steady state:** SIS reaches equilibrium for subcritical parameters

Tests: `tests/test_dynamics.py` validates SIR, SIS, and other models with reference runs.

16.4 Running Tests

16.4.1 Basic Test Execution

```
# Run all tests
pytest tests/

# Run specific test file
pytest tests/test_dsl.py

# Run tests matching a pattern
pytest tests/ -k "test_community"
```

```
# Run with coverage
pytest tests/ --cov=py3plex --cov-report=html

# Verbose output
pytest tests/ -v
```

16.4.2 Using Makefile

```
make test           # Run all tests
make test-coverage  # Generate HTML coverage report
make test-fast      # Run only fast tests (skip slow integration tests)
```

Test markers:

- `@pytest.mark.slow` — Tests that take >5 seconds
- `@pytest.mark.integration` — End-to-end integration tests
- `@pytest.mark.hypothesis` — Property-based tests

16.5 Continuous Integration

py3plex uses GitHub Actions for automated testing on pull requests and branch updates.

16.5.1 GitHub Actions

The CI pipeline tests across:

- **Python versions:** 3.8, 3.9, 3.10, 3.11, 3.12
- **Operating systems:** Ubuntu Linux, macOS, Windows
- **Test suites:** Core, DSL, algorithms, dynamics, I/O

Build status: Tests must pass on all platforms before merging.

Coverage expectations: New code should maintain or improve practical confidence in touched areas.

CI Configuration

The GitHub Actions workflow files are located in `.github/workflows/` in the repository. Key workflows include `test.yml` (main test suite), `lint.yml` (code quality checks), and `docs.yml` (documentation builds).

16.6 Closing Note

Treat testing claims in this book as workflow guidance, not blanket guarantees. For any critical study, record the exact test commands, environment details, and commit hash used for your own run.

For detailed test scripts and validation examples, see Appendix C.

Next chapter: Reproducible environments and deployment practices

CHAPTER

SEVENTEEN

REPRODUCIBLE ENVIRONMENTS

Reproducibility is fundamental to scientific computing. This chapter covers practices for creating consistent, reproducible analysis environments using py3plex. **For detailed Docker configurations and deployment recipes, see Appendix B.**

17.1 Scope and Verification Boundaries

This chapter describes reproducibility practices, not a guarantee that all historical scripts in the repository will run unchanged forever. Reproducibility depends on:

- pinned dependencies,
- archived input data,
- explicit random seeds, and
- preserved execution context (platform, Python version, py3plex version).

17.2 Reproducibility Principles

Reproducible research requires:

1. **Isolated environments** — Avoid conflicts between projects and system packages
2. **Dependency pinning** — Lock package versions to prevent drift
3. **Seed management** — Control randomness in stochastic processes
4. **Environment documentation** — Record exact configurations used

Without these practices, analyses may produce different results on different machines or at different times, undermining scientific validity.

17.3 Environment Management

17.3.1 Python Virtual Environments

Use `venv` or `conda` to isolate py3plex installations:

Using venv (built-in):

```
# Create virtual environment
python3 -m venv py3plex-env

# Activate (Linux/macOS)
```

```
source py3plex-env/bin/activate

# Activate (Windows)
py3plex-env\Scripts\activate

# Install py3plex
pip install py3plex

# Deactivate when done
deactivate
```

Using conda:

```
# Create conda environment
conda create -n py3plex python=3.10

# Activate
conda activate py3plex

# Install py3plex
pip install py3plex

# Deactivate
conda deactivate
```

Recommendation: Use `venv` for simple projects, `conda` when you need non-Python dependencies (e.g., graph-tool, igraph bindings).

17.3.2 Dependency Pinning

Lock exact package versions to ensure reproducibility:

Two-file approach (recommended):

1. `requirements.in` — High-level dependencies with version ranges

```
# requirements.in
py3plex>=1.0.0
numpy>=1.24.0
networkx>=3.0
```

2. `requirements.txt` — Exact pinned versions (generated)

```
# Generate exact pins from requirements.in
pip-compile requirements.in --output-file requirements.txt

# This creates entries like:
# py3plex==1.1.4
# numpy==1.24.3
# networkx==3.1
# matplotlib==3.7.1
# scipy==1.10.1
# pandas==2.0.2
# (includes all transitive dependencies)
```

Install from pinned file:

```
pip install -r requirements.txt
```

Single-file approach (simpler but less flexible):

```
# Generate requirements file with exact versions
pip freeze > requirements.txt

# Reproduce environment on another machine
pip install -r requirements.txt
```

Trade-offs:

- **Exact pins** (==) ensure reproducibility but may miss security patches
- **Version ranges** (>=) allow updates but may introduce drift
- **Recommended:** Use exact pins in `requirements.txt` for reproducible research, maintain `requirements.in` with ranges for development

Best practices:

- Commit `requirements.txt` to version control
- Regenerate when dependencies change
- Use separate files for development (`requirements-dev.txt`) and production

Example requirements.txt structure (with exact pins):

```
# Core dependencies (exact versions)
py3plex==1.1.4
numpy==1.24.3
scipy==1.10.1
networkx==3.1

# Optional features
matplotlib==3.7.1
pandas==2.0.2
```

17.4 Docker Containers (Overview)

Docker provides the strongest reproducibility guarantees by packaging the entire environment—OS, Python, libraries, and code—into a container.

17.4.1 Why Docker?

- **Complete isolation** — No system Python conflicts, no OS differences
- **Perfect reproducibility** — Identical environment across machines and time
- **Portability** — Share complete analysis environment with collaborators
- **Deployment** — Easy transition from development to production

17.4.2 Basic Docker Usage

```
# Build py3plex image
docker build -t py3plex:latest .

# Run analysis script
docker run --rm -v $(pwd):/workspace py3plex:latest \
    python /workspace/analysis.py

# Interactive shell
docker run --rm -it py3plex:latest bash
```

When to use Docker:

- **Multi-machine reproducibility** — Running on clusters, cloud, or different dev machines
- **Long-term archival** — Preserve exact environment for future replication
- **Production deployment** — Consistent behavior from dev to production

For detailed Dockerfile examples, docker-compose configurations, and deployment recipes, see Appendix B.

17.5 Seed Management

17.5.1 Controlling Randomness

Set seeds for all stochastic processes to ensure deterministic results:

```
import random
import numpy as np

# Set global seeds
random.seed(42)
np.random.seed(42)

# For dynamics models
from py3plex.dynamics import SIRDynamics

sir = SIRDynamics(network, beta=0.3, gamma=0.1)
sir.set_seed(42)  # Reproducible epidemic simulation
results = sir.run(steps=100)

# For random graph generation
import networkx as nx
G = nx.erdos_renyi_graph(100, 0.1, seed=42)
```

Seed management checklist:

1. Set seeds at the **start** of scripts before any randomness
2. Document seed values in code comments and papers
3. Use **different seeds** for different experiments (42, 43, 44, ...)
4. Re-run with multiple seeds to assess variability

17.6 Recording Environments

17.6.1 Document Analysis Environments

Record environment details for publication and replication:

```
import platform
import sys
import py3plex
import numpy as np
import networkx as nx

# Print environment summary
print("Environment Information:")
print(f"  Python: {sys.version}")
print(f"  Platform: {platform.platform()}")
print(f"  py3plex: {py3plex.__version__}")
print(f"  NumPy: {np.__version__}")
print(f"  NetworkX: {nx.__version__}")
```

Include in papers/reports:

- Python version (e.g., Python 3.10.8)
- py3plex version (e.g., 1.1.4)
- Key dependency versions (NumPy, NetworkX, SciPy)
- Operating system (Linux, macOS, Windows)
- Hardware details for performance-critical analyses

17.6.2 Version Control for Code

Use Git to track code changes:

```
# Initialize repository
git init
git add analysis.py requirements.txt
git commit -m "Initial analysis code"

# Tag important versions
git tag -a v1.0 -m "Version used for paper submission"
```

Best practices:

- Commit `requirements.txt` and environment configs
- Use meaningful commit messages
- Tag versions used for publications
- Don't commit large datasets (use Git LFS or external storage)

17.6.3 Version Control for Data

For input datasets:

- **Small datasets (<100 MB):** Commit directly to Git
- **Medium datasets (100 MB - 1 GB):** Use Git LFS
- **Large datasets (>1 GB):** Store externally (Zenodo, OSF, Dataverse) and include download script

Example download script:

```
# download_data.py
import urllib.request

# Dataset DOI and URL
DOI = "10.5281/zenodo.1234567"
URL = "https://zenodo.org/record/1234567/files/network.edgelist"

print(f"Downloading dataset from {DOI}")
urllib.request.urlretrieve(URL, "data/network.edgelist")
print("Download complete")
```

17.7 Summary

Essential practices for reproducible py3plex analyses:

1. **Use virtual environments** (venv or conda) to isolate dependencies
2. **Pin exact versions** with `pip freeze > requirements.txt`
3. **Set random seeds** for all stochastic processes
4. **Record environment details** in papers and code comments
5. **Use version control** (Git) for code, Git LFS or external storage for data
6. **Consider Docker** for maximum reproducibility and deployment

For production deployments:

- Use Docker containers for complete environment isolation
- See Appendix B for Dockerfile templates and docker-compose configurations
- See Appendix A for repository structure best practices

Next chapter: Overview of the py3plex GUI for local exploratory workflows

CHAPTER

EIGHTEEN

THE PY3PLEX GUI (OVERVIEW)

The py3plex GUI is a web-based interface for interactive multilayer network exploration. It provides point-and-click access to py3plex features without requiring programming. **For detailed deployment configurations, see Appendix B.**

Status: Experimental

The py3plex GUI is **experimental** and designed for **local use only**. It is not hardened for public internet deployment. Use it for exploration, demos, and teaching—not as a primary production analysis interface.

18.1 What is the Py3plex GUI?

The GUI is a FastAPI + SvelteKit web application that provides:

- **Network upload** — Load networks from files (edgelist, GraphML, JSON)
- **Interactive visualization** — D3.js-powered network rendering
- **Query interface** — Execute DSL queries via web forms
- **Analysis workflows** — Common operations (centrality, communities) without coding
- **Export results** — Download analysis outputs as CSV/JSON

Target users: Researchers learning py3plex, educators demonstrating concepts, non-programmers exploring networks.

18.2 Running Locally

18.2.1 Installation

Install py3plex with GUI dependencies:

```
# Install with GUI extras
pip install 'py3plex[gui]'

# Or from source
cd py3plex
pip install -e '.[gui]'
```

18.2.2 Manual Development Setup

For development without Docker:

```
# Terminal 1: Start backend
cd gui/api
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000

# Terminal 2: Start frontend
cd gui/frontend
npm install
npm run dev

# Open browser to http://localhost:5173
```

Note: Manual setup requires Redis running separately for background tasks.

18.2.3 Configuration

Basic configuration in `gui/api/config.py`:

```
# Development settings
DEBUG = True
HOST = '0.0.0.0'
PORT = 8000

# File handling
UPLOAD_FOLDER = './uploads'
MAX_UPLOAD_SIZE = 100 * 1024 * 1024  # 100 MB

# Analysis limits
MAX_NODES = 10000  # Prevent performance issues
```

18.3 Key Workflows

18.3.1 Upload and Explore

Basic workflow:

1. **Upload network** — Click "Upload Network", select file (edgelist, GraphML, etc.)
2. **View statistics** — See node count, edge count, layer info
3. **Visualize** — Interactive network rendering with zoom/pan
4. **Layer selection** — Toggle layers on/off for focused viewing
5. **Export** — Download visualization as PNG or SVG

18.3.2 Query Interface

Execute DSL queries without writing code:

1. Navigate to "Query" tab
2. Build query using form:

- Select target: nodes or edges
- Choose layers
- Add filters (degree, centrality, etc.)
- Select measures to compute
- Set ordering and limits

3. Click "Execute"
4. View results in table
5. Export as CSV or JSON

Example: "Find top 20 nodes by betweenness centrality in the social layer"

18.3.3 Analysis Pipelines

Pre-built workflows for common analyses:

- **Community Detection** — Run Louvain or Infomap, visualize communities
- **Centrality Analysis** — Compute degree, betweenness, PageRank
- **Layer Comparison** — Compare statistics across layers
- **Ego Networks** — Extract neighborhoods around selected nodes

Each pipeline provides a guided interface with sensible defaults.

18.4 Limitations and Safety

18.4.1 Security Considerations

WARNING: The GUI is **NOT secure for public deployment**.

- **No authentication** — Anyone with URL access can use it
- **No rate limiting** — Vulnerable to resource exhaustion
- **No input sanitization** — File uploads not sandboxed
- **No HTTPS** — Data transmitted in plaintext

Safe use cases:

- Local machine only (`localhost`)
- Trusted network (lab/office LAN)
- Single-user exploration

Unsafe use cases:

- Public internet exposure
- Multi-tenant environments
- Sensitive data processing

For Docker, reverse proxy configuration, or controlled-network deployment guidance, see Appendix B.

18.4.2 Scalability Limits

Performance degrades with network size:

- **<1,000 nodes** — Full interactivity, all features work well
- **1,000-5,000 nodes** — Some slowness, simplify visualizations
- **5,000-10,000 nodes** — Limited features, use minimal layouts
- **>10,000 nodes — Use CLI/library instead**, GUI becomes unusable

Tips for large networks:

- Use layer filtering to reduce size
- Disable interactive visualization
- Export to CSV and analyze programmatically

18.5 When to Use the GUI vs CLI

Use GUI for:

- **Exploratory analysis** — Quick network inspection and hypothesis testing
- **Learning** — Understanding py3plex features interactively
- **Demonstration** — Presenting to non-programmers or stakeholders
- **Rapid prototyping** — Testing analysis ideas before writing code

Use CLI/library for:

- **Large networks** — >5,000 nodes
- **Production workflows** — Automated, reproducible pipelines
- **Complex analyses** — Chaining multiple operations
- **Performance-critical tasks** — Maximum speed and control
- **Reproducibility** — Version-controlled scripts

18.6 Closing Note

Use the GUI as a local exploratory front-end. For reproducible or large-scale workflows, prefer scripts, CLI, and pinned environments.

For deployment details and security hardening, see Appendix B. For GUI API reference, see: `docfiles/gui/gui_api_reference.rst`

CHAPTER

NINETEEN

APPENDIX A: REPOSITORY LAYOUT AND SCRIPTS

This appendix provides a reference for the py3plex repository structure and maps book examples to actual scripts.

19.1 Repository Structure

19.1.1 Top-Level Organization

```
py3plex/
├── py3plex/              # Main package
├── tests/                # Test suite
├── examples/             # Example scripts (see section below for current layout)
├── docfiles/             # Documentation source (Sphinx)
├── book/                 # This book's source files
├── gui/                  # Web GUI application
├── benchmarks/           # Performance benchmarks
├── datasets/             # Sample datasets
├── pyproject.toml        # Package configuration
├── Makefile              # Development automation
└── README.md             # Project overview
```

19.1.2 Main Package Structure

```
py3plex/
├── __init__.py
├── core/                 # Data structures
│   ├── multinet.py       # multi_layer_network class
│   └── random_generators.py
├── algorithms/           # Analysis algorithms
│   ├── statistics/
│   ├── centrality/
│   ├── community_detection/
│   └── paths/
├── dynamics/             # Dynamical processes
│   ├── models.py         # SIS, SIR, SEIR
│   ├── core.py
│   └── compartmental.py
├── dsl/                  # Query language
│   ├── __init__.py
│   ├── builder.py        # Builder API
│   ├── executor.py
```

```
│   └── export.py
├── visualization/        # Plotting
├── io/                   # I/O handlers
│   ├── readers.py
│   └── writers.py
├── cli.py                # Command-line interface
├── graph_ops.py          # Dplyr-style API
├── pipeline.py           # sklearn-style pipelines
└── workflows.py          # Config-driven workflows
```

19.2 Examples Directory Structure

The `examples/` directory currently contains topic-oriented subdirectories. The exact file count changes over time, so treat this appendix as a navigation map rather than a fixed inventory.

```
examples/
├── getting_started/
├── network_analysis/
├── dsl_zoo/
├── io_and_data/
├── visualization/
├── pipelines/
├── out_of_core/
└── advanced/
```

For current counts and file-level taxonomy, inspect the repository directly (for example: `find examples -maxdepth 2 -type d`).

19.3 Mapping Book Chapters to Examples

19.3.1 Installation and Getting Started (Installation and Getting Started)

- `examples/getting_started/` — Introductory quickstart scripts

19.3.2 Data Loading and Representation (Data Loading)

- `examples/io_and_data/` — Data loading and conversion workflows

19.3.3 Core Algorithms: Communities, Centrality, Dynamics (Core Algorithms)

- `examples/network_analysis/` — Core algorithm and analysis workflows

19.3.4 Introduction to the Py3plex DSL and Advanced Queries and Workflows (DSL)

- `examples/dsl_zoo/` — DSL-focused patterns and helper datasets

19.3.5 The Py3plex GUI (Overview) (GUI)

- `gui/app.py` — Main application
- `gui/README.md` — Setup instructions

19.4 Key Scripts Reference

19.4.1 Makefile Targets

The `Makefile` provides common development tasks:

```
make setup           # Create virtual environment
make dev-install     # Install with dev dependencies
make test            # Run test suite
make test-coverage   # Generate coverage report
make lint            # Run code quality checks
make format          # Format code with black
make docs            # Build documentation
make clean           # Clean build artifacts
```

19.4.2 Development Scripts

```
scripts/
├── run_tests.py              # Test runner
├── generate_docs.py          # Documentation generator
└── benchmark.py              # Performance benchmarking
```

19.5 Configuration Files

19.5.1 Package Configuration

pyproject.toml — Modern Python package configuration (PEP 518):

- Package metadata (name, version, authors)
- Dependencies and extras (`[infomap]`, `[viz]`, etc.)
- Build system configuration
- Tool configurations (black, ruff, mypy)

19.5.2 Testing Configuration

pytest.ini — Test framework configuration:

- Test discovery patterns
- Coverage settings
- Markers for test categories

19.5.3 Documentation Configuration

docfiles/conf.py — Sphinx documentation configuration:

- Extensions (autodoc, napoleon, mathjax)
- Theme settings (sphinx_rtd_theme)
- Build options

19.5.4 Docker Configuration

Dockerfile — Container image definition **docker-compose.yml** — Multi-service orchestration

[Detailed Docker configs in Appendix B]

19.6 Data Files

19.6.1 Sample Datasets

```
datasets/
├── karate_multiplex.edgelist
├── example_biological.graphml
└── README.md
```

19.6.2 External Datasets

The `multilayer_datasets/` directory contains links and scripts for downloading larger datasets used in case studies.

19.7 Summary

Key directories:

- `py3plex/` — Main package code
- `tests/` — Test suite
- `examples/` — Examples mapped to book chapters (counts evolve with releases)
- `docfiles/` — Documentation source
- `gui/` — Web interface

Development workflow:

1. Clone repository
2. `make setup` to create environment
3. `make dev-install` to install dependencies
4. Modify code, run `make test`
5. Use `examples/` as reference

Finding code:

- Algorithms → `py3plex/algorithms/`
- DSL → `py3plex/dsl/`
- Examples → `examples/` (organized by topic)

CHAPTER

TWENTY

APPENDIX B: DOCKER, DOCKER COMPOSE, AND DEPLOYMENT

This appendix provides Docker configurations and deployment instructions for py3plex, including controlled-network deployment notes for the GUI.

20.1 Dockerfile Reference

20.1.1 Main Dockerfile

The repository includes a Dockerfile for containerized execution:

```
# File: Dockerfile
FROM python:3.10-slim

WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    git \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

# Copy requirements
COPY pyproject.toml .
COPY README.md .

# Install py3plex
RUN pip install --no-cache-dir -e .

# Copy application code
COPY py3plex/ ./py3plex/
COPY examples/ ./examples/

# Set up entry point
ENTRYPOINT ["python", "-m", "py3plex"]
CMD ["--help"]
```

20.1.2 Building the Image

```
# Build image
docker build -t py3plex:latest .
```

```
# Build with specific Python version
docker build --build-arg PYTHON_VERSION=3.11 -t py3plex:3.11 .

# Build with version tag matching book release
docker build -t py3plex:1.1.4 .
```

20.1.3 Running Containers

```
# Run command
docker run --rm py3plex:latest --version

# Run analysis script with uv
docker run --rm py3plex:latest uv run examples/getting_started/01_basic_query.py

# Or using python directly
docker run --rm py3plex:latest python examples/getting_started/01_basic_query.py

# Mount data directory
docker run --rm -v $(pwd)/data:/data py3plex:latest python analysis.py

# Interactive shell
docker run --rm -it py3plex:latest /bin/bash
```

20.2 Docker Compose

Note

Docker Compose Command: This book uses the modern `docker compose` command (Docker CLI plugin, available since Docker 20.10). If you have an older Docker version, you may need to use the legacy `docker-compose` command instead. The configuration file is always named `docker-compose.yml` regardless of which command you use.

20.2.1 GUI Deployment (Experimental)

```
# File: docker-compose.yml
version: '3.8'

services:
  gui:
    build:
      context: .
      dockerfile: gui/Dockerfile
    ports:
      - "8000:8000"
    volumes:
      - ./data:/app/data
      - ./uploads:/app/uploads
    environment:
      - ENV=production
      - SECRET_KEY=${SECRET_KEY}
```

```
    restart: unless-stopped

  # Optional: nginx reverse proxy
  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
      - ./ssl:/etc/nginx/ssl:ro
    depends_on:
      - gui
    restart: unless-stopped
```

20.2.2 Running with Docker Compose

```
# Start services
docker compose up -d

# View logs
docker compose logs -f gui

# Stop services
docker compose down

# Rebuild after changes
docker compose up -d --build
```

20.3 Controlled Environment Deployment

Warning

The GUI is experimental and not designed for public internet deployment. The configurations below are for controlled, trusted environments (for example, an internal lab network or VPN). They reduce risk but do not convert the GUI into a public-facing hardened service.

20.3.1 Security Hardening (Trusted Networks Only)

1. Use environment variables for secrets:

```
# .env file (DO NOT commit to git)
SECRET_KEY=your-strong-secret-key-here
DATABASE_URL=postgresql://user:pass@host/db
```

```
# docker-compose.yml
services:
  gui:
    env_file:
      - .env
```

2. Run as non-root user:

```
# Add to Dockerfile
RUN useradd -m -u 1000 py3plex
USER py3plex
```

3. Use read-only file systems where possible:

```
services:
  gui:
    read_only: true
    tmpfs:
      - /tmp
      - /var/tmp
```

20.3.2 Nginx Reverse Proxy

nginx.conf:

```
events {
    worker_connections 1024;
}

http {
    upstream gui {
        server gui:8000;
    }

    server {
        listen 80;
        server_name yourdomain.com;

        # Redirect to HTTPS
        return 301 https://$server_name$request_uri;
    }

    server {
        listen 443 ssl;
        server_name yourdomain.com;

        ssl_certificate /etc/nginx/ssl/cert.pem;
        ssl_certificate_key /etc/nginx/ssl/key.pem;

        # Security headers
        add_header X-Frame-Options "SAMEORIGIN";
        add_header X-Content-Type-Options "nosniff";
        add_header X-XSS-Protection "1; mode=block";

        location / {
            proxy_pass http://gui;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
```

```
        }

        # File upload limits
        client_max_body_size 100M;
    }
}
```

20.3.3 TLS/SSL Configuration

Using Let's Encrypt:

```
# Install certbot
docker run --rm -v $(pwd)/ssl:/etc/letsencrypt \
    certbot/certbot certonly --standalone \
    -d yourdomain.com \
    --email your@email.com \
    --agree-tos
```

Self-signed certificates (development only):

```
# Generate self-signed certificate
openssl req -x509 -newkey rsa:4096 \
    -keyout ssl/key.pem \
    -out ssl/cert.pem \
    -days 365 -nodes
```

20.3.4 Authentication Setup

Basic HTTP authentication (simple):

```
server {
    ...

    location / {
        auth_basic "Restricted Access";
        auth_basic_user_file /etc/nginx/.htpasswd;

        proxy_pass http://gui;
    }
}
```

```
# Generate .htpasswd file
htpasswd -c .htpasswd username
```

OAuth2 Proxy (advanced):

Use `oauth2-proxy` as an additional service for integration with GitHub, Google, etc.

20.4 Security Checklist

20.4.1 Before Public Deployment

- [ ] Use HTTPS (TLS/SSL) for all connections

- [ ] Set strong `SECRET_KEY` environment variable
- [ ] Enable authentication (HTTP Basic, OAuth2, etc.)
- [ ] Run containers as non-root user
- [ ] Use read-only file systems where possible
- [ ] Set appropriate file upload limits
- [ ] Enable security headers in nginx
- [ ] Regularly update base images (`docker pull python:3.10-slim`)
- [ ] Monitor logs for suspicious activity
- [ ] Implement rate limiting (via nginx or application)
- [ ] Sanitize user inputs (especially file uploads)
- [ ] Restrict file types for uploads
- [ ] Scan uploaded files for malware
- [ ] Use network isolation (Docker networks)
- [ ] Keep dependencies up to date (`pip install --upgrade`)

20.4.2 Monitoring

```yaml
# Add health checks to docker-compose.yml
services:
  gui:
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
      interval: 30s
      timeout: 10s
      retries: 3
```

20.5 Cloud Deployment

20.5.1 AWS Deployment

Using ECS (Elastic Container Service):

1. Push image to ECR (Elastic Container Registry)
2. Create ECS task definition
3. Deploy to ECS cluster
4. Use ALB (Application Load Balancer) for routing

Using EC2:

1. Launch EC2 instance
2. Install Docker
3. Copy docker-compose.yml to instance
4. Run `docker compose up -d`

20.5.2 Azure Deployment

Using Azure Container Instances:

```
az container create \
    --resource-group myResourceGroup \
    --name py3plex-gui \
    --image py3plex:latest \
    --cpu 2 --memory 4 \
    --ports 8000
```

20.5.3 GCP Deployment

Using Cloud Run:

1. Push image to Google Container Registry
2. Deploy to Cloud Run
3. Configure domain and SSL

20.6 Performance Tuning

20.6.1 Container Resource Limits

```
services:
  gui:
    deploy:
      resources:
        limits:
          cpus: '2.0'
          memory: 4G
        reservations:
          cpus: '1.0'
          memory: 2G
```

20.6.2 Caching

Use Docker layer caching to speed up builds:

```
# Install dependencies first (cached layer)
COPY pyproject.toml .
RUN pip install -e .

# Copy code second (changes frequently)
COPY py3plex/ ./py3plex/
```

20.6.3 Multi-Stage Builds

For smaller production images:

```
# Build stage
FROM python:3.10 as builder
WORKDIR /app
```

```
COPY pyproject.toml .
RUN pip install --user -e .

# Runtime stage
FROM python:3.10-slim
WORKDIR /app
COPY --from=builder /root/.local /root/.local
COPY py3plex/ ./py3plex/
ENV PATH=/root/.local/bin:$PATH
```

20.7 Troubleshooting

20.7.1 Common Issues

Issue: Container exits immediately

Check logs:

```
docker logs <container_id>
```

Issue: Permission denied on mounted volumes

Fix permissions safely by matching container user to host user:

Method 1: Run container with your user ID (recommended)

```
docker run --user "$(id -u):$(id -g)" -v $(pwd)/data:/data py3plex-image
```

Method 2: Change ownership to match container user

```
# If container runs as user 1000:1000 (typical)
sudo chown -R 1000:1000 ./data
```

Method 3: Use docker compose with user mapping

```
services:
  py3plex:
    user: "${UID}:${GID}"
    volumes:
      - ./data:/data
```

Then run: `UID=$(id -u) GID=$(id -g) docker compose up`

> **Warning**
>
> **Avoid** `chmod -R 777 ./data` as it grants world-writable permissions and creates a security risk. Use user/group mapping instead.

Issue: Out of memory

Increase Docker memory limits or container resources.

20.8 Summary

This appendix provided:

- Complete Dockerfile and docker-compose.yml examples
- Security hardening guidelines
- Production deployment configurations (nginx, TLS, authentication)
- Cloud deployment options (AWS, Azure, GCP)
- Performance tuning strategies

Key recommendations for production:

1. Always use HTTPS
2. Enable authentication
3. Run as non-root
4. Monitor and log
5. Keep dependencies updated
6. Follow security checklist

See also: *The Py3plex GUI (Overview)* (page 133) for high-level GUI overview

CHAPTER

TWENTYONE

APPENDIX C: DETAILED VALIDATION SCRIPTS

This appendix provides detailed validation strategies and test scripts for ensuring correctness of multilayer network algorithms. These complement the high-level overview in *Testing and Validation* (page 124).

21.1 Validation Philosophy

py3plex uses multiple validation strategies:

1. **Unit tests** — Test individual functions in isolation
2. **Property-based tests** — Test invariants that should always hold
3. **Reference runs** — Compare against known-good results
4. **Conservation tests** — Verify physical/mathematical laws (e.g., probability conservation)

21.2 Random Walk Validation

21.2.1 Conservation Tests

Random walks must conserve probability—the sum of transition probabilities from any state must equal 1:

```
# File: tests/test_random_walk_conservation.py
import pytest
import numpy as np
from py3plex.algorithms.paths import random_walk

def test_probability_conservation():
    """Verify random walk conserves probability."""
    network = create_test_network()

    # Run random walk
    walks = random_walk(network, num_walks=1000, walk_length=100)

    # Compute transition matrix
    trans_matrix = compute_transition_matrix(walks)

    # Check row sums = 1
    row_sums = trans_matrix.sum(axis=1)
    assert np.allclose(row_sums, 1.0, atol=1e-6)
```

21.2.2 Bias Validation

Node2Vec introduces biases (return parameter p, in-out parameter q). Validate that biases have the intended effect:

```
# File: tests/test_node2vec_bias.py
def test_return_bias():
    """Higher p should reduce probability of returning to previous node."""
    network = create_test_network()

    # Low p (encourage return)
    walks_low_p = node2vec_walk(network, p=0.25, q=1.0, num_walks=1000)
    return_rate_low = count_immediate_returns(walks_low_p)

    # High p (discourage return)
    walks_high_p = node2vec_walk(network, p=4.0, q=1.0, num_walks=1000)
    return_rate_high = count_immediate_returns(walks_high_p)

    assert return_rate_high < return_rate_low
```

21.2.3 Reference Runs

Store reference outputs for deterministic tests:

```
# File: tests/test_dynamics_reference_runs.py
REFERENCE_DATA = {
    'sir_small_graph_42': {
        'beta': 0.3,
        'gamma': 0.1,
        'steps': 100,
        'seed': 42,
        'expected_final_recovered': 0.75
    }
}

def test_sir_matches_reference():
    """Ensure SIR dynamics match reference run."""
    ref = REFERENCE_DATA['sir_small_graph_42']

    network = make_tiny_chain_graph()
    sir = SIRDynamics(network, beta=ref['beta'], gamma=ref['gamma'])
    sir.set_seed(ref['seed'])

    results = sir.run(steps=ref['steps'])
    final_recovered = results.get_measure('state_counts')[2][-1] / network.number_of_nodes()

    assert abs(final_recovered - ref['expected_final_recovered']) < 0.01
```

21.3 Community Detection Validation

21.3.1 Modularity Bounds

Modularity Q must be in range [-0.5, 1.0]:

```python
# File: tests/test_community_bounds.py
def test_modularity_bounds():
    """Modularity must be in valid range."""
    network = create_test_network()

    communities = multilayer_louvain.best_partition(network.core_network)
    Q = compute_modularity(network, communities)

    assert -0.5 <= Q <= 1.0
```

21.3.2 Partition Quality

Every node must be assigned to exactly one community:

```python
def test_partition_completeness():
    """Every node must be in exactly one community."""
    network = create_test_network()
    communities = multilayer_louvain.best_partition(network.core_network)

    # All nodes accounted for
    assert set(communities.keys()) == set(network.get_nodes())

    # No overlapping assignments (in non-overlapping methods)
    assert len(communities) == len(set(communities.values()))
```

21.3.3 Singleton Detection

Validate that isolated nodes form their own communities:

```python
def test_singleton_communities():
    """Isolated nodes should form singleton communities."""
    network = create_network_with_isolates()
    communities = multilayer_louvain.best_partition(network.core_network)

    isolates = [n for n in network.get_nodes() if network.core_network.degree(n) == 0]

    # Each isolate in its own community
    for node in isolates:
        community = communities[node]
        same_community = [n for n, c in communities.items() if c == community]
        assert same_community == [node]
```

21.4 Centrality Validation

21.4.1 Degree Centrality

Degree centrality should match actual degree counts:

```python
# File: tests/test_centrality_correctness.py
def test_degree_centrality():
    """Degree centrality should match computed degrees."""
    network = create_test_network()
```

```
    import networkx as nx
    degree_cent = nx.degree_centrality(network.core_network)

    for node in network.get_nodes():
        computed = degree_cent[node]
        expected = network.core_network.degree(node) / (network.number_of_nodes() - 1)
        assert abs(computed - expected) < 1e-10
```

21.4.2 Betweenness Centrality

Validate using known structures (e.g., star graphs):

```
def test_betweenness_star_graph():
    """In a star graph, center has betweenness 1.0."""
    network = create_star_graph(n=5)  # 1 center + 4 leaves

    import networkx as nx
    bc = nx.betweenness_centrality(network.core_network, normalized=True)

    center = ('center', 'layer')
    assert bc[center] == 1.0  # All paths go through center

    leaves = [n for n in network.get_nodes() if n != center]
    for leaf in leaves:
        assert bc[leaf] == 0.0  # Leaves are not on any shortest paths
```

21.5 Dynamics Validation

21.5.1 SIS Model Conservation

In SIS model, S(t) + I(t) = N for all t:

```
# File: tests/test_dynamics_conservation.py
def test_sis_conservation():
    """SIS model must conserve population."""
    network = create_test_network()
    N = network.number_of_nodes()

    sis = SISDynamics(network, beta=0.3, gamma=0.1)
    sis.set_seed(42)

    results = sis.run(steps=100)

    for t in range(100):
        S_t = results.get_measure('state_counts')[0][t]
        I_t = results.get_measure('state_counts')[1][t]
        assert S_t + I_t == N
```

21.5.2 SIR Model Conservation

In SIR model, S(t) + I(t) + R(t) = N for all t:

```
def test_sir_conservation():
    """SIR model must conserve population."""
    network = create_test_network()
    N = network.number_of_nodes()

    sir = SIRDynamics(network, beta=0.3, gamma=0.1)
    sir.set_seed(42)

    results = sir.run(steps=100)

    for t in range(100):
        S_t = results.get_measure('state_counts')[0][t]
        I_t = results.get_measure('state_counts')[1][t]
        R_t = results.get_measure('state_counts')[2][t]
        assert S_t + I_t + R_t == N
```

21.5.3 Steady State Validation

SIS model should reach steady state:

```
def test_sis_steady_state():
    """SIS should reach equilibrium."""
    network = create_test_network()

    sis = SISDynamics(network, beta=0.3, gamma=0.1)
    sis.set_seed(42)

    results = sis.run(steps=1000)
    prevalence = results.get_measure('prevalence')

    # Check last 100 steps are stable
    last_100 = prevalence[-100:]
    variance = np.var(last_100)
    assert variance < 0.01  # Low variance = steady state
```

21.6 Property-Based Testing

21.6.1 Using Hypothesis

Property-based tests use the `hypothesis` library to generate random inputs:

```
# File: tests/property/test_paths_properties.py
from hypothesis import given, strategies as st
from hypothesis import assume

@given(
    num_nodes=st.integers(min_value=3, max_value=20),
    num_layers=st.integers(min_value=1, max_value=5),
    seed=st.integers(min_value=0, max_value=1000)
)
```

```
def test_shortest_path_length_property(num_nodes, num_layers, seed):
    """Shortest path length should be <= graph diameter."""
    network = generate_random_network(num_nodes, num_layers, seed)
    assume(nx.is_connected(network.core_network))  # Skip disconnected graphs

    # Pick random source and target
    nodes = list(network.get_nodes())
    source, target = random.sample(nodes, 2)

    # Compute shortest path
    path = nx.shortest_path(network.core_network, source, target)
    path_length = len(path) - 1

    # Path length should be <= diameter
    diameter = nx.diameter(network.core_network)
    assert path_length <= diameter
```

21.6.2 Multilayer-Specific Properties

```
@given(num_nodes=st.integers(min_value=2, max_value=10))
def test_node_activity_bounds(num_nodes):
    """Node activity must be in [0, 1]."""
    network = generate_multiplex_network(num_nodes)

    for node_id in range(num_nodes):
        activity = mls.node_activity(network, str(node_id))
        assert 0.0 <= activity <= 1.0
```

21.7 Running Validation Tests

21.7.1 Full Test Suite

```
# Run all tests
pytest tests/

# Run only validation tests
pytest tests/ -k validation

# Run with coverage
pytest tests/ --cov=py3plex --cov-report=html
```

21.7.2 Property-Based Tests

```
# Run property tests (slower)
pytest tests/property/

# Run with more examples
pytest tests/property/ --hypothesis-profile=thorough
```

21.7.3 Continuous Validation

Tests run automatically on:

- Every commit (GitHub Actions)
- Pull requests
- Scheduled nightly runs

21.8 Summary

This appendix detailed validation strategies:

1. **Conservation tests** — Physical/mathematical laws must hold
2. **Reference runs** — Deterministic tests against known-good outputs
3. **Boundary tests** — Values must be in valid ranges
4. **Property-based tests** — Invariants hold for random inputs
5. **Structure tests** — Algorithmic correctness on known structures

Key test files:

- `tests/test_dynamics_reference_runs.py` — Reference dynamics data
- `tests/test_random_walk_conservation.py` — Probability conservation
- `tests/property/` — Property-based tests
- `tests/test_centrality_correctness.py` — Centrality validation

See also: *Testing and Validation* (page 124) for high-level testing overview

CHAPTER

TWENTYTWO

APPENDIX D: ERROR HANDLING AND EXCEPTION HIERARCHY

This appendix documents py3plex's exception classes and error handling conventions.

22.1 Exception Hierarchy

py3plex defines a hierarchy of exceptions for different error categories:

```
Py3plexException (base)
├── ValidationError
│   ├── InvalidNetworkError
│   ├── InvalidLayerError
│   └── InvalidNodeError
├── IOError
│   ├── FileFormatError
│   ├── FileNotFoundError
│   └── SerializationError
├── DSLError
│   ├── QuerySyntaxError
│   ├── QueryExecutionError
│   └── UnsupportedFeatureError
├── AlgorithmError
│   ├── ConvergenceError
│   └── InsufficientDataError
└── ConfigurationError
```

22.2 Base Exception

```python
class Py3plexException(Exception):
    """Base exception for all py3plex errors."""
    pass
```

All py3plex exceptions inherit from this base class, allowing users to catch any library error:

```python
from py3plex.exceptions import Py3plexException

try:
    result = some_py3plex_operation()
except Py3plexException as e:
    print(f"py3plex error: {e}")
```

22.3 Validation Errors

22.3.1 InvalidNetworkError

Raised when a network is in an invalid state:

```
from py3plex.exceptions import InvalidNetworkError

# Example: Empty network
if network.number_of_nodes() == 0:
    raise InvalidNetworkError("Cannot compute metrics on empty network")
```

Common causes:

- Empty network (no nodes or edges)
- Disconnected graph when algorithm requires connected
- Directed graph when undirected expected (or vice versa)

22.3.2 InvalidLayerError

Raised when referencing a nonexistent layer:

```
from py3plex.exceptions import InvalidLayerError

if layer_name not in network.get_layers():
    raise InvalidLayerError(f"Layer '{layer_name}' does not exist")
```

Common causes:

- Typo in layer name
- Layer removed after reference created
- Case sensitivity mismatch

22.3.3 InvalidNodeError

Raised when referencing a nonexistent node:

```
from py3plex.exceptions import InvalidNodeError

if node not in network.core_network:
    raise InvalidNodeError(f"Node {node} not found in network")
```

22.4 I/O Errors

22.4.1 FileFormatError

Raised when file format is invalid or unsupported:

```
from py3plex.exceptions import FileFormatError

try:
    network.load_network("data.xyz", input_type="xyz")
```

```
except FileFormatError as e:
    print(f"Unsupported format: {e}")
```

Common causes:

- Unrecognized file extension
- Malformed file content
- Missing required fields in CSV/JSON

22.4.2 SerializationError

Raised during read/write operations:

```
from py3plex.exceptions import SerializationError

try:
    write(network, "output.arrow")
except SerializationError as e:
    print(f"Failed to serialize: {e}")
```

22.5 DSL Errors

22.5.1 QuerySyntaxError

Raised for invalid DSL query syntax:

```
from py3plex.dsl import execute_query
from py3plex.exceptions import QuerySyntaxError

try:
    result = execute_query(network, 'SELECT nodes WHRE degree > 5')  # Typo
except QuerySyntaxError as e:
    print(f"Query syntax error: {e}")
    # Suggestion: "Did you mean 'WHERE'?"
```

Common causes:

- Typos in keywords (WHRE instead of WHERE)
- Missing quotes around layer names
- Invalid comparison operators

22.5.2 QueryExecutionError

Raised when query execution fails:

```
from py3plex.exceptions import QueryExecutionError

try:
    result = execute_query(network, 'SELECT nodes COMPUTE nonexistent_measure')
except QueryExecutionError as e:
    print(f"Query execution failed: {e}")
```

Common causes:

- Referencing nonexistent measures
- Applying measure to unsuitable graph (e.g., eigenvector centrality on disconnected graph)
- Resource limits exceeded

22.5.3 UnsupportedFeatureError

Raised when using experimental or unimplemented features:

```
from py3plex.exceptions import UnsupportedFeatureError

try:
    result = Q.edges().where(weight__gt=0.5).execute(network)
except UnsupportedFeatureError as e:
    print(f"Feature not yet supported: {e}")
```

22.6 Algorithm Errors

22.6.1 ConvergenceError

Raised when iterative algorithms fail to converge:

```
from py3plex.exceptions import ConvergenceError

try:
    centrality = compute_eigenvector_centrality(network, max_iter=100)
except ConvergenceError as e:
    print(f"Algorithm did not converge: {e}")
```

22.6.2 InsufficientDataError

Raised when there's not enough data for analysis:

```
from py3plex.exceptions import InsufficientDataError

if network.number_of_nodes() < 3:
    raise InsufficientDataError("Need at least 3 nodes for community detection")
```

22.7 Configuration Errors

22.7.1 ConfigurationError

Raised for invalid configuration or parameters:

```
from py3plex.exceptions import ConfigurationError

if beta <= 0 or gamma <= 0:
    raise ConfigurationError("SIR parameters beta and gamma must be positive")
```

22.8 Error Handling Best Practices

22.8.1 Catch Specific Exceptions

Prefer catching specific exceptions over generic ones:

```
# Good: Specific exception handling
from py3plex.exceptions import InvalidLayerError

try:
    result = Q.nodes().from_layers(L["social"]).execute(network)
except InvalidLayerError:
    print("Layer 'social' not found. Available layers:", network.get_layers())

# Avoid: Generic exception (too broad)
try:
    result = Q.nodes().from_layers(L["social"]).execute(network)
except Exception as e:  # Too generic
    print(f"Something went wrong: {e}")
```

22.8.2 Provide Context

Include helpful context in error messages:

```
# Good: Helpful context
if layer_name not in network.get_layers():
    available = ", ".join(network.get_layers())
    raise InvalidLayerError(
        f"Layer '{layer_name}' not found. Available layers: {available}"
    )

# Less helpful
if layer_name not in network.get_layers():
    raise InvalidLayerError(f"Invalid layer")
```

22.8.3 Validation at Entry Points

Validate inputs early:

```
def compute_centrality(network, measure='degree'):
    """Compute node centrality."""
    # Validate early
    if network.number_of_nodes() == 0:
        raise InvalidNetworkError("Cannot compute centrality on empty network")

    if measure not in ['degree', 'betweenness', 'closeness']:
        raise ValueError(f"Unknown measure: {measure}")

    # Proceed with computation
    ...
```

22.8.4 Clean Resource Cleanup

Use context managers for resources:

```
# Good: Automatic cleanup
from py3plex.io import read

try:
    graph = read('network.arrow')
    # Process graph
except IOError as e:
    print(f"Failed to read file: {e}")
finally:
    # Resources automatically cleaned up
```

22.9 Error Messages Guidelines

1. **Be specific:** State what went wrong and why
2. **Suggest fixes:** Offer alternatives when possible
3. **Include context:** Show relevant values (layer names, node counts, etc.)
4. **Be concise:** One sentence explanation, then details if needed

Examples:

```
# Good error message
raise InvalidLayerError(
    f"Layer 'socail' not found. Did you mean 'social'? "
    f"Available layers: {', '.join(network.get_layers())}"
)

# Good error message
raise InsufficientDataError(
    f"Community detection requires at least 3 nodes, but network has only "
    f"{network.number_of_nodes()}"
)

# Less helpful
raise InvalidLayerError("Layer error")  # Too vague
```

22.10 Logging

py3plex uses Python's logging module:

```
import logging

# Configure logging level
logging.basicConfig(level=logging.INFO)

# py3plex will log warnings and errors
network.load_network("data.edgelist")  # Logs any warnings
```

Available log levels:

- `DEBUG` — Detailed information for debugging
- `INFO` — General information
- `WARNING` — Recoverable issues
- `ERROR` — Serious problems
- `CRITICAL` — Fatal errors

22.11 Summary

Exception hierarchy:

- Base: `Py3plexException`
- Categories: Validation, I/O, DSL, Algorithm, Configuration

Best practices:

1. Catch specific exceptions
2. Provide helpful context in messages
3. Validate early
4. Suggest fixes when possible
5. Use logging for non-fatal issues

Example usage:

```
from py3plex.core import multinet
from py3plex.exceptions import InvalidLayerError, Py3plexException

try:
    network = multinet.multi_layer_network()
    network.load_network("data.edgelist")

    result = Q.nodes().from_layers(L["social"]).execute(network)

except InvalidLayerError as e:
    print(f"Layer not found: {e}")
except Py3plexException as e:
    print(f"py3plex error: {e}")
```

See also: *Advanced Queries and Workflows* (page 83) for error recovery strategies in workflows

22.12 Pedagogical Error Messages (DSL v2)

Py3plex DSL v2 implements Rust-style pedagogical error messages that help users understand and fix issues quickly. Errors include:

1. **What the user likely intended**
2. **Why the operation failed**
3. **1-2 corrected query examples**
4. **Common pitfall notes (when applicable)**

22.12.1 Error Philosophy

Guidance over Obscurity:

Traditional error messages say "what went wrong". Py3plex errors explain "what to do about it".

```
# Traditional error (not helpful)
Error: Invalid syntax

# Pedagogical error (helpful)
DslSyntaxError: Cannot call .where() after .end_grouping()

 You probably wanted to: Filter nodes before grouping

 Why this failed: After .end_grouping(), the query context
   returns to ungrouped state. The .where() clause filters the
   entire result, not individual groups.

 Corrected examples:
  1. Q.nodes().where(degree__gt=3).per_layer().top_k(5)
  2. Q.nodes().per_layer().filter_within_groups(degree__gt=3)

  Common pitfall: Grouping operations (.per_layer(), .group_by())
   create a new context. Apply filters before grouping for clarity.
```

Core Principles:

- **Actionable**: Always suggest a fix, never just complain
- **Contextual**: Explain *why* in terms of DSL/multilayer semantics
- **Educational**: Help users understand the underlying concepts
- **Concise**: Keep examples short and focused

22.12.2 Enhanced DSL Error Classes

DslSyntaxError — Enhanced with pedagogical context

Raised for invalid DSL syntax or query structure:

```
from py3plex.dsl.errors import DslSyntaxError

# Example: Invalid method chaining
try:
    Q.nodes().execute(net).where(degree__gt=3)  # Execute too early
except DslSyntaxError as e:
    print(e)
# Output:
# DslSyntaxError: Cannot call .where() on QueryResult
#
#  You probably wanted to: Filter before executing
#
#  Why this failed: .execute() returns a QueryResult object,
#    which is immutable. Query building methods like .where()
#    must be called before .execute().
#
#  Corrected examples:
```

```
#   1. Q.nodes().where(degree__gt=3).execute(net)
#   2. result = Q.nodes().execute(net); df = result.to_pandas().query("degree > 3")
```

DslExecutionError — Enhanced with runtime context

Raised when query execution fails due to runtime conditions:

```
from py3plex.dsl.errors import DslExecutionError

# Example: Missing required metric
try:
    Q.nodes().where(betweenness__gt=0.1).execute(net)  # Betweenness not computed
except DslExecutionError as e:
    print(e)
# Output:
# DslExecutionError: Cannot filter on 'betweenness' (not computed)
#
#  You probably wanted to: Compute betweenness before filtering
#
#  Why this failed: The .where() clause references 'betweenness',
#    but this metric hasn't been computed yet. Autocompute is
#    disabled for filters to prevent silent expensive operations.
#
#  Corrected examples:
#   1. Q.nodes().compute("betweenness_centrality").where(betweenness__gt=0.1)
#   2. Q.nodes().where(degree__gt=5).compute("betweenness_centrality")
#
#   Common pitfall: Some metrics (degree) are cheap and autocomputed.
#    Expensive metrics (betweenness, closeness) require explicit .compute()
```

MultilayerSemanticError — Multilayer-specific guidance

Raised for common multilayer network semantic issues:

```
from py3plex.dsl.errors import MultilayerSemanticError

# Example: Node replica confusion
try:
    # User expects 100 unique nodes, gets 300 node replicas
    result = Q.nodes().execute(multilayer_net)
    assert result.count == 100  # Fails if network has 3 layers
except AssertionError:
    raise MultilayerSemanticError(
        "Node count mismatch: Expected 100, got 300",
        semantic_issue="Counting node replicas instead of physical nodes",
        multilayer_context=(
            "In multilayer networks, each physical node appears in multiple "
            "layers as a 'node replica'. Q.nodes() returns ALL replicas, "
            "not unique physical nodes."
        ),
        examples=[
            "To count physical nodes: len(set(n[0] for n in result.items))",
            "To work per-layer: Q.nodes().per_layer().execute(net)",
            "To get single layer: Q.nodes().from_layers(L['social']).execute(net)"
        ]
```

```
    )
```

22.12.3 Example: Before and After

Before (traditional error):

```
AttributeError: 'QueryResult' object has no attribute 'coverage'

# User is confused: What's coverage? Where do I use it?
```

After (pedagogical error):

```
DslExecutionError: .coverage() requires active grouping

 You probably wanted to: Apply coverage after .per_layer()

 Why this failed: The .coverage() method filters items based
   on their presence across groups. You must create groups first
   with .per_layer() or .group_by().

 Corrected examples:
  1. Q.nodes().per_layer().top_k(5, "degree").end_grouping().coverage(mode="all")
  2. Q.nodes().from_layers(L["*"]).per_layer().compute("degree").coverage(mode="any")

  Common pitfall: .coverage() is a post-grouping filter, not a
   pre-grouping filter. It answers "which items appear across groups?"
```

22.12.4 Using Pedagogical Errors Effectively

As a user:

1. **Read the entire error** — Don't stop at the first line
2. **Try the corrected examples** — They're tested and known to work
3. **Learn the pitfall** — Understand the underlying concept
4. **Check documentation** — Errors reference relevant sections

As a developer:

When raising DSL errors, provide rich context:

```
from py3plex.dsl.errors import DslExecutionError

# Good: Rich pedagogical error
raise DslExecutionError(
    f"Cannot filter on '{field}' (not computed)",
    intent=f"Compute {field} before filtering",
    why_failed=(
        f"The .where() clause references '{field}', but this metric "
        f"hasn't been computed yet. Autocompute is disabled for filters."
    ),
    examples=[
        f"Q.nodes().compute('{field}').where({field}__gt=threshold)",
        f"Q.nodes().where(degree__gt=5).compute('{field}')"  # Filter first
```

```
    ],
    pitfall=(
        "Some metrics (degree) are cheap and autocomputed. "
        "Expensive metrics require explicit .compute()"
    )
)
```

22.12.5 Error Recovery Patterns

Pattern 1: Inspect-Fix-Retry

```
try:
    result = Q.nodes().where(betweenness__gt=0.1).execute(net)
except DslExecutionError as e:
    print(e)  # Read pedagogical message

    # Apply suggested fix
    result = (
        Q.nodes()
         .compute("betweenness_centrality")
         .where(betweenness__gt=0.1)
         .execute(net)
    )
```

Pattern 2: Catch-and-Adapt

```
from py3plex.dsl.errors import MultilayerSemanticError

try:
    # Assume single-layer semantics
    result = Q.nodes().compute("degree").execute(net)
    avg_degree = sum(result.attributes["degree"].values()) / result.count
except (KeyError, MultilayerSemanticError):
    # Adapt to multilayer semantics
    result = (
        Q.nodes()
         .per_layer()
         .compute("degree")
         .aggregate("mean")  # Per-layer averages
         .execute(net)
    )
```

Pattern 3: Defensive Query Building

```
# Check capabilities before building complex queries
if len(net.get_layers()) > 1:
    # Multilayer network - use layer-aware query
    result = (
        Q.nodes()
         .per_layer()
         .compute("degree")
         .top_k(10, "degree")
         .end_grouping()
         .coverage(mode="any")  # Nodes in any layer
```

```
        .execute(net)
    )
else:
    # Single-layer network - use simple query
    result = (
        Q.nodes()
         .compute("degree")
         .order_by("-degree")
         .limit(10)
         .execute(net)
    )
```

22.12.6 Summary: Error Handling Philosophy

Traditional approach:

- Error: "What went wrong"
- User: Googles error message
- Result: Frustrated user, support ticket

Py3plex pedagogical approach:

- Error: "What went wrong" + "Why" + "How to fix" + "Learn more"
- User: Reads error, applies fix
- Result: Self-sufficient user, learned a concept

Key benefits:

1. **Faster debugging**: Users fix issues without leaving their editor
2. **Better learning**: Errors teach multilayer network concepts
3. **Reduced support**: Common mistakes are explained inline
4. **Higher quality code**: Users learn best practices from error messages

See also:

- DSL Builder API (Chapter 9) for query construction patterns
- Advanced Queries (Chapter 10) for error recovery in workflows
- Multilayer Basics (Chapter 2) for core multilayer network concepts

CHAPTER

TWENTYTHREE

APPENDIX E: EXTENDED API AND DSL REFERENCE

This appendix provides a quick reference for the py3plex API and DSL syntax.

23.1 Core API Reference

23.1.1 Creating Networks

```
from py3plex.core import multinet

# Create empty network
network = multinet.multi_layer_network(
    directed=False,           # True for directed networks
    network_type='multilayer' # or 'multiplex'
)
```

23.1.2 Adding Nodes and Edges

```
# Add edges (nodes created implicitly)
network.add_edges([
    ['Alice', 'friends', 'Bob', 'friends', 1.0],
    ['Bob', 'friends', 'Carol', 'friends', 1.0],
], input_type="list")

# Add edges with attributes
network.add_edges([{
    'source': 'Alice',
    'source_type': 'friends',
    'target': 'Bob',
    'target_type': 'friends',
    'weight': 0.8,
    'timestamp': '2023-01-15'
}])
```

23.1.3 Loading Networks

```
# From file
network.load_network("data.edgelist", input_type="edgelist")

# From NetworkX
import networkx as nx
```

```
G = nx.karate_club_graph()
network.load_network_from_networkx(G, layer_name='social')
```

23.1.4 Basic Operations

```
# Get network information
layers = network.get_layers()
nodes = list(network.get_nodes())
edges = list(network.get_edges())

# Statistics
network.basic_stats()
num_nodes = network.get_number_of_nodes()
num_edges = network.get_number_of_edges()

# Layer-specific
nodes_per_layer = network.get_number_of_nodes_per_layer()
layer_subgraph = network.get_layer_subgraph('friends')
```

23.2 Algorithms API

23.2.1 Statistics

```
from py3plex.algorithms.statistics import multilayer_statistics as mls

# Layer density
density = mls.layer_density(network, 'friends')

# Node activity (fraction of layers a node appears in)
activity = mls.node_activity(network, 'Alice')

# Layer overlap
overlap = mls.layer_overlap(network, 'friends', 'colleagues')
```

23.2.2 Community Detection

```
from py3plex.algorithms.community_detection import multilayer_louvain

# Detect communities
communities = multilayer_louvain.best_partition(network.core_network)

# communities = {('Alice', 'friends'): 0, ('Bob', 'friends'): 0, ...}
```

23.2.3 Centrality

```
import networkx as nx

# Standard centralities (use NetworkX)
G = network.core_network
degree_cent = nx.degree_centrality(G)
```

```
betweenness = nx.betweenness_centrality(G)
closeness = nx.closeness_centrality(G)

# Explainable centrality (py3plex-specific)
from py3plex.algorithms.centrality.explain import explain_node_centrality

explanation = explain_node_centrality(
    network,
    node='Alice',
    measure='degree'
)
# Returns: {'score': 4, 'layer_breakdown': {...}, ...}
```

23.2.4 Dynamics

```
from py3plex.dynamics.models import SIRDynamics, SISDynamics

# Configure SIR model
sir = SIRDynamics(
    network,
    beta=0.3,      # Infection rate
    gamma=0.1,     # Recovery rate
    initial_infected={'Alice': 0}  # Initial conditions
)

# Set seed for reproducibility
sir.set_seed(42)

# Run simulation
results = sir.run(steps=100)

# Extract measures
prevalence = results.get_measure('prevalence')
recovered = results.get_measure('state_counts')[2]  # State 2 = recovered
```

23.3 DSL Quick Reference

23.3.1 Builder API (Recommended)

```
from py3plex.dsl import Q, L, Param

# Basic query
result = Q.nodes().execute(network)

# Filter by layer
result = Q.nodes().from_layers(L["social"]).execute(network)

# Filter by condition
result = Q.nodes().where(degree__gt=5).execute(network)

# Multiple conditions
```

```
result = Q.nodes().where(layer="social", degree__gt=3).execute(network)

# Layer algebra
result = Q.nodes().from_layers(L["social"] + L["work"]).execute(network)      # Union
result = Q.nodes().from_layers(L["social"] - L["bots"]).execute(network)     # Difference
result = Q.nodes().from_layers(L["social"] & L["work"]).execute(network)     # Intersection

# Compute measures
result = Q.nodes().compute("degree", "betweenness_centrality").execute(network)

# Order and limit
result = (
    Q.nodes()
     .compute("degree")
     .order_by("-degree")
     .limit(10)
     .execute(network)
)

# Export results
result = (
    Q.nodes()
     .compute("degree")
     .execute(network)
)
result.to_pandas().to_csv("output.csv", index=False)

# Parameterized queries
result = (
    Q.nodes()
     .where(degree__gt=Param('min_degree'))
     .execute(network, params={'min_degree': 5})
)
```

23.3.2 String DSL Syntax

```
from py3plex.dsl import execute_query

# Basic syntax: SELECT target WHERE conditions COMPUTE measures

# Select all nodes
execute_query(network, 'SELECT nodes')

# Filter by layer
execute_query(network, 'SELECT nodes WHERE layer="social"')

# Filter by degree
execute_query(network, 'SELECT nodes WHERE degree > 5')

# Multiple conditions
execute_query(network, 'SELECT nodes WHERE layer="social" AND degree > 3')

# Compute measures
```

```
execute_query(network, 'SELECT nodes COMPUTE betweenness_centrality')

# Combined
execute_query(network,
    'SELECT nodes WHERE layer="social" AND degree > 3 '
    'COMPUTE degree betweenness_centrality'
)
```

23.3.3 DSL Operators

Comparison operators:

- = — Equal to
- != — Not equal to
- > — Greater than
- < — Less than
- >= — Greater than or equal
- <= — Less than or equal

Logical operators:

- `AND` — Logical AND
- `OR` — Logical OR
- `NOT` — Logical NOT

Filter modifiers (Builder API):

- `field=value` — Equal
- `field__ne=value` — Not equal
- `field__gt=value` — Greater than
- `field__gte=value` — Greater or equal
- `field__lt=value` — Less than
- `field__lte=value` — Less or equal

23.3.4 Available Measures

- `degree` — Node degree
- `degree_centrality` — Normalized degree
- `betweenness_centrality` — Betweenness
- `closeness_centrality` — Closeness
- `eigenvector_centrality` — Eigenvector
- `pagerank` — PageRank
- `clustering` — Clustering coefficient

23.3.5 Working with Results

```
result = Q.nodes().compute("degree").execute(network)

# Iteration
for node in result:
    print(node)

# Count
print(f"Found {result.count} nodes")

# Measures
degrees = result.measures['degree']
for node, degree in degrees.items():
    print(f"{node}: {degree}")

# To pandas
df = result.to_pandas()
```

23.4 I/O API

23.4.1 Modern I/O System

```
from py3plex.io import read, write, MultiLayerGraph

# Read (auto-detects format from extension)
graph = read('network.json')
graph = read('network.arrow')
graph = read('network.parquet')

# Write
write(network, 'output.json')
write(network, 'output.arrow')
write(network, 'output.parquet')

# Specify format explicitly
graph = read('file.dat', format='json')
write(network, 'file.dat', format='arrow')
```

23.4.2 Supported Formats

- **JSON** — `.json` — Human-readable
- **JSONL** — `.jsonl` — Streaming
- **CSV** — `.csv` — Spreadsheet-compatible
- **Arrow** — `.arrow` — High-performance
- **Parquet** — `.parquet` — Compressed

23.5 Visualization API

23.5.1 Basic Visualization

```
from py3plex.visualization.multilayer import draw_multilayer_default

# Simple visualization
draw_multilayer_default(network.get_layers(), display=True)

# With layout options
draw_multilayer_default(
    network.get_layers(),
    display=True,
    layout='force_directed',
    node_size=300,
    edge_width=2
)
```

23.5.2 Matrix Visualization

```
# Visualize supra-adjacency matrix
network.visualize_matrix({"display": True})
```

23.6 Command-Line Interface

23.6.1 Basic Usage

```
# Show version
python -m py3plex --version

# Run self-test
python -m py3plex selftest

# Analyze network
python -m py3plex analyze network.edgelist

# Community detection
python -m py3plex communities network.edgelist --algorithm louvain
```

23.7 Summary

This appendix provided quick references for:

- Core API (creating networks, adding nodes/edges)
- Algorithms API (statistics, communities, centrality, dynamics)
- DSL syntax (both Builder API and string syntax)
- I/O API (reading/writing various formats)
- Visualization API
- Command-line interface

For detailed documentation:

- Main text chapters for concepts and examples
- Appendix A for repository layout
- Appendix D for error handling
- Online API docs: https://skblaz.github.io/py3plex/

CHAPTER

TWENTYFOUR

BIBLIOGRAPHY

This section provides references for further reading on multilayer networks and py3plex.

24.1 How to Cite This Book

Use the following citation for this manuscript edition:

```
Škrlj, B. (2025). Practical Multilayer Network Analysis with Py3plex (Version 1.1.4).
https://github.com/SkBlaz/py3plex
```

BibTeX:

```
@book{Skrlj2025book,
  author = {Škrlj, Blaž},
  title = {Practical Multilayer Network Analysis with Py3plex},
  year = {2025},
  version = {1.1.4},
  url = {https://github.com/SkBlaz/py3plex}
}
```

24.2 Core References

24.3 Py3plex Publications

24.4 Community Detection

24.5 Network Embeddings

24.6 Dynamics and Processes

24.7 Applications

24.7.1 Biological Networks

24.7.2 Social Networks

24.7.3 Transportation Networks

24.8 Software and Tools

24.9 Online Resources

Py3plex:

- Documentation: https://skblaz.github.io/py3plex/
- GitHub: https://github.com/SkBlaz/py3plex
- PyPI: https://pypi.org/project/py3plex/

Community:

- Network Science community: https://netscisociety.net/
- Complex Networks: https://www.complexnetworks.org/

Datasets:

- Network Repository: https://networkrepository.com/
- SNAP: Stanford Network Analysis Project: https://snap.stanford.edu/data/
- Netzschleuder: https://networks.skewed.de/

24.10 Recommended Reading Path

24.10.1 For Graduate Students

1. Start with [Kivelä2014] for comprehensive foundations
2. Read [Skrlj2019a] for py3plex overview
3. Read [Mucha2010] for community detection
4. Explore [Bianconi2018] textbook for deeper theory

24.10.2 For Practitioners

1. Start with [Skrlj2019a] for py3plex capabilities
2. Skim [Kivelä2014] for key concepts
3. Focus on application papers relevant to your domain
4. Use this book and online docs for implementation

24.10.3 For Researchers

1. Read [Kivelä2014] and [Boccaletti2014] thoroughly
2. Study [Domenico2013] for mathematical rigor
3. Read domain-specific application papers
4. Explore [Bianconi2018] for advanced topics

BIBLIOGRAPHY

[Kivelä2014] Kivelä, M., Arenas, A., Barthelemy, M., Gleeson, J. P., Moreno, Y., & Porter, M. A. (2014). Multilayer networks. *Journal of Complex Networks*, 2(3), 203-271. https://doi.org/10.1093/comnet/cnu016

Comprehensive review of multilayer network theory, mathematics, and applications. Essential reading.

[Boccaletti2014] Boccaletti, S., Bianconi, G., Criado, R., Del Genio, C. I., Gómez-Gardenes, J., Romance, M., … & Zanin, M. (2014). The structure and dynamics of multilayer networks. *Physics Reports*, 544(1), 1-122. https://doi.org/10.1016/j.physrep.2014.07.001

Physics-focused perspective on multilayer networks with emphasis on dynamics.

[Bianconi2018] Bianconi, G. (2018). *Multilayer Networks: Structure and Function*. Oxford University Press.

Comprehensive textbook covering theory, methods, and applications.

[Domenico2013] De Domenico, M., Solé-Ribalta, A., Cozzo, E., Kivelä, M., Moreno, Y., Porter, M. A., … & Arenas, A. (2013). Mathematical formulation of multilayer networks. *Physical Review X*, 3(4), 041022. https://doi.org/10.1103/PhysRevX.3.041022

Rigorous mathematical formulation of multilayer network representations.

[Skrlj2019a] Škrlj, B., Kralj, J., & Lavrač, N. (2019). Py3plex toolkit for visualization and analysis of multilayer networks. *Applied Network Science*, 4(1), 94. https://doi.org/10.1007/s41109-019-0203-7

Primary py3plex paper. Cite this when using py3plex in research.

[Skrlj2019b] Škrlj, B., Kralj, J., & Lavrač, N. (2019). Py3plex: A library for scalable multilayer network analysis and visualization. In *International Conference on Complex Networks and Their Applications* (pp. 757-768). Springer. https://doi.org/10.1007/978-3-030-05411-3_60

Conference paper focusing on visualization and scalability.

[Blondel2008] Blondel, V. D., Guillaume, J. L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10), P10008. https://doi.org/10.1088/1742-5468/2008/10/P10008

Louvain algorithm for community detection.

[Rosvall2008] Rosvall, M., & Bergstrom, C. T. (2008). Maps of random walks on complex networks reveal community structure. *Proceedings of the National Academy of Sciences*, 105(4), 1118-1123. https://doi.org/10.1073/pnas.0706851105

Infomap algorithm for community detection.

[Mucha2010] Mucha, P. J., Richardson, T., Macon, K., Porter, M. A., & Onnela, J. P. (2010). Community structure in time-dependent, multiscale, and multiplex networks. *Science*, 328(5980), 876-878. https://doi.org/10.1126/science.1184819

Multilayer community detection with modularity optimization.

[Grover2016] Grover, A., & Leskovec, J. (2016). node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 855-864). https://doi.org/10.1145/2939672.2939754

Node2Vec algorithm for network embeddings.

[Perozzi2014] Perozzi, B., Al-Rfou, R., & Skiena, S. (2014). DeepWalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 701-710). https://doi.org/10.1145/2623330.2623732

DeepWalk algorithm for learning node representations.

[Pastor-Satorras2015] Pastor-Satorras, R., Castellano, C., Van Mieghem, P., & Vespignani, A. (2015). Epidemic processes in complex networks. *Reviews of Modern Physics*, 87(3), 925. https://doi.org/10.1103/RevModPhys.87.925

Comprehensive review of epidemic modeling on networks.

[Buldyrev2010] Buldyrev, S. V., Parshani, R., Paul, G., Stanley, H. E., & Havlin, S. (2010). Catastrophic cascade of failures in interdependent networks. *Nature*, 464(7291), 1025-1028. https://doi.org/10.1038/nature08932

Cascading failures in interdependent networks.

[Gligorijevic2019] Gligorijević, V., Barot, M., & Bonneau, R. (2019). deepNF: deep network fusion for protein function prediction. *Bioinformatics*, 35(5), 757-766. https://doi.org/10.1093/bioinformatics/bty440

Application to biological network analysis.

[Magnani2013] Magnani, M., & Rossi, L. (2013). Pareto distance for multilayer network analysis. In *Social Computing, Behavioral-Cultural Modeling and Prediction* (pp. 249-256). Springer. https://doi.org/10.1007/978-3-642-37210-0_27

Social network analysis with multilayer methods.

[Gallotti2014] Gallotti, R., & Barthelemy, M. (2014). Anatomy and efficiency of urban multimodal mobility. *Scientific Reports*, 4(1), 6911. https://doi.org/10.1038/srep06911

Transportation networks as multilayer systems.

[NetworkX] Hagberg, A., Swart, P., & Schult, D. A. (2008). Exploring network structure, dynamics, and function using NetworkX. *Los Alamos National Lab.(LANL), Los Alamos, NM (United States)*.

NetworkX documentation: https://networkx.org/

[Arrow] Apache Arrow Development Team. (2023). Apache Arrow. https://arrow.apache.org/

High-performance columnar data format.

INDEX